



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Integrated Autonomous Delivery System: Enhancing Vehicle-to- User Interaction and Dynamic Path Planning

TESI DI LAUREA MAGISTRALE IN
MECHANICAL ENGINEERING - INGEGNERIA MECCANICA

Author: **John Maroon RIZKALLAH**

Student ID: 214392

Advisor: Prof. Stefano ARRIGONI

Co-advisors: Satyesh Shanker AWASTHI

Academic Year: 2024-2025

Acknowledgments

I would like to thank Professor Stefano ARRIGONI, my thesis supervisor, for proposing this thesis topic and providing decisive measures to help throughout this research.

A special thanks to Satyesh Shanker AWASTHI for his continuous and sustained feedback and help throughout this thesis.

I would also like to extend my appreciation to Politecnico Di Milano for providing the necessary resources and facilities to conduct this research.

I am grateful for the connections I have made throughout this Master's journey and for the new friendships I have made along the way.

Finally, my heartfelt gratitude goes to my mother, father and sister for their unwavering love, patience, and encouragement, without which this journey would not have been possible.

Abstract

The rapid advancement of autonomous vehicle technologies has opened new possibilities in last-mile delivery solutions. Autonomous Delivery Vehicles (ADV) are emerging as a promising alternative to traditional courier services, offering improved efficiency, cost-effectiveness, and sustainability. This thesis presents the development and implementation of an autonomous delivery system using the EasyMile EZ10 vehicle, designed to facilitate package transportation across a controlled environment. The research focuses on three key aspects: navigation, user interaction, and behavioral planning.

A comprehensive navigation system was developed, integrating occupancy grid mapping, Rapidly-exploring Random Trees Star with KD-tree optimization (RRT*KD) for path planning, and Model Predictive Control (MPC) for motion execution.

For user interaction, a Vehicle-to-User (V2U) communication system was implemented, leveraging a QR code-based interface and integrated in a prototype composed by a Raspberry Pi-based module and a proximity sensor, to facilitate seamless package retrieval and delivery requests.

The behavioral planning module was designed using a Finite State Machine (FSM), implemented in Python, to regulate the vehicle's operational states and transitions.

Although full-scale real-world testing was limited due to unavailability of the vehicle, results were obtained from simulations and partial experiments to validate the feasibility of the proposed autonomous delivery system.

Keywords: Autonomous Delivery Vehicles, Path Planning, Model Predictive Control, Finite State Machine, Vehicle-to-User Interaction, EasyMile EZ10

Abstract in lingua italiana

Il rapido progresso delle tecnologie dei veicoli autonomi ha aperto nuove possibilità nelle soluzioni di consegna dell'ultimo miglio. I veicoli autonomi per le consegne (ADV) stanno emergendo come una promettente alternativa ai servizi di corriere tradizionali, offrendo una maggiore efficienza, economicità e sostenibilità. Questa tesi presenta lo sviluppo e l'implementazione di un sistema di consegna autonomo che utilizza il veicolo EasyMile EZ10, progettato per facilitare il trasporto di pacchi in un ambiente controllato. La ricerca si concentra su tre aspetti chiave: navigazione, interazione con l'utente e pianificazione comportamentale.

È stato sviluppato un sistema di navigazione completo, che integra la mappatura della griglia di occupazione, il Rapidly-exploring Random Trees Star con KD-tree optimization (RRT*KD) per la pianificazione del percorso e il Model Predictive Control (MPC) per l'esecuzione del movimento.

Per l'interazione con l'utente, è stato implementato un sistema di comunicazione Vehicle-to-User (V2U), sfruttando un'interfaccia basata su un codice QR e integrata in un prototipo composto da un modulo basato su Raspberry Pi e un sensore di prossimità, per facilitare il recupero dei pacchi e le richieste di consegna senza soluzione di continuità. Il modulo di pianificazione comportamentale è stato progettato utilizzando una macchina a stati finiti (FSM), implementata in Python, per regolare gli stati operativi e le transizioni del veicolo.

Sebbene i test su scala reale siano stati limitati a causa dell'indisponibilità del veicolo, sono stati ottenuti risultati da simulazioni ed esperimenti parziali per convalidare la fattibilità del sistema di consegna autonomo proposto.

Il modulo di pianificazione comportamentale è stato progettato utilizzando una macchina a stati finiti (FSM), implementata in Python, per regolare gli stati operativi e le transizioni del veicolo.

Sebbene i test in scala reale siano stati limitati a causa di vincoli meccanici, sono stati ottenuti risultati da simulazioni ed esperimenti parziali per convalidare la fattibilità del sistema di consegna autonomo proposto.

Parole chiave: Veicoli autonomi per le consegne, pianificazione del percorso, con-

trollo predittivo del modello, macchina a stati finiti, interazione veicolo-utente, EasyMile
EZ10

Contents

Acknowledgments	i
Abstract	iii
Abstract in lingua italiana	v
Contents	vii
Introduction	1
1 State of the Art	5
1.1 Autonomous Delivery Vehicles	6
1.1.1 Technology	7
1.1.2 Important Implementations of Autonomous Delivery Vehicles	8
1.2 Navigation and Motion Strategies	10
1.2.1 Mapping	11
1.2.2 Path planning	12
1.2.3 Motion control	13
1.3 Human-Machine Interface	14
1.3.1 QR Codes Integration	15
1.4 Behavioral Planning	16
1.4.1 Behavioral Planning in Autonomous Vehicles	16
2 Navigation	19
2.1 Overview of the EasyMile EZ10	19
2.2 Mapping	20
2.2.1 Gmapping for Occupancy Grid Mapping	21
2.3 Path Planning	28
2.3.1 Overview of the RRT	28
2.3.2 The RRT*	30

2.3.3	KD-tree and Nearest Neighbor Searching	31
2.3.4	Comparison of the RRTs	32
2.4	Motion control	37
2.4.1	Mathematical Model	38
2.4.2	Results	41
3	Vehicle-to-User Interaction	45
3.1	Hardware Design and Requirements	45
3.2	Software Design and Requirements	52
3.3	User Interface	54
4	Behavioral Planning	57
4.1	The Finite State Machine	57
4.2	A ROS Finite State Machine	58
5	Results	63
5.1	Simulation Settings	63
5.2	Accidental Triggering of Proximity Sensor	66
5.3	One Goal Delivery With Long Loading Time	68
5.4	Two Goals at Once	71
5.5	Adding More Goals at a Delivery Point and Retrieving Packages	74
5.6	Entire Loop Trajectory Comparison	78
	Conclusion	83
	List of Figures	85
	List of Tables	87
	Bibliography	89
	Appendix A	95

Introduction

The development of autonomous delivery vehicles (ADV) has gained significant momentum in recent years due to the increasing demand for efficient, cost-effective, and sustainable logistics solutions. Road transport in general is a major contributor to greenhouse gas emissions, which can lead to climate change. Therefore, the European Commission has set emission targets to reach net-zero greenhouse gas emissions concerning all sectors by 2050 [1]. The rise of e-commerce, coupled with the need for faster and more reliable delivery services, has led to a growing interest in the deployment of autonomous systems that can navigate urban environments, interact with users, and optimize transportation routes without human intervention. Applications of ADVs using QR code as a mean of communication with the end-user already exist, such as Ottonomy's Ottobots [2]. However there is very little literature about implementing an entire Vehicle-to-User (V2U) that uses QR code technology. That is, a V2U system capable of taking also delivery orders from the end-user.

This would allow the ADV to be initialized with a specific delivery route from the delivery entity and also evolve dynamically by accepting packages from user to be delivered via the ADV (and thus the delivery company) to other end-users. This concept would take full advantage of the service provided by the ADV and avoid unnecessary waiting time in a physical office to process delivering packages. Some research such as the article [3] model the concept without an actual implementation. It is therefore important to create a basis for such a system that has a great potential of revolutionizing the user interaction and satisfaction regarding package delivery.

This thesis focuses on contributing to the field of autonomous delivery by implementing an autonomous delivery system using the EasyMile EZ10 vehicle within the LaMasa Campus of Politecnico di Milano.

Thesis Objectives

The primary goal of this research is to establish a framework for future autonomous delivery vehicles capable of handling end-user deliveries and addressing key challenges

such as navigation, user interaction, and behavioral planning. The EZ10 autonomous shuttle serves as the core vehicle for experimentation, aiming to transport heavy or large items between different buildings on campus, allowing for a continuous uninterrupted delivery loop between users.

To achieve this objective, the research methodology follows a structured approach that involves reviewing the state of the art in autonomous delivery vehicle technologies, integrating advanced navigation and motion control strategies, and developing an interactive Vehicle-to-User (V2U) communication system as well as an initial prototypical design of it. The work is structured into three main tasks:

Navigation Management: The thesis integrates mapping, path planning, and a validated motion control system to enable the EZ10 vehicle to navigate efficiently. By employing occupancy grid mapping, Rapidly-exploring Random Trees (RRT) path planning, and Model Predictive Control (MPC) for trajectory execution, the navigation system ensures accurate and reliable movement of the vehicle.

Vehicle-to-User Interaction System: A critical aspect of autonomous delivery is the ability of the vehicle to interact with users. This work develops a V2U system using a combination of QR codes and a Human-Machine Interface (HMI) to facilitate seamless communication between the user and the EZ10 vehicle. Users can request deliveries, retrieve packages, and control vehicle functions in a user-friendly manner.

Behavioral Planning: The thesis implements a Finite State Machine (FSM) to regulate the different states of the vehicle and their transitions. This ensures that the EZ10 can effectively manage delivery requests, optimize goal sequences, and adapt to unexpected changes in the environment.

The proposed system is designed to be tested on the LaMasa Campus, where the entire architecture—including navigation, V2U interaction, and behavioral planning—will be validated through simulations and real-world experiments.

In particular, in the *first chapter* the state of the art of the autonomous delivery vehicles, their navigation module, user interface and behavior planning is discussed and the choice for a method in each subsections is made.

The *second chapter* introduces the EZ10 and its relevant features. Then, the Gmapping method is explained and developed to reach acceptable results. Moreover, the RRT path planning method is explained with results of its RRT* variations as well as post process smoothing. Furthermore, the motion controller is then presented and referenced with simulation results over the trajectories obtained from the path planning.

The *third chapter* presents and justifies the selected design for the Vehicle-to-User system (sensors and architecture), as well as the proposed web interface and QR code input method chosen.

The *fourth chapter* reviews the behavioral method chosen of the Finite State Machine. It also illustrates it by citing the different states of the vehicle and explaining the actions contained within them as well as the necessary arguments to transition from one state to another.

The *last chapter* Finally, the last chapter presents experimental results while evaluating system performance.

By addressing critical challenges in autonomous delivery vehicle technology, this thesis aims to contribute to the advancement of smart transportation solutions and pave the way for the widespread adoption of ADVs in urban logistics.

1 | State of the Art

The global shift toward automation and intelligent transportation systems has allowed for significant advancements in autonomous vehicle (AV) technology. This shift aligns with the growing demand for efficiency, sustainability, and innovation in the delivery and logistics sector. Moreover, the rapid growth of e-commerce and urbanization emphasizes the need to develop solutions that are not only efficient but also adaptable to congested urban environments. Autonomous delivery vehicles offer scalable solutions that optimize delivery routes, reduce operational costs, and minimize environmental impact.

Autonomous vehicles are equipped with a vast array of sensor that allows vehicles to measure their localization and perceive their environment. To determine what to do, the AV path planner receives the output of localization and environment perception. The path planner exploits these outputs and generates a reference path or trajectory to be followed by the AV. This trajectory is then sent to the AV motion control, which tracks and stays on the path using feedback control techniques by providing the actuators (steering, brakes, and accelerator) with the best control inputs [4].

To create a path, both a starting and an ending position are required. These can be determined automatically or be inputted manually. This is where the interaction between the user and the vehicle is important. In fact, for a delivery vehicle it is of the utmost importance to have some sort of interface that allows not only the user to dispatch a delivery but also for the one receiving such delivery to be able to access their package. These interface are commonly called Human Machine Interfaces (HMIs) and allow a communication between the user and, in this case, the autonomous vehicle. These HMIs can be customized based on the user and on their use case.

As mentioned previously, the autonomous vehicle is using sensors, algorithms and controller as well as input from the user in order to properly function. This creates the need for an entity that is capable of handling all this information and have a coherent flow to automatize the decision process of the AV. This is the role of the behavioral planner, taking information from all necessary input and evaluating them to propose the best decision to follow.

Therefore each of the four topics that were introduced will now be further investigated through a review on the already existing literature.

1.1. Autonomous Delivery Vehicles

Recent years have seen great progress in autonomous driving technology, primarily due to developments in AI, sensors, and communication systems. The development of autonomous delivery vehicles has the potential to revolutionize the transportation and logistics industries, as it addresses last-mile delivery challenges, operational efficiency in logistics, such as the capability of 24 hours delivery and reducing the cost of labor, which can represent up to 50% of the cost of the transport [5]. The need for autonomous delivery vehicles is therefore clear from an economic perspective; it is also safer, as studies have shown that autonomous vehicles could significantly decrease the risks of accidents and congestion [5]. Autonomous vehicles present themselves through various solutions such as autonomous drones (or unmanned aerial vehicles UAVs), autonomous delivery cars and trucks, and autonomous delivery robots [6].

Autonomous delivery drones offer the possibility of delivering in remote location or can also provide a good alternative in places where the traffic is too intense. They come in different sizes to account for different payload capacity, or even their deployment location (smaller drones for city use). Drones are usually subjected to less of a dynamic environment than ground vehicles, which could also help make the autonomous process easier.

Autonomous delivery robots are of smaller size which help them access some spaces like the sidewalk and bike lanes. This is particularly helpful to avoid traffic on the roads but most importantly, this allows the robots to deal with last-mile deliveries in difficult to access areas. Depending on their size and mobility, they could in theory deliver packages up to the door of houses and apartments, and thus deliver a better customer experience.

Autonomous delivery cars and trucks come in different sizes. While trucks are made for long trips and to handle a lot of packages (usually of higher weight and volume) they lack the maneuverability to perform well in cities. This is where autonomous delivery cars (or vans) are at an advantage. They offer more space than delivery robot, more maneuverability than delivery truck and they do not require as much governmental authorizations as the delivery drones. Autonomous delivery vans also inherit the same technology used on consumer autonomous driving cars, which also helps as the technology will be developed on both ends.

This part of the State of the Art will focus on the autonomous delivery vans as it is the platform on which this thesis is based.

1.1.1. Technology

Similar to self-driving consumer vehicles, autonomous delivery vehicles (ADV) are equipped with a vast array of sensors. These sensors fall into two main categories: environment perception, which detects and determines the vehicle's surroundings, and localization, which measures the position of the vehicle on the road. Together, these systems allow ADVs to navigate complex environments, avoid obstacles, and deliver goods safely and efficiently.

Localization is a critical component of ADV technology, as it ensures the vehicle knows its precise position in real-time. ADVs typically rely on a combination of the Global Navigation Satellite System (GNSS), Inertial Measurement Units (IMUs), and vehicle odometry sensors to determine their location. GNSS provides global positioning data, but its accuracy can be limited in urban environments with tall buildings or dense foliage, a phenomenon known as the "urban canyon effect" [7]. To address this, IMUs and odometry sensors supplement GNSS data by tracking the vehicle's acceleration, orientation, and wheel movements, enabling more accurate localization in challenging environments. Advanced ADVs also employ Simultaneous Localization and Mapping (SLAM) algorithms, which use sensor data to create and update a map of the environment while simultaneously tracking the vehicle's position within it [8].

For an ADV to safely navigate its environment, it must have a comprehensive understanding of its surroundings. This is achieved through a suite of sensors, including cameras, Radio Detection and Ranging (Radar), and Light Detection and Ranging (LiDAR). Cameras provide high-resolution visual data, enabling the ADV to detect and classify objects such as pedestrians, vehicles, and traffic signs. Recent advancements in computer vision and deep learning have significantly improved the accuracy of object detection and recognition in real-time [9]. Radar systems use radio waves to detect objects and measure their speed and distance. Radar is particularly effective in adverse weather conditions, such as rain or fog, where cameras and LiDAR may struggle [10]. LiDAR sensors emit laser pulses to create detailed 3D maps of the environment. LiDAR is highly accurate for detecting small objects and measuring distances, making it a key component of many ADV systems [11].

These sensors work in tandem to provide a robust perception system that allows the ADV to identify hazards, navigate complex routes, and make real-time driving decisions

as can be seen in figure 1.1.

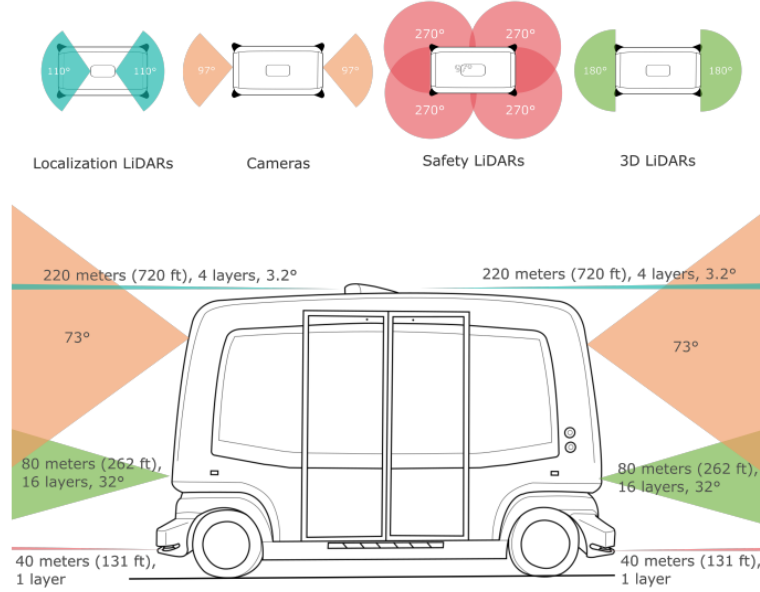


Figure 1.1: Localization and Detection Sensors of the Easy Mile EZ10

1.1.2. Important Implementations of Autonomous Delivery Vehicles

In addition to navigation and perception, ADVs are usually equipped with advanced robotics and storage systems to manage and protect goods during transit. These systems often include automated compartments, robotic arms for loading and unloading, and climate controlled areas for temperature-sensitive items. For example, companies like Amazon and Nuro have developed ADVs with customizable storage units that can securely transport packages, groceries, and even medical supplies [12] (figure 1.2). These innovations not only improve delivery efficiency but also enhance the user experience by ensuring that goods arrive in optimal condition.



Figure 1.2: Nuro Customized Storage [13]

Some notable and successful examples of ADVs include the company Nuro [14], which has collaborated with businesses like Kroger and Domino's Pizza to deliver groceries and pizzas in suburban neighborhoods, is one of the more successful examples in the autonomous delivery space. The human driver area on Nuro's autonomous delivery vehicles is removed to maximize storage capacity. The vehicles are built exclusively for the transportation of products. A major regulatory milestone was reached in 2020 when the U.S. Department of Transportation gave Nuro permission to run their vehicles on public highways.

Another significant participant in this space is Starship Technologies [15], which runs a fleet of small, self-sufficient delivery robots on university campuses and in selected urban areas. These robots are utilized for last-mile delivery of food and parcels. They move at walking speeds and navigate sidewalks using computer vision. Starship offers an alternative to conventional distribution methods through partnerships with academic institutions such as George Mason University and Purdue University.

In China the Meituan retail and services corporation has developed internally fully autonomous delivery cars [16]. They navigate the street of China delivering packages with a full array of sensors to avoid obstacles and other encumbrance on the road. They are active in the city and drive alongside normal traffic (figure 1.3). Their small size is adapted to last-mile delivery.



Figure 1.3: Meituan AUtonomous Delivery Vehicle [17]

The technological aspect of the ADVs is not enough to enable them to navigate. In fact, the output of the hardware that is providing location and perception has to be understood and interpreted. This is where the navigation and motion strategies are used.

1.2. Navigation and Motion Strategies

The field of autonomous vehicles (AVs) relies on fundamental components of navigation and motion strategies. The data collected by localization and perception systems is processed by sophisticated software algorithms, sometimes built on machine learning and artificial intelligence (AI) models. Mapping, Path Planning, and Motion Control are the most important regarding pre-planned autonomous driving. Each of these fields involves a combination of algorithms, sensor integration, and optimization techniques.

Mapping provides a spatial representation of the environment, which is essential for vehicle localization and path planning. Modern mapping techniques are categorized based on data type, processing methods, and intended applications. This step is crucial to better visualize what the AV will be driving through. Map representation is an important aspect of mapping as it helps visualize the actual maps that have been derived from the previously mentioned methods. Then the map resolution is important as it needs to be balanced regarding precision and computational time. And lastly the algorithm that is building the map is of the utmost importance. These algorithms come in various level of precision and computational expense. Each of their specificity has to be analyzed to ensure they answer to the demand. The algorithms will help create the base of the map (and distinguish between occupied and free space) which will be used by the path planner to create the path.

Path planning is used to determine the most efficient and safe route from a starting

point to a destination. The algorithms in this domain must consider constraints such as obstacles, road rules, and vehicle dynamics. Two main types of path planning exist, Global Path Planning which generates an optimal route based on a pre-existing map, and Local Path Planning which refines the global path in real-time by accounting for dynamic obstacles and environment changes. The algorithms then needed to create the path or trajectory are associated to each of the two types of path planning. The algorithms then produce a path that serves as an input to the motion control.

Motion control translates the planned path into executable commands for the vehicle's actuators, to make decisions about speed, direction, and obstacle avoidance. It must also be accurate to enable the vehicle to follow the planned path precisely as well provide stability to ensure the vehicle remains balanced and controllable and that it is not subjected to jerks that could damage the AV's components.

All of these 3 part contribute to the navigation of the AVs. It is thus important to review in more detail each of them.

1.2.1. Mapping

As mentioned before mapping is crucial in the navigation of AVs. The first aspect of a map is its representation. This includes Metric Maps (Geometric), which provide precise spatial representations, including distances, angles, and coordinates (e.g., occupancy grids or point clouds), Topological Maps, which simplify the environment into a network of connected nodes, and Hybrid Maps, which combine metric and topological elements for comprehensive understanding. The grid-based Metric Maps are used for high-precision localization and navigation, as they can identify detailed road layouts, lane markings and traffic signs. Moreover these maps are built using LiDAR, GPS, cameras, and IMU sensors. This is a better application for an AV as it provides a lot more information than the topological maps. Furthermore Metric Maps are usually based on High Resolution maps that while use more storage space and take more time to compute, allow for better perception of the environment of the AV. The most common type of Metric Maps are called Occupancy Grid Map which partition the robot's surrounding into cell that stores a probability whether it is occupied by an obstacle or free [18].

To build this Occupancy Grid map the better method to use is Simultaneous Localization and Mapping (SLAM) as it is the cornerstone of mapping for autonomous vehicles [8]. SLAM involves building a map while simultaneously localizing the vehicle within it. Two of the most popular methods of SLAM are LiDAR based SLAM and Visual based SLAM [19]. Compared to the Visual based SLAM, which relies on the use of cameras,

LiDAR-SLAM, which relies on LiDAR sensors, can accomplish its own localization and mapping under more diversified environmental conditions. The information collected by LiDAR presents a series of scattered points with accurate angle and distance information, which is called a point cloud [20]. Moreover LiDAR based SLAM can be coupled with the pose from the IMU to provide an even more precise mapping. This is crucial to provide a good map as a base for the path-planner.

The Occupancy Grid is the map representation which has been selected for this thesis work and a more comprehensive review will take place in chapter 2.

1.2.2. Path planning

From the map obtained, the path planning is the next step of the navigation for AVs. As mentioned before, two main types of path planning exist, Global and Local Path Planning. There is also a categorization present among the path planning algorithms. The Grid-Based Planning algorithms include Dijkstra's Algorithm [21] which finds the shortest path by exploring all nodes systematically, and A* [22] which enhances Dijkstra's with a heuristic function to prioritize promising paths. Both algorithms have to discretize the whole map they are given and then start with planning the path. This presents several challenges, including increased computational time for the initial discretization as well as update the map whenever the environment changes, as this requires a complete re-discretization of the map. This is problematic as usually the environment of an AV (outside of controlled spaces) is subject to dynamic changes. This is where the Sampling-Based Planning algorithms prove to be useful.

They incrementally search, in the perceivable environment of the AV, for obstacles and dynamic changes. These algorithms include Probabilistic Roadmaps (PRMs) [23], a multi-query planner, which builds a roadmap of possible paths by sampling the environment and Rapidly-Exploring Random Trees (RRT) [24], a single-query planner, that is suitable for high-dimensional spaces, balancing exploration and exploitation. As ADVs are generally pursuing a single query trajectory (point A to point B) the RRT seems like a better solution. In the case the ADV is subject to multiple delivery point a good algorithm to order the paths in the most efficient way will be used. With so many use cases, evolutions of the RRT have been developed, such as the RRT* [25] which finds the shortest path by calculating the cost-to-go. Finding a lower cost to go is an important aspect as it helps reduce energy consumption and time of the ADV. The trajectory now known it is possible to feed it to the Motion Control.

The RRT* is the path planning algorithm which has been selected for this thesis work

and a more comprehensive review will take place in chapter 2.

1.2.3. Motion control

Control strategies are divided in different groups that reflect the simplicity and approach of the controller. Proportional Integral Derivative (PID) controllers [26] are low-level controllers widely used for their simplicity and effectiveness in maintaining speed and steering stability. However, they are not always sufficient to provide the desired results, and thus it is necessary to rely on another type of controller, such as Model-Based controllers or Learning Based controllers. Model-based controllers include the Dynamic Inversion Control, which controls non-linear vehicle dynamics for agility, and the Model Predictive Control (MPC), which forecasts future states and optimizes control inputs over a horizon and is frequently used in trajectory tracking. Learning-based controllers like reinforcement learning adjust to complex situations by learning control rules via trial and error.

Control strategies for motion controllers can be categorized into distinct groups based on their complexity, design, and application. Among these, Proportional Integral Derivative (PID) controllers [26] represent one of the most widely used low-level control strategies due to their simplicity, reliability, and effectiveness in maintaining speed and steering stability. PID controllers operate by continuously adjusting the control input based on the error between the desired and actual states, making them suitable for a variety of linear and near-linear systems. However, they rely on fixed gains and linear assumptions which often limits their performance in highly dynamic or non-linear environments, where more advanced control strategies are required. In contrast, Learning-Based Controllers offer a data-driven alternative, adapting to complex and uncertain environments through experience. A notable example is Reinforcement Learning (RL), which learns control policies via trial and error, optimizing actions based on rewards or penalties received from the environment. RL-based controllers excel in scenarios where system dynamics are difficult to model analytically or where the environment is highly variable. However, they typically require extensive training data and computational effort, which can limit their practicality in some real-world applications.

To address these limitations, Model-Based Controllers have been developed, using mathematical models of the system to achieve more precise and adaptive control. One prominent example is Model Predictive Control (MPC), which distinguishes itself by forecasting future system states over a finite horizon and optimizing control inputs accordingly [27]. This predictive capability makes the MPC particularly well-suited for

non-linear systems and applications such as trajectory tracking, where anticipating and mitigating deviations from the desired path is required. Model-based controllers often require accurate system models and can require non-negligible computational resources, however its usability in obstacle avoidance and great trajectory tracking make it very suitable for ADVs.

The MPC is the Motion Controller which has been selected for this thesis work and a more comprehensive review will take place in chapter 2.

Navigation is at the essence of what makes autonomy possible, it alleviates the needs from the user to give velocity and steering input. However autonomous vehicles in general still need the user to input the final destination. This is where a human machine interface is useful.

1.3. Human-Machine Interface

Over the past few decades, Human-Machine Interfaces (HMIs) have evolved significantly. While early HMIs relied on physical controls such as switches, buttons, and dials, modern interfaces incorporate touchscreens, voice recognition, augmented reality (AR), and gesture-based controls, offering users a more natural and immersive experience by integrating these cutting-edge technologies to enhance efficiency, usability, and interactivity [28]. At their core, HMIs serve as the bridge between humans and machines, enabling users to interact with complex systems seamlessly. The effectiveness of an HMI directly impacts productivity, accessibility, and user satisfaction across various domains, including industrial automation, consumer electronics, automotive systems, healthcare devices, and smart infrastructure.

HMIs can be classified based on the type of interaction, application domain, and level of adaptability. The Traditional HMIs primarily involve physical controls and mechanical input devices, which have been historically used in manufacturing, transportation, and home appliances such as keyboards and buttons. The Graphical User Interfaces provide visual interaction elements such as menus, icons, and windows, which allow users to navigate and operate systems easily such as computer operating systems (Windows, macOS, Linux). The Natural User Interfaces aim to minimize the learning curve by enabling intuitive interaction methods that mimic human actions. Examples include gesture recognition (Microsoft Kinect) [29], voice recognition (Amazon Alexa), and eye-tracking systems. The Adaptive and AI-Powered HMIs that leverage machine learning and AI to analyze user behavior and dynamically adjust their responses such as AV interfaces like Tesla's adaptive driver-assist systems. And the augmented and Virtual Reality (AR/VR)

Interfaces which integrate 3D visualization and immersive interaction methods to enhance user experience such as flight simulators and wearable AR glasses. Each of these HMI categories plays a significant role in modern automation, accessibility, and user experience improvements. However the cost of the evolved HMIs are not always justified and the appropriate HMI type depends on the use case where there needs to be a balance between complexity and usability.

1.3.1. QR Codes Integration

QR codes are a cost-effective and efficient solution to enhance Human-Machine Interaction, particularly in industries where quick access to information, seamless authentication, and remote control of systems are required. They bridge the gap between physical and digital interfaces, allowing users to access configuration settings instantly. Moreover, they reduce the hardware complexity as instead of expensive touchscreens and physical control interfaces, QR codes allow users to interact with systems using their personal devices.

These two-dimensional barcodes encode a significant amount of machine-readable data, allowing users to scan and retrieve information instantly. Furthermore, they can be scanned from any angle within a 360-degree range, eliminating the need for precise alignment between the scanner and the code. QR codes are also highly accessible, enabling users to generate customized codes for specific purposes, such as linking to websites for advertising. As a result, QR codes find extensive application across diverse domains, including retail, dining, and digital platforms, enhancing user experience and operational efficiency [30].

One innovative example of QR-based HMI integration is the QR-enabled autonomous bartender. This system allows customers to customize their drink orders by scanning a QR code with their smartphones. The user selects their preferred drink parameters (e.g., ingredients, volume, strength) via a mobile interface, and the autonomous machine prepares the beverage accordingly [31], which delivers a lot of customization without having to rely on expensive hardware.

A cost effective solution using the QR code as an HMI output has been developed for the purpose of this thesis as well as an innovative implementation and both will be explained in the chapter 3.

1.4. Behavioral Planning

Behavioral planning lies between high-level mission planning (which defines long-term goals) and low-level motion planning (which determines precise trajectories and control inputs). A behavioral planner interprets the current situation, evaluates possible actions, and selects the most appropriate behavior to navigate the environment while considering safety, efficiency, and interaction with other agents (e.g., vehicles, pedestrians) [32]. Behavioral planning is a critical component of intelligent decision-making in autonomous systems, particularly in robotics, artificial intelligence (AI), and autonomous vehicles. It involves determining what actions an agent should take in a dynamic environment to achieve its objectives efficiently and safely.

Behavioral planning can be implemented using various techniques. One common method is Finite State Machines (FSMs), where predefined states and transitions guide its decision-making. For example, an autonomous vehicle might switch between states such as "Cruise," "Follow Vehicle," and "Stop at Intersection" based on traffic conditions. Another approach is Rule-Based Systems, which use logical conditions to determine behavior selection, such as a robotic vacuum deciding to turn upon detecting an obstacle. More advanced methods include Machine Learning and Reinforcement Learning (RL), which enable autonomous agents to adapt and improve through experience like a self-driving car learning optimal lane-changing strategies from past interactions. Finally, Hybrid Approaches combine rule-based logic with AI-based adaptability, allowing for both structured decision-making and flexibility. A practical example of this is an industrial robot that follows programmed safety rules but optimizes its movements using reinforcement learning. These diverse methods ensure that behavioral planning systems can operate effectively in a wide range of environments.

Moreover FSMs are particularly useful in handling asynchronous situations typical of human-machine interaction (HMI), where the system must dynamically respond to user inputs, environmental changes, and unexpected events. By modeling behaviors as a set of states with defined transitions, FSMs facilitate robust and predictable interaction management, improving the adaptability and responsiveness of autonomous systems.

1.4.1. Behavioral Planning in Autonomous Vehicles

The behavioral planner acts as an intermediary between perception systems (which analyze the environment) and motion planning systems (which generate precise trajectories). It ensures that decisions align with traffic rules, safety constraints, and user comfort [33].

Examples of Behavioral Planning in Autonomous Vehicles Finite State Machines (FSMs) are used in a number of scenarios due to their various complexity possibilities as well as providing predictable and interpretable behavior. This is more notable on the driving scene where, depending on the situation and the environment, more or less complexity is required. That is why most of the cars in the DARPA Urban Challenge (DUC) had their decision made with FSMs of various degree of complexity [33]. More recently [34] designed an FSM-based behavior planner for vehicles operating on rough terrain. The state transitions account for obstacle avoidance, path following, and terrain adaptation, enabling autonomous vehicles to navigate challenging landscapes efficiently. Moreover the research by [35] proves that even higher level of behavior complexities can be achieved by FSMs.

A FSM has been used in the context of this thesis as a behavioral planner, to implement the state changes necessary for the good functioning of the ADV.

2 | Navigation

In section 1.2, the navigation and motion strategies used in autonomous vehicles were presented, emphasizing various mapping techniques, path planning algorithms, and motion control methods. As previously mentioned, the navigation module for this thesis will consist of three key components: occupancy grid mapping, RRT* path planning, and an MPC controller. Additionally, an overview of the vehicle used in this study will be included, as reviewing the hardware is essential for understanding its navigation capabilities.

This chapter starts with a brief introduction to the EZ10, which is equipped with various sensors needed for the perception and localization which are crucial for the proper functioning of an autonomous vehicle as discussed in section 1.1. It then goes into the mapping technique used in this study, the occupancy grid, explaining its methodology in detail. Following this, the chapter explores the specific RRT (Rapidly-exploring Random Tree) developed for this application, with a particular attention to its formulation and implementation. Finally, an overview of the MPC (Model Predictive Control) controller employed in the system will be presented, highlighting its role and functionality.

2.1. Overview of the EasyMile EZ10

A complete and very detailed presentation of the EasyMile EZ10 is present in the the thesis work [36]. This overview will simply introduce the hardware that has been used for this thesis.

Being a platform for an autonomous shuttle bus, the EZ10 is equipped with the necessary sensors that have been mentioned in 1.1. Moreover the interior layout of the EZ10 make it an ideal candidate to perform delivery tasks as not only does it have a significant empty space in its middle, with facing seats on the edge of the bus, but comes also equipped with a ramp that would make delivering heavy payloads easier to load. The EZ10 is equipped with various perception sensors but the ones of interest for this thesis are the rear and front Velodyne LiDAR sensor, a 3D LiDAR that is capable of mapping its surroundings at 180° up to 100 meters (figure 2.1).

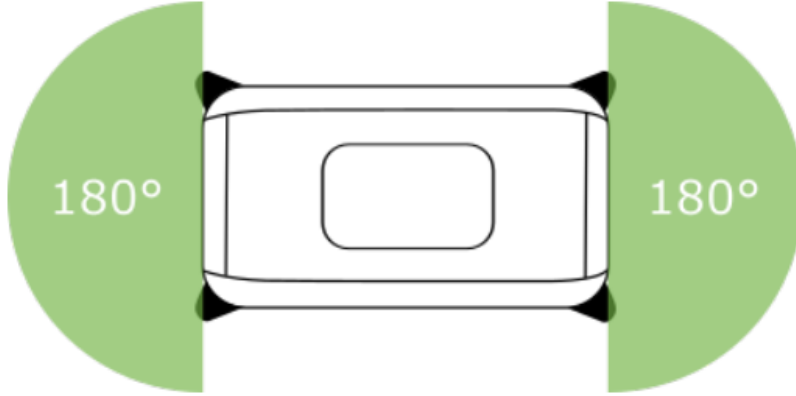


Figure 2.1: Front and rear Velodyne range of detection

Regarding the localization aspect, the EZ10 is equipped with a GNSS (Global Navigation Satellite System) antenna [36]. To enhance the tracking position of the bus a Real-Time Kinematic (RTK) correction has been implemented by referencing over to a base station present at Politecnico Di Milano Building B23. Moreover, the EZ10 is equipped with ROS (Robot Operating System) and with certain packages allows the data from the LiDARs and the GNSS to be acquired (using ROS messages) and even stored (using ROS bags). This makes the EZ10 equipped for Simultaneous Localization and Mapping (SLAM) by using its LiDARs and GNSS, which as mentioned previously, is the optimal method to create an occupancy grid.

Now that the hardware has been introduced, it seems necessary to explain how the data is retrieved and can be visualized for mapping.

2.2. Mapping

As there already exists several tools in literature for mapping and SLAM and developing a tool is not a goal of this thesis work, it was decided to use one already available and tailored to this work's needs.

Therefore, to map the area, the occupancy grid mapping method was chosen as not only is the EZ10 capable of handling SLAM to optimally create this type of map but also because the occupancy grid is a simple yet effective way of visualizing the free and occupied space (obstacles). This type of map would prove very useful for the next step of path planning as the algorithm will deal with a clear go and no-go zone in which the trajectory can be created.

As mentioned previously the EZ10 is equipped with ROS, which allows to record the data from the rear and front Velodyne sensors as well as the odometry data. ROS also allows the use of the package Gmapping which essentially creates a 2D occupancy grid from scan and pose data. Therefore the data recorded by the LiDAR will serve as the scan from which it is possible to detect boundaries (as well as obstacles) and the odometry data is from the Inertial Measurement Unit (IMU) of the vehicle coupled with the proper localization from the GNSS.

2.2.1. Gmapping for Occupancy Grid Mapping

The Gmapping has to deal with multiple 3-D coordinate systems that change over time which can be challenging. Therefore, a tool like the tf2 package is used to keep track of all the transformation information over time. This package is able to maintain the relationship between coordinate frames in a tree structure and lets the user customize points, vectors, etc. between co-ordinate frames to suit their need. The exact transforms used for this thesis are present in figure 23.

To use the Gmapping package it is also necessary to obtain the odometry of the vehicle. The odometry of the vehicle can be obtained by adding the state of the vehicle with its location. While the state of the vehicle is usually determined by the IMU the location of the vehicle is based, in the case of the EZ10, on the GNSS signal and its RTK correction. The GNSS signal is transmitted from satellites to the antenna of the EZ10, and by a usually simple triangulation, an estimate of the position of the vehicle can be made in the range of meters [36]. For autonomous driving this range of error would very-likely result in incidents. That is why, for this thesis, a GNSS with RTK correction is used. This correction allows for accuracy in the range of millimeters. The RTK correction requires a base station that is known accurately and a signal is transmitted to the vehicle to further enhance the precision of its location. This is illustrated in figure 2.2.

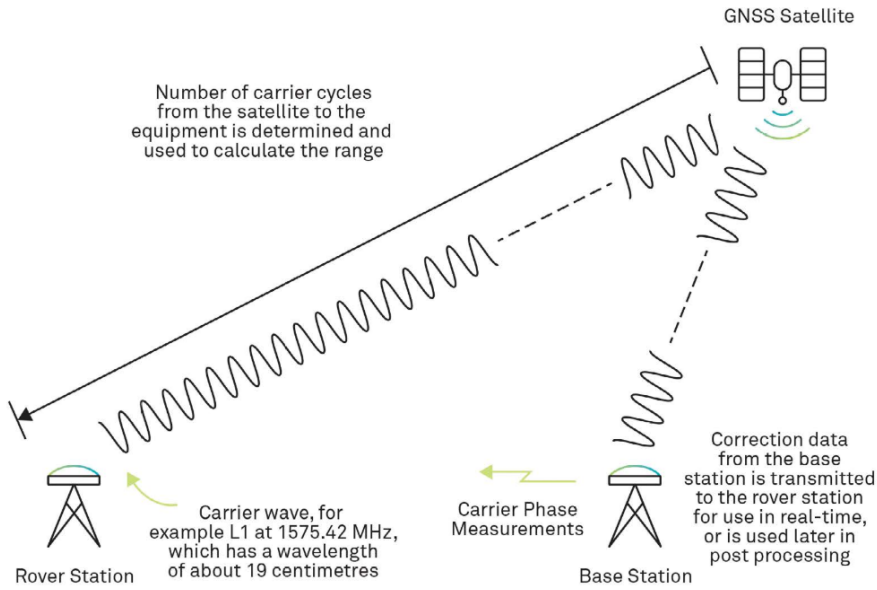


Figure 2.2: Overview of GNSS with RTK correction

The Gmapping package comes with parameters that through fine-tuning will improve the map. The paper [37] offers a more detailed classification of the parameters. It is possible to count 6 different groups of parameters aimed at tuning different parts of the Gmapping with a total of 36 different parameters to tune. The list of classified parameters and their default values can be found in Appendix A. Paper [37], also states that it is mandatory to fine-tune the Gmapping parameters so that the data from the sensors is properly processed into an occupancy grid map. This fine tuning has to be done manually as there is no real guidance except to go lower or higher in the parameter's values.

This also brings the subject of how to see the map that is being generated to determine what parameters have to be changed. For the visualization of the map, the software RViz (a 3-D visualization tool built-in the ROS framework commonly used to visualize sensor data) has been used as it allows to see in real time the creation of the occupancy grid via the Gmapping package as well as save the final map. The saved map has a joint YAML type file which contains information about the saved map. This information is crucial to obtain a map in real coordinates, as the occupancy grid map is just a pixilated map without any real definition or origin as can be seen in figure 2.3

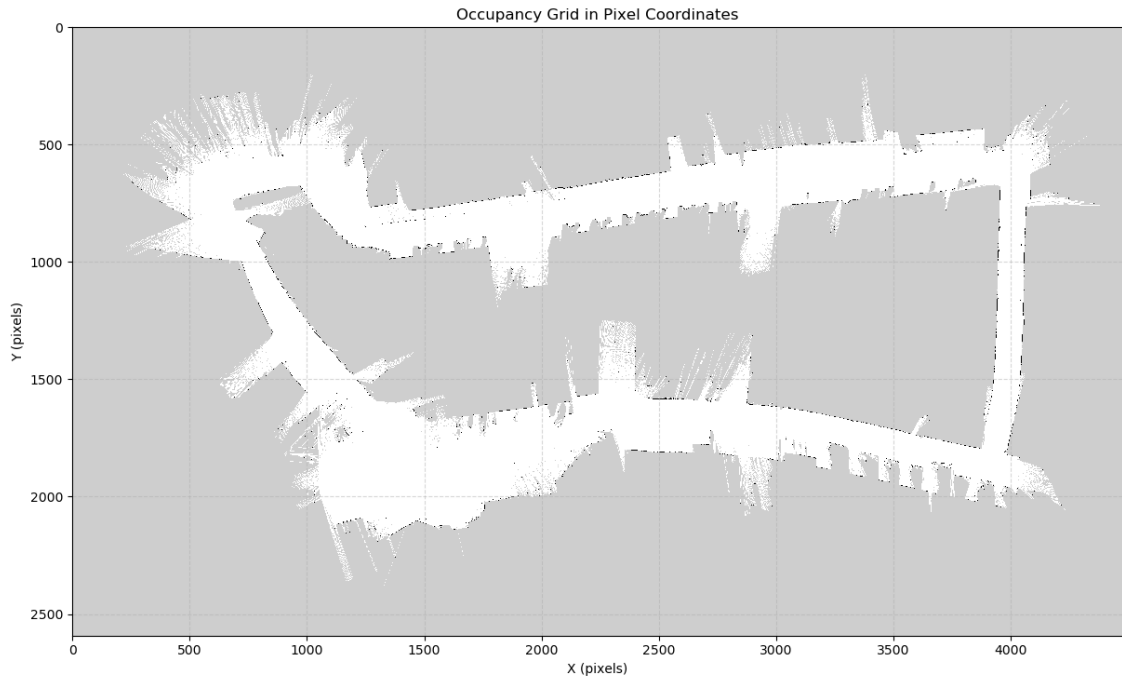


Figure 2.3: Default Gmapping parameters in pixel coordinates

The RTK correction implies that the origin of the map must be located at the base i.e. building B23 which is located at the south east of the center of the mapped campus LaMasa. This also provides a mean to verify if the map has been properly generated. Having started the with default parameter values, present in table 2, the obtained occupancy grid map was unusable. The figure 2.4 has a gray background which shows (in the conditions this map was created) that the cells outside the path were unidentified. Moreover, the location of the origin is within the path which is not the case in reality. Furthermore, the general shape is not correct as there are no straight roads like the real life road has. This means the Gmapping parameter need to be tuned is required.

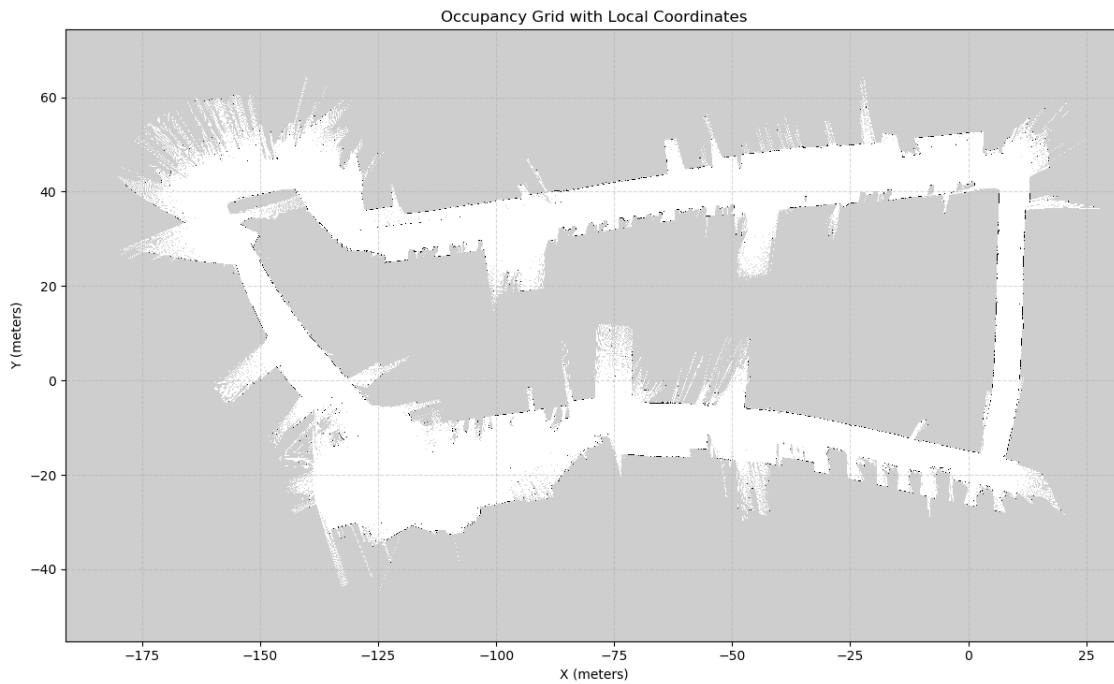


Figure 2.4: Default Gmapping parameters in local coordinates

Tuning of the Gmapping required trial and error experimenting, as the parameters are to be tuned for each specific mapping. The tuned Gmapping parameters used can be found in table 3. The resulting map was from the data of the front LiDAR and is more than acceptable as the expected straight lines can be seen in figure 2.5. Moreover the origin is in a realistic location and all the cells outside the path are black and thus properly identified as occupied space.

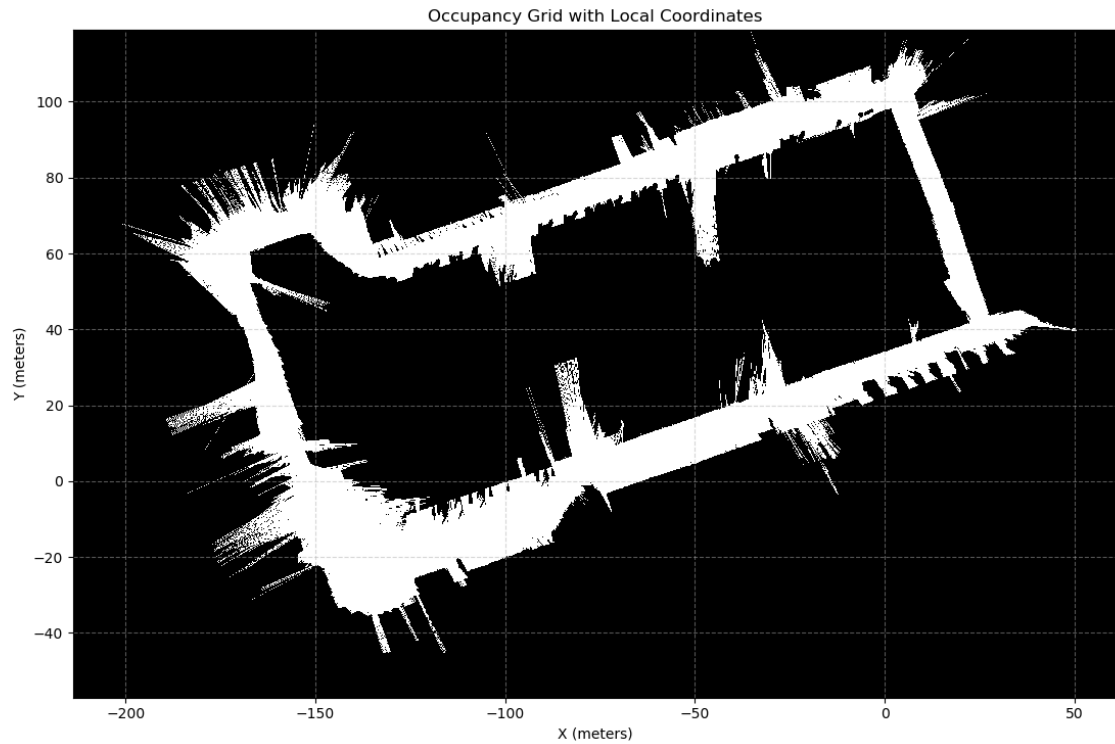


Figure 2.5: Tuned Gmapping parameters in local coordinates

To make sure that the map was indeed correct and properly represent the outline of the actual campus road, the idea of superimposing the occupancy grid to a satellite image of the campus was thought of. By using the QGIS software [38], a free and open source for spatial visualization, it allowed to put common reference points, red points in figure 2.6 on the occupancy grid and on the satellite image to see if they would match.



Figure 2.6: Superimposed images

To reduce the amount of uncertainty that may be involved with the motion of the vehicle, sensor fusion, which consist of merging the data of multiple sensors, can be used. For this purpose the `ira_laser_tools` package, a ROS tool for LiDAR scan fusion, was used to enable the user to merge multiple LiDAR scans. This allowed for a better representation of narrow obstacles that were not clearly defined with the front LiDAR only mapping such as areas with trees and posts for bicycle. Without this tool, the mapping could have led to a wrong evaluation of occupied and free space and an improper path planning. Tuning details can be found in figure 24. By merging the data of the front and rear velodyne sensors, the outlines of the map were clearly better defined as seen on figure 2.7.

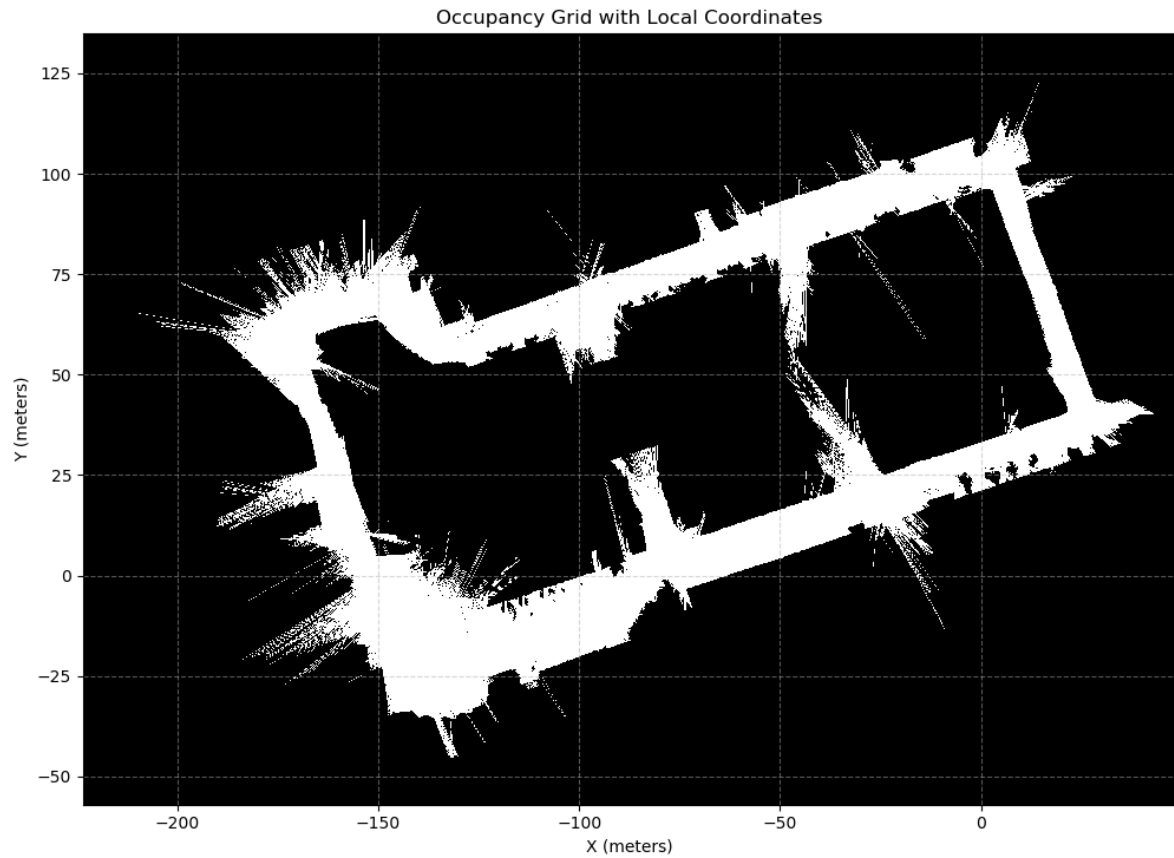


Figure 2.7: Map after sensor fusion in local coordinates

The final occupancy grip map obtained (figure 2.7) is the most accurate map produced from the data gathered. Thus the occupancy grid of the whole road going around the campus was made with a resolution of 0.1 m per pixel (each pixel represents 10 cm in real life). This resolution was taken based on having a proper delimitation of the map with any and all obstacles present properly defined and maintaining a manageable computational time for the path planning as a higher resolution would result in having more nodes to explore.

An important thing to note is that the gmapping not only registers the free and occupied cells but also is aware of the origin of the map as it needs the odometry of the vehicle. as discussed before the localization of the vehicle is done by GNSS with RTK correction meaning the origin of the map is the base tower located in B23 (outside of the mapped campus).

2.3. Path Planning

The obtained occupancy grid map acts as an input to the path planning. In fact the occupancy grid map is composed of free and occupied cells (sometimes also unknown cells which are labeled as occupied for safety). This straightforward classification allows the logic of the path planning to be simpler, only allowing the path to be extended in the free cells. The path planning method used for this thesis is the Rapidly-Exploring Random Trees Star (RRT*), as the primary objective is to make the EZ10 capable to autonomously deliver packages in the campus. This requires the vehicle to be equipped with a more local focused planner. Moreover, the road that the vehicle will take is a loop and the direction of motion will be dependent on the first delivery as will be discussed later. The RRT will be introduced in detail as well as the useful evolutions that were used for this thesis.

2.3.1. Overview of the RRT

Rapidly-Exploring Random Trees (RRT) is a fundamental algorithm introduced by Steven M. LaValle in his paper [39] and has revolutionized motion planning, particularly in high-dimensional spaces where traditional methods struggle. RRT provides an efficient and probabilistically complete solution for navigating complex environments, making it a cornerstone of robotic path planning, autonomous navigation, and kinodynamic motion planning (path planning while obeying kinematic and dynamic constraints).

RRT is a data structure and an incremental search algorithm that builds a tree rooted at the start position and progressively explores the search space through random sampling. The core idea behind RRT is biased exploration, which allows it to quickly explore large and complex spaces while ensuring feasibility as shown in figure 2.8 and 2.9

```

BUILD_RRT( $x_{init}$ )
1   $\mathcal{T}.\text{init}(x_{init});$ 
2  for  $k = 1$  to  $K$  do
3       $x_{rand} \leftarrow \text{RANDOM\_STATE}();$ 
4       $\text{EXTEND}(\mathcal{T}, x_{rand});$ 
5  Return  $\mathcal{T}$ 

```

```

EXTEND( $\mathcal{T}, x$ )
1   $x_{near} \leftarrow \text{NEAREST\_NEIGHBOR}(x, \mathcal{T});$ 
2  if  $\text{NEW\_STATE}(x, x_{near}, x_{new}, u_{new})$  then
3       $\mathcal{T}.\text{add\_vertex}(x_{new});$ 
4       $\mathcal{T}.\text{add\_edge}(x_{near}, x_{new}, u_{new});$ 
5      if  $x_{new} = x$  then
6          Return Reached;
7      else
8          Return Advanced;
9  Return Trapped;

```

Figure 2.8: Basic RRT Construction Algorithm [24]

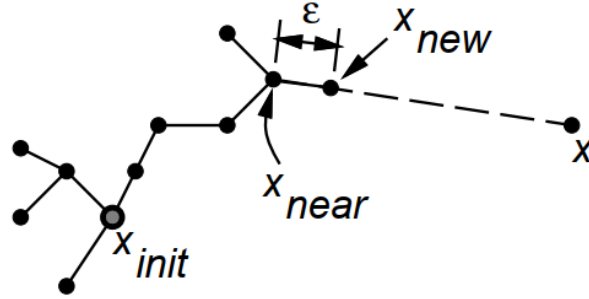


Figure 2.9: The Extend Part [24]

RRTs have emerged as an online, single-query, counterpart to PRMs as most online planning problems do not need multiple queries, as the robot may transition between different environments or operate in an environment that is not known in advance. Furthermore, in certain applications, precomputing a roadmap can be computationally demanding or even impractical [40]. Additionally, RRT offers probabilistic completeness, ensuring that if a feasible path exists, the algorithm will find it given sufficient time and iterations. Its scalability further enhances its applicability across diverse problem domains, including both holonomic and non-holonomic systems. Moreover, RRT is easy to implement due to its simple and modular structure, allowing seamless integration with other algorithms and optimization techniques. Since its early development, RRT has evolved

into various forms to enhance performance in specific applications.

2.3.2. The RRT*

For the purpose of this thesis the RRT* will be used. It is an optimal version of the RRT, that rewires the tree to improve path quality by ensuring that the minimal cost solution has been found. This modification can be seen in the figure 2.10 that represent the basic algorithm for an RRT*.

```

1  $V \leftarrow \{x_{init}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{rand} \leftarrow \text{SampleFree}_i;$ 
4    $x_{nearest} \leftarrow \text{Nearest}(G = (V, E), x_{rand});$ 
5    $x_{new} \leftarrow \text{Steer}(x_{nearest}, x_{rand});$ 
6   if  $\text{ObstacleFree}(x_{nearest}, x_{new})$  then
7      $X_{near} \leftarrow \text{Near}(G = (V, E), x_{new}, \min\{\gamma_{\text{RRT}^*}(\log(\text{card}(V))/\text{card}(V))^{1/d}, \eta\});$ 
8      $V \leftarrow V \cup \{x_{new}\};$ 
9      $x_{min} \leftarrow x_{nearest}; c_{min} \leftarrow \text{Cost}(x_{nearest}) + c(\text{Line}(x_{nearest}, x_{new}));$ 
10    foreach  $x_{near} \in X_{near}$  do // Connect along a minimum-cost path
11      if  $\text{CollisionFree}(x_{near}, x_{new}) \wedge \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new})) < c_{min}$  then
12         $x_{min} \leftarrow x_{near}; c_{min} \leftarrow \text{Cost}(x_{near}) + c(\text{Line}(x_{near}, x_{new}))$ 
13     $E \leftarrow E \cup \{(x_{min}, x_{new})\};$ 
14    foreach  $x_{near} \in X_{near}$  do // Rewire the tree
15      if  $\text{CollisionFree}(x_{new}, x_{near}) \wedge \text{Cost}(x_{new}) + c(\text{Line}(x_{new}, x_{near})) < \text{Cost}(x_{near})$ 
16        then  $x_{parent} \leftarrow \text{Parent}(x_{near});$ 
17         $E \leftarrow (E \setminus \{(x_{parent}, x_{near})\}) \cup \{(x_{new}, x_{near})\}$ 
18 return  $G = (V, E);$ 

```

Figure 2.10: Basic RRT* Construction Algorithm [41]

With this rewiring evolution over the standard RRT, the RRT* is capable to find the shortest path possible that can be backtracked from the RRT tree created. This ability is very important for path planning as the goal is to make the vehicle safely reach its destination but also while taking the less distance to save on time and energy consumption.

However the RRT* with the rewiring and tree updating part, is computationally heavy. That is it takes much more time for it to arrive to the goal than it would for the normal RRT. In fact, the RRT* has been also updated to make it faster and to be able to near the computational time of the RRT while offering the optimal path. Many of these evolution such as the RRT*-Smart work well [25] and make the RRT* faster, but are usually not compatible with dynamic environments or online implementations. Instead one particular aspect of the RRT in general needs improvement, the nearest-neighbor search.

2.3.3. KD-tree and Nearest Neighbor Searching

In the paper [39] by Steven LaValle, the need for efficient nearest neighbors (NNs) search techniques is characterized crucial for computational efficiency. This problem has been reiterated by him in a following paper [24], and resulted in a paper being published about Nearest-Neighbor Searching for improving motion planning [42]. In this paper, the possible improvement that KD-trees could give to the NNs searching problem is studied. The construction and query of the KD-tree are represented in figure 2.11 and 2.12 respectively.

```

BUILD_KD_TREE( $P, d, T, m, b, s$ )
Input: A set of points,  $P$ , the dimension of the space,  $d$ , the
topology of the space,  $T$ , the number of points to store in a
leaf,  $m$ , the bounding box,  $b$ , for  $P$ , and the splitting rule,  $s$ .
Output: The root of a kd-tree storing  $P$ 
1  if  $P$  contains less than  $m$  points
2      then return a Leaf storing these points
3      else split  $b$  into two subboxes,  $b_1, b_2$ , according
           to  $s$  by plane  $l$ , orthogonal to dimension  $k$ .
4          Find  $P_1$  and  $P_2$ , the sets of the data points
           falling into boxes  $b_1$  and  $b_2$ .
5           $v_1 = \text{BUILD\_KD\_TREE}(P_1, d, T, m, b_1, s)$ 
6           $v_2 = \text{BUILD\_KD\_TREE}(P_2, d, T, m, b_2, s)$ 
7          Create a Node  $v$  storing the splitting plane,  $l$ ,
           the splitting dimension,  $k$ , the topology of the
           space  $T_K$  of this dimension, the projection
           of the box,  $b$ , on  $T_K$ , and  $v_1$  and  $v_2$ , the
           children of  $v$ .
8      return  $v$ 

```

Figure 2.11: KD-tree construction algorithm [42]

```

KDTree::SEARCH( $q$ )
Output: the closest to  $q$  point  $p$  stored in kd-tree.
1  Calculate squared distance  $d_{box}$  from the box
   associated with the root node to  $q$ .
2   $p = NULL$ 
3   $root \rightarrow SEARCH(d_{box}, \infty, p)$ 
4  return  $p$ 

```

```

Node::SEARCH( $d_{box}, d_{best}, p$ )
Input: squared distance,  $d_{box}$ , from  $q$  to the box containing
the current Node, and squared distance,  $d_{best}$ , from  $q$  to the
closest point,  $p$ , seen so far;  $d_{best}$  and  $p$  are to be updated.
1  if  $d_{box} < d_{best}$ 
2    Split  $b_K$  (the projection of the current Node onto
   the space  $T_K$ , stored in this Node) into two
   subboxes,  $b_{K_1}$  and  $b_{K_2}$ , by the splitting line  $l$ ,
   corresponding to  $v_1$  and  $v_2$  respectively.
3     $d_1 = dist_{T_K}^2(q, b_{K_1})$ 
4     $d_2 = dist_{T_K}^2(q, b_{K_2})$ 
5    if  $d_1 < d_2$ 
6      then  $v_1 \rightarrow SEARCH(d_{box}, d_{best}, p)$ 
7           $v_2 \rightarrow SEARCH(d_{box} - d_1 + d_2, d_{best}, p)$ 
8      else  $v_2 \rightarrow SEARCH(d_{box}, d_{best}, p)$ 
9           $v_1 \rightarrow SEARCH(d_{box} - d_2 + d_1, d_{best}, p)$ 

```

```

Leaf::SEARCH( $d_{box}, d_{best}, p$ )
Input: squared distance,  $d_{box}$ , from  $q$  to the box containing
the current Leaf, and squared distance,  $d_{best}$ , from  $q$  to the
closest point,  $p$ , seen so far;  $d_{best}$  and  $p$  are to be updated.
1  Calculate squared distances from  $q$  to all the points
   in the current Leaf, and update  $p$  and  $d_{best}$ .

```

Figure 2.12: KD-tree query algorithm [42]

Since that paper was published back in 2007, there has been little to none research on the subject of using KD-trees as a more efficient NNs searching method. Moreover the feasibility of the KD-trees as efficient NNs searching method was studied on older now outdated systems. With the evolution of the technology it is necessary to review the feasibility of that method as a way to find the optimal path without the handicap of increased computational time of the simple RRT*.

2.3.4. Comparison of the RRTs

To select the better RRT algorithm, 3 algorithms were created, all written in C++ (10 to 30 times faster compiling time than Python). An algorithm for the RRT, RRT* and RRT* with KD-tree implementation (RRT*KD). All algorithms have the exact same functions (except the added specific function for each variation) to better compare their results.

As this work aims to simulate a real experiment, the algorithms need to take into

account the size of the EZ10 to simulate the problems that it will generate throughout the map. For this purpose a footprint of the vehicle was created. Being that the footprint should represent the vehicle in all possible positions, the idea to create a circle with a radius equal to the length of the bus was thought of as a simple alternative to creating the vehicle in all possible positions.

This assumption, while simplifying the problem, also introduces certain drawbacks. For instance, using a circular footprint reduces the feasible domain of the solution, as it overestimates the space occupied by the vehicle in some configurations. Additionally, this approach does not consider kinematic constraints, which affects its maneuverability and turning radius. However, this simplification remains acceptable for the purposes of the simulation because it ensures a conservative approach, preventing unrealistic path solutions that could lead to collisions in a real-world scenario. Moreover, it significantly reduces computational complexity, making the simulation more efficient while still capturing the essential spatial constraints imposed by the EZ10's presence on the map.

A safety factor of 25% was taken as a precaution to really ensure that the EZ10 would pass safely. Of course creating a footprint of that size is computationally expensive as not only the footprint needs to be created at every iteration but also, every node forming the footprint has to be analyzed for collision avoidance. This is due to the fact that only checking the outer perimeter of the footprint for collision could neglect an obstacle in the middle of the road, i.e. the position of the vehicle would be such that the obstacle would spawn inside of the footprint without prior contact with the outer layer. To balance the computational expense and the collision avoidance, the footprint was build with various circles at some defined radius that would better analyze obstacle avoidance while not being to computationally expensive. The final footprint of the vehicle is illustrated as the orange concentric circles in figure 2.13, representing here the size, with safety factor, of the EZ10 at the start position.

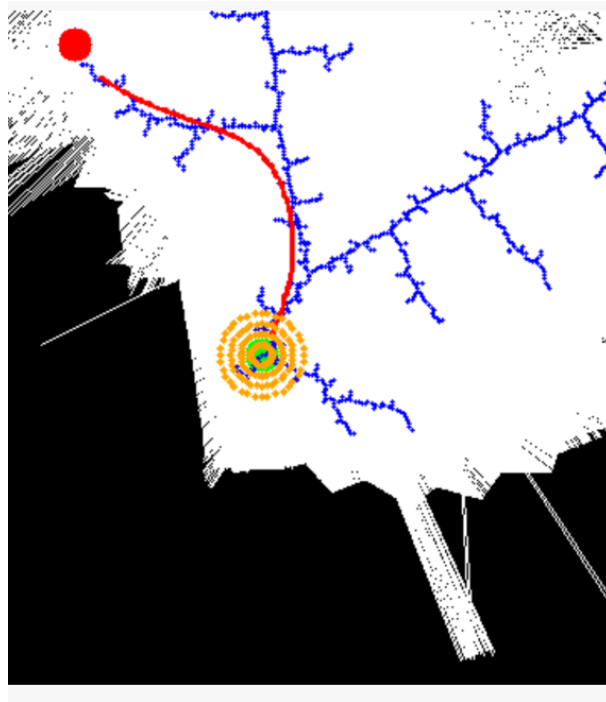


Figure 2.13: Footprint of the EZ10

As reference points, 4 strategic locations of the campus were selected, as they are laboratories, garages and road entrance as seen on figure 2.14. A single starting point was selected (Green point in figure 2.14), for the purpose of simulation, to ensure a proper comparison between the RRT, RRT* and RRT* with KD-tree.

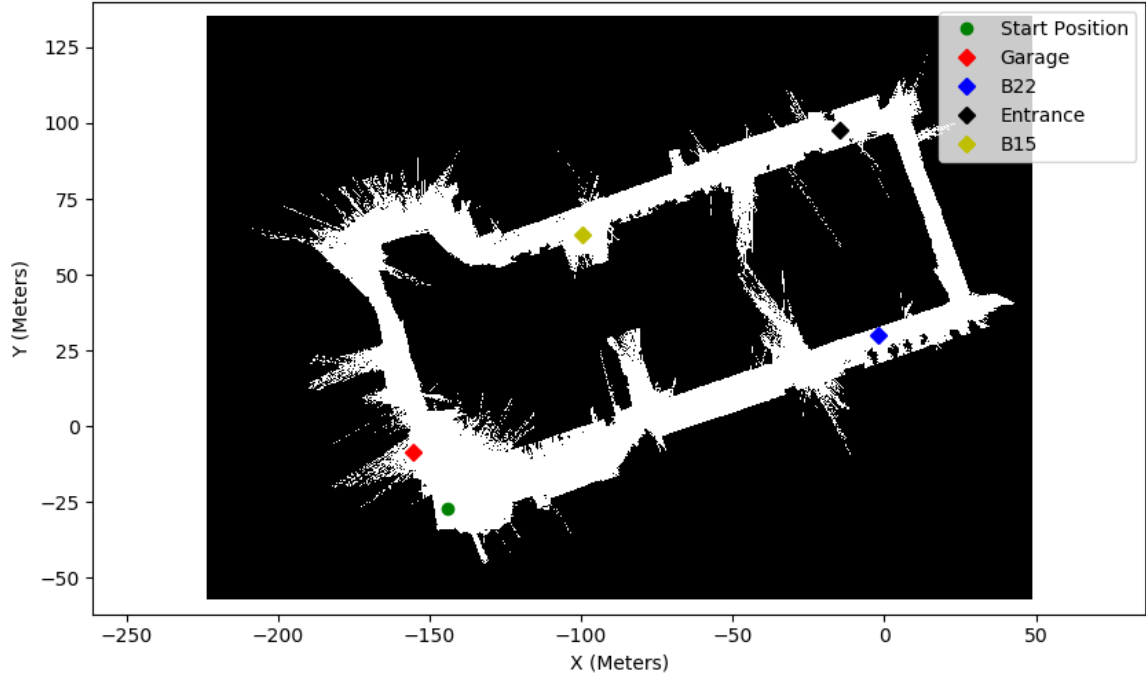


Figure 2.14: Start and Delivery locations

Every RRT was run 100 times from the start to each location, to provide a good approximation of the results. Table 2.1 shows the average results of the various RRTs on each of the 4 locations and starting from the same point.

	RRT		RRT*		RRT*KD	
	Time (sec)	Length(m)	Time (sec)	Length(m)	Time (sec)	Length(m)
Garage	0.98	35.5	1.25	28.1	3.21	26.1
B22	1.86	182.6	1.94	156.9	3.70	157.1
Entrance	8.78	304.9	10.46	259.9	8.48	263.5
B15	10.97	278.2	15.77	219.3	8.47	205.4

Table 2.1: RRT Comparison Table

From these results it is clear that the RRT* is a much needed improvement from the RRT, to find the shortest path possible to the goal, as the length is significantly lower in each case. However the rewiring necessary for the RRT* causes it to be slower than the RRT with bigger time differences as the path gets more complex (B15 requires 3 turns and is not on the middle of the road RRT* takes almost 50% more time to reach it).

The implementation of the KD-tree as the method for nearest neighbor research (instead of the brute force method of euclidean distance in the normal RRT*) results in somewhat similar path length with the RRT*, as the rewiring and optimization method

are the same. The KD-tree is optimized to reduce the computational time taken to look through a large amount of nodes and find the nearest neighbor, but it must also rebuild its tree often enough to stay updated, which is computationally heavy. That is why the use of KD-tree allows to reduce significantly the run time for longer paths. However, it is not the case in short paths where the small number of nodes is more rapidly looked through by the euclidean distance method than the KD-tree method.

The KD-tree implementation also renders the algorithm more reliable. Taking the values from the more elaborated path (Start-Entrance), a box and whiskers plot can be created for run time and path length. These plots are representative, in terms of consistency, of the other paths as well. As can be seen from figure 2.15 the time taken to run the RRT*KD algorithm is much more consistent than the RRT and RRT*. Moreover, in figure 2.16 the RRT presents a very spread out length that is characteristic of its random process whereas the path length is a lot more consistent in the RRT*KD.

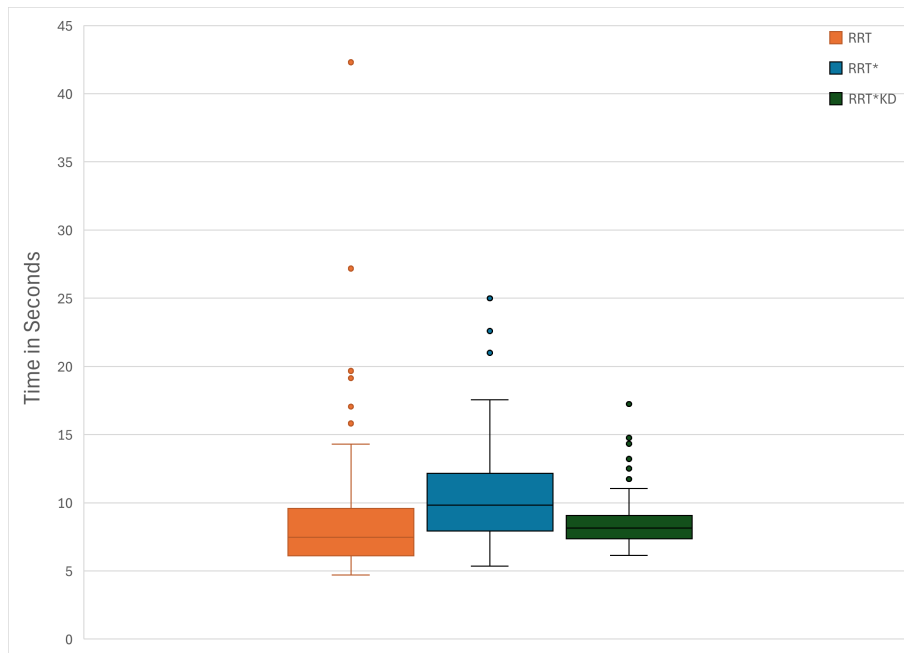


Figure 2.15: Run Time Box Plot

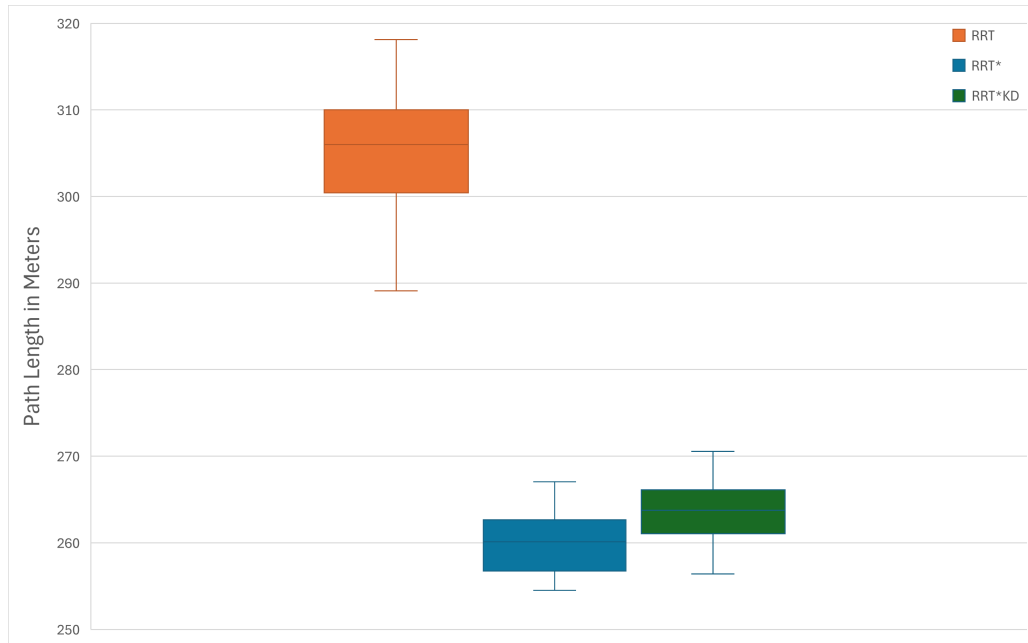


Figure 2.16: Path Length Box Plot

Despite the advantages of the RRT*KD algorithm, there are some limitations to consider. While the use of a KD-tree improves computational efficiency for long paths, it introduces additional overhead due to the need for frequent tree updates. This can make the algorithm less efficient for shorter paths where the brute-force Euclidean distance method performs better. Additionally, the inherent randomness of RRT-based algorithms means that while RRT*KD improves consistency, it does not guarantee an optimal path and may still result in suboptimal solutions in certain cases. Nonetheless, the RRT*KD is a more reliable algorithm and therefore the better algorithm to use to ensure consistent and good results to create the trajectory to be given to the motion controller.

2.4. Motion control

The obtained trajectory from the path planning acts as an input to the motion controller. As previously discussed in section 1.2, the controller chosen for this thesis' purpose is the MPC (Model Predictive Controller). In fact, the model-based approaches offer a better motion control solution when there is the possibility to model properly the physical problem. In the case of the MPC, it has been used extensively on autonomous vehicles for motion planning such as in papers [43] and [44]. The MPC is an iterative procedure over time and one iteration is usually composed of the measure of the current state of the vehicle, then a finite horizon Optimal Control Problem is implemented and solved based

on the information about the vehicle state and model, taking into account road and actuation constraints, and predicting future movements of any other traffic. This predicts the expected future behavior of the AV and its sequence of control actions. However, only the first control action of this sequence is fed to the vehicle actuators before the horizon shifts to the next iteration, and the previous process is repeated. This process is illustrated in figure 2.17

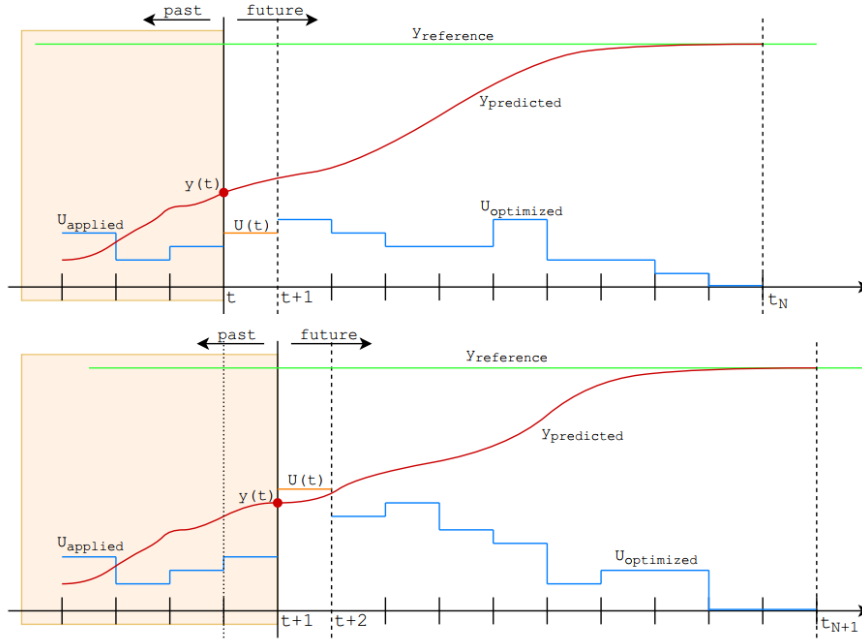


Figure 2.17: MPC iteration

2.4.1. Mathematical Model

The MPC's mathematical model used for this thesis is based on the one present in paper [44]. Some degree of customization have been made to accommodate for the EZ10 dimensions and constraints, as well as taking advantage of the EZ10 double steering system and modifying the weights to obtain a smoother motion. The model is run on MATLAB-Simulink and uses the ACADOS [45] toolkit for solving OCP (Optimal Control Problem).

The EZ10 has been mathematically approximated using a Single Track Model (STM) with double steering as shown in figure 2.18. The kinematic expressions that describe the motion of STM along the constrained XY space are expressed in 2.1.

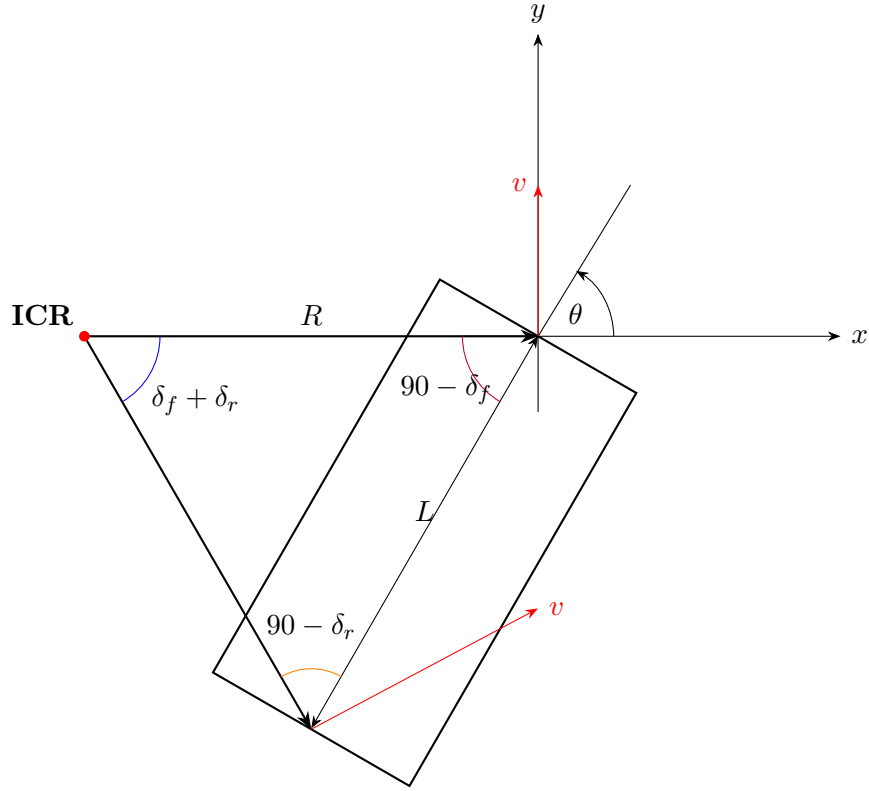


Figure 2.18: Vehicle Model Diagram

$$\begin{cases} \dot{x} = v * \sin(\theta + \delta_f + \delta_r) \\ \dot{y} = v * \cos(\theta + \delta_f + \delta_r) \\ \dot{\theta} = \frac{v}{L} * (\sin(\delta_f) + \tan \delta_r * \cos \delta_f) \end{cases} \quad (2.1)$$

The parameter L represent the total length of the vehicle. The variables v, δ_f, δ_r and θ represent the velocity of the vehicle, the steering angle at the front wheels, the steering angle at the rear wheels and the orientation of the vehicle with respect to the global XY plane respectively. Integrating 2.1 yields x, y and θ where the quantities x and y denote the position of the vehicle along the global XY frame. The variables v, δ_f and δ_r are the control variables to vehicle. Control actions have been placed on acceleration u_{acc} and steering rate ψ , to obtain a smoother motion, and are defined in 2.2.

$$\begin{cases} \dot{v} = u_{acc} \\ \dot{\delta}_f = \psi_f \\ \dot{\delta}_r = \psi_r \end{cases} \quad (2.2)$$

Taking the equations in 2.1 as $\underline{X}$ and those in 2.2 as $\underline{U}$ it is possible to define the total non-linear dynamic equation 2.3 of STM to be inputted as a dynamic equality constraint to the optimization algorithm.

$$\dot{\underline{X}} = f(\underline{X}, \underline{U}) \quad (2.3)$$

From the fundamentals of optimal control theory, a problem formulation based on a quadratic cost function can be defined. This cost function is then subjected to the constraints arising from the vehicle dynamics 2.3, and from vehicle state and control limits. This OCP is then propagated in time along a predefined receding time horizon T leading to a MPC based formulation as defined in 2.4.

$$\begin{cases} \min_{\underline{U}} & J(\underline{X}, \underline{U}) = \frac{1}{2} \sum_{k=0}^{N-1} \left(\|\underline{X}_k - \underline{X}_{\text{ref},k}\|_Q^2 + \|\underline{U}_k - \underline{U}_{\text{ref},k}\|_R^2 \right) + \frac{1}{2} \|\underline{X}_N - \underline{X}_{\text{ref},N}\|_P^2 \\ \text{s.t.} & \begin{cases} \dot{\underline{X}}_k - f(\underline{X}_k, \underline{U}_k) = 0 & (\text{Dynamic Constraints}) \\ \underline{U}_{\min} \leq \underline{U} \leq \underline{U}_{\max} & (\text{Control Constraints}) \\ \underline{X}_{\min} \leq \underline{X} \leq \underline{X}_{\max} & (\text{State Constraints}) \end{cases} \end{cases} \quad (2.4)$$

Where N indicates the number of discretization time steps distributed along the MPC time horizon and is fixed as 50 steps while T is set at 2.5s leading to a ΔT of 0.05s for the MPC.

$\underline{X}_N$ stands for terminal states of the ADV, while Q, R and P correspond to the weights for the MPC. The way-points that define the path along which AV needs to travel have been set in $\underline{X}_{\text{ref},k}$ in the problem definition 2.4.

For the MPC to represent properly the EZ10 the control and state constraint were based on the actual limitations of the vehicle and can be found in the expressions 2.5.

$$\left\{ \begin{array}{l} \dot{v}_{min} = -1.5 \text{ m.s}^{-2} \\ \dot{\delta}_{fmin} = -0.25 \text{ rad.s}^{-1} \\ \dot{\delta}_{rmin} = -0.25 \text{ rad.s}^{-1} \\ v_{min} = -0.1 \text{ m.s}^{-1} \\ \dot{\delta}_{fmin} = -0.314 \text{ rad} \\ \dot{\delta}_{rmin} = -0.314 \text{ rad} \end{array} \right\} \left\{ \begin{array}{l} \dot{v}_{max} = 1.5 \text{ m.s}^{-2} \\ \dot{\delta}_{fmax} = 0.25 \text{ rad.s}^{-1} \\ \dot{\delta}_{rmax} = 0.25 \text{ rad.s}^{-1} \\ v_{max} = 1 \text{ m.s}^{-1} \\ \dot{\delta}_{fmax} = 0.314 \text{ rad} \\ \dot{\delta}_{rmax} = 0.314 \text{ rad} \end{array} \right. \quad (2.5)$$

The maximum solving time for Simulink can be less the ΔT of 0.05s (20Hz) required by MPC. This means that in theory, the real-time integration of this motion planner is possible. Moreover, the 20Hz control frequency ensures the system can dynamically react to unexpected situations within a suitable reaction time.

2.4.2. Results

As previously discussed the waypoints that are given to the MPC are from the RRT*KD path planner. Figure 2.19 shows the simulations results obtained with the MPC, from the start position to the 4 various goals set around the map. The trajectory features a lateral error that indicates based on the color how far the MPC trajectory is from the original trajectory (given by the RRT*KD). As can be seen, the lateral error reaches a maximum of 0.7 meters. This value is acceptable as the locations of this maximum is generally along sharp turns which indicates the MPC is properly considering the vehicle constraints.

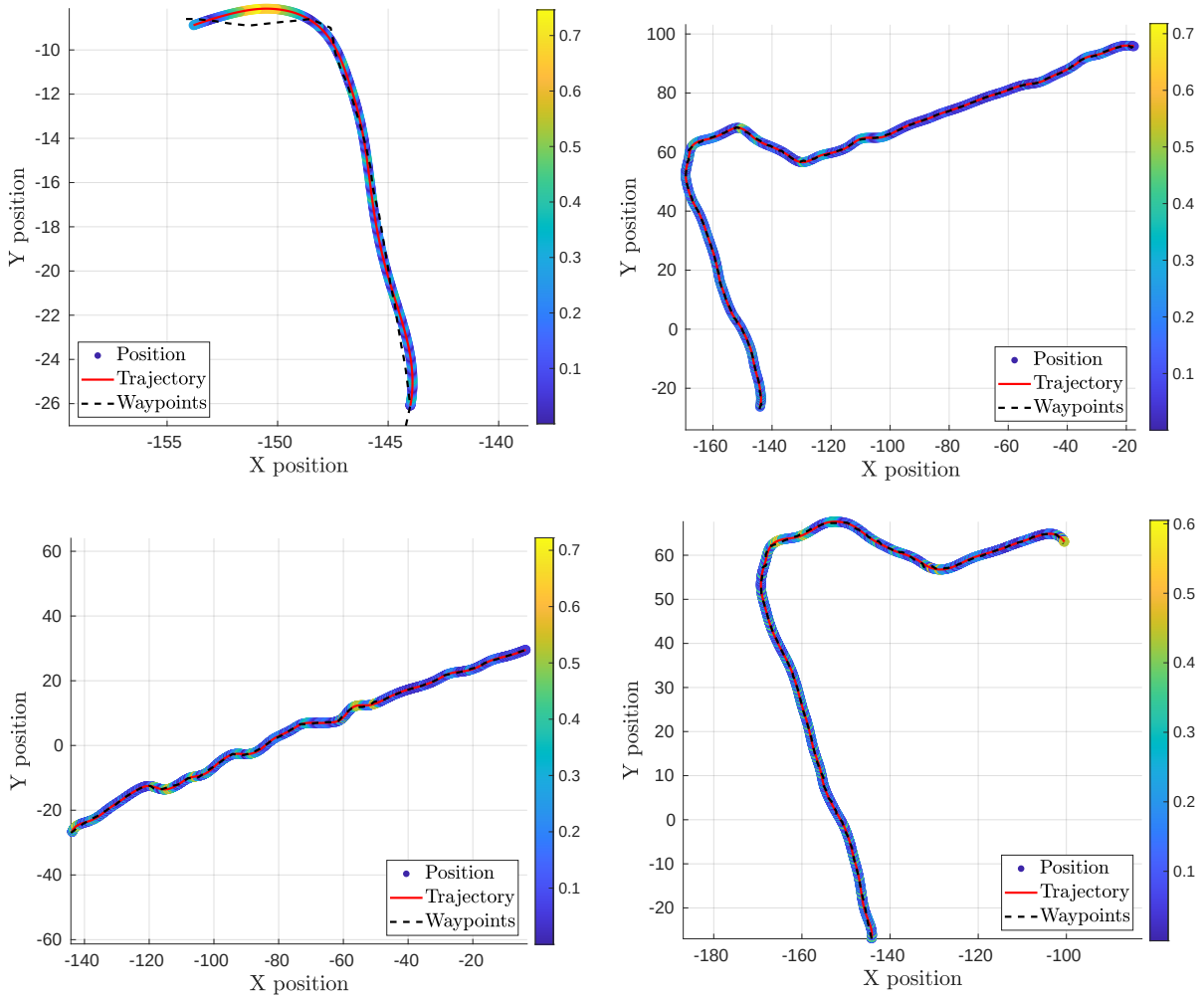


Figure 2.19: MPC results

The trajectory adopted by the motion controller needs to be feasible and be smooth enough to avoid jerk that could damage hardware components on the vehicle. As seen in figure 2.19 the trajectories present smooth curves without any sharp turns. This is due to the MPC's ability to generate a dynamically feasible trajectory rather than rigidly following predefined waypoints, significantly reducing sharp turns and making the motion more predictable and stable. This is done through several mechanisms. First, the MPC optimizes a trajectory over a predictive horizon while minimizing a cost function that accounts for factors such as deviation from the desired path, control effort, and jerk. Second, it enforces kinematic and dynamic constraints, preventing abrupt changes in velocity or steering, which contributes to a more natural and stable motion. Additionally, MPC incorporates feedback control to continuously correct deviations caused by external disturbances, such as road friction or wind, improving overall tracking accuracy.

However, some of the turns to reach certain locations on the campus are quite sharp, and the EZ10 is limited (as all vehicles) to a certain turning radius. This is why it is important to have the trajectory being as smooth as possible and it must take longer and wider turns to accommodate the size of the EZ10. In figures 2.20 and 2.21, the obtained MPC trajectory has been overlayed onto the map. Using a similar method to the one of figure 2.6, it is clearly visible that the trajectory fits well the map constraints.

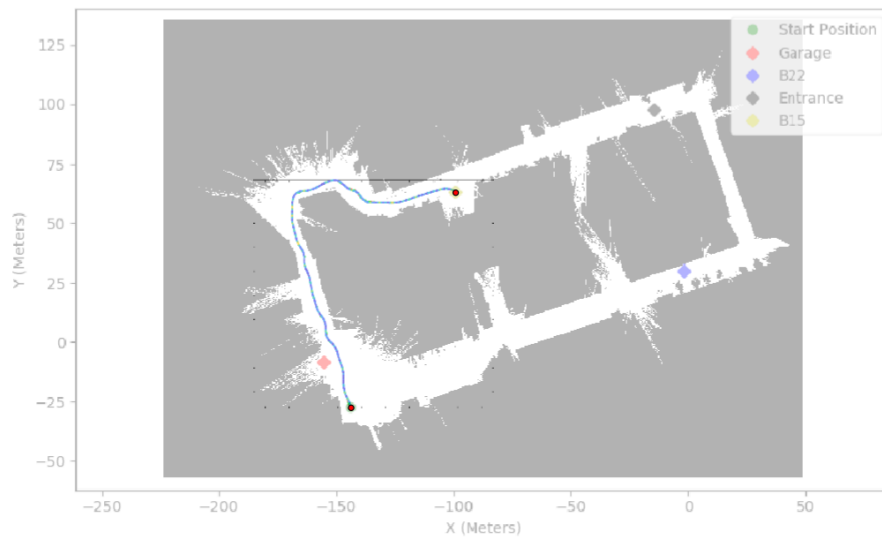


Figure 2.20: Start to B15 MPC Trajectory Overlay

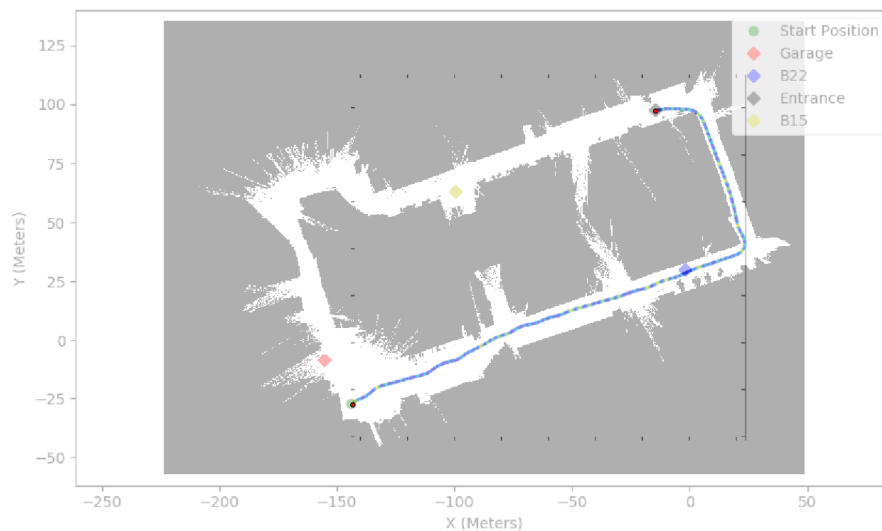


Figure 2.21: Start to Entrance MPC Trajectory Overlay

It is important to note that the MPC is not aware of its environment which means that there is no guarantee of safety. It is aware of the vehicle it is representing and of its

dynamic constraints but not of the surrounding environment. That is why it is possible to see an unfeasible part of the trajectory at the right upper corner of the map in figure 2.21. This trajectory is unfeasible, in a real-life scenario, due to the environment and not the dynamics of the vehicle.

However, the MPC has the capacity to dynamically adjust its trajectory based on obstacle detection. While this function was not explored in this work, in a real world experimentation, the assumption is that the vehicle will be communicating its LiDAR data with the MPC and it will be able to adjust dynamically the environment constraints and re-evaluate the predicted trajectory over the horizon to take into account the newly found obstacle. Therefore in a real world test where the EZ10 is actually trying to follow this path, the MPC would be able to dynamically re-evaluate its trajectory, unlike what is being shown in this work.

This promising performance is why the MPC is generally chosen as a motion controller for ADVs. It is able to not only consider the physics of the vehicle but also adapts to dynamically changing environment which is critical in an urban environment. This chapter covered the navigation part of the vehicle, however for the vehicle to navigate it must be given a goal to reach. The following chapter covers how the user is able to communicate with the EZ10 to give it a goal to reach and other instructions in a simple and innovative implementation.

3 | Vehicle-to-User Interaction

In section 1.3, the history and new technology of Human-Machine Interfaces (HMIs) have been discussed. An emphasis has been given to the cost of newer, more complex HMIs that are not always justified for some use cases. That is why the concept of QR code integration in a HMI systems has been introduced. QR codes are easy to use as they are a part of general life. They can contain enough information to have a great amount of customization and thus can be applied to various cases.

Defining a working prototype, as done in this work, is a cheaper task due to the sensors, needed to enable QR code detection and reading, being readily available and relatively cheap (proximity sensor and camera module). These sensors coupled with a micro controller such as a Raspberry Pi are enough to create a standalone system to enable QR detection and communicate with the vehicle as will be discussed in this chapter.

This chapter will cover the innovative design and implementation of the created Vehicle-to-User (V2U) system. A brief overview of the design will be shown. Then, the implementation of QR detection within the ROS system will be discussed. After that, the user input required to interact with the vehicle will be explained. Finally, the overall system and solution implemented will be defined.

3.1. Hardware Design and Requirements

The EZ10 is equipped with sensors that could serve as proximity and image detection, however the cameras are located too high on the vehicle and the energetic cost of leaving the LiDARs running and communicating with the vehicle would severely impact the battery life. That is why a standalone system of micro controller with sensors would be less energy consuming. Moreover this system being standalone, would not require permanent fixing to the vehicle and thus could be portable and used in other cases.

For this system, a case has to be designed to contain both the micro controller and sensors necessary for the detection and reading of the QR codes. The micro controller chosen is the Raspberry Pi (RPi) and the image reading is a Raspberry Camera V2

chosen for its compatibility with the RPi. The detection sensor is in charge of informing the presence of the user in front of the box to avoid the camera feed being constantly sent to the EZ10. This allows the system to be less intrusive (respect of the privacy [46]) and reduce the computational load. To have a non-intrusive design, after initial inspection, it was decided that placing the stand-alone module behind the windscreen to allow QR reading was the better solution.

This decision presents some challenges as the windscreens present on the EZ10 are coated and coil heated. This required some experiments on various sensors to determine which was the better performing. After some research [47] it was found that the better sensors to test were an infrared sensor and a 5.8 GHz Doppler effect microwave sensor. As the coils were certainly going to be a problem, the black surrounding area of the windscreen (figure 3.1 to be changed) was also tested. Each sensor was tested for 20 detections, 10 where movement was happening outside the glass and 10 where no movement happened.



Figure 3.1: EZ10 Windscreen

The results of the experiment are represented in form of confusion matrices as table 3.1 & table 3.2. Both the infrared and microwave sensor were able to detect through the transparent part of the glass but only the microwave was able to detect through the opaque black surrounding of the windscreen. Moreover the detection behind the opaque part was more reliable than the one behind the transparent glass. This is probably due to the coils which due to their very close spacing can confuse the sensor as it needs to travel

2 distinct medium at once (opaque for the coil and transparent for the glass). Therefore the microwave sensor was chosen to be used on the extremity of the window.

Transparent Glass - Infrared Sensor

	Predicted Positive	Predicted Negative	Total
Actual Positive	6	3	9
Actual Negative	4	7	11
Total	10	10	20

Opaque Glass - Infrared Sensor

	Predicted Positive	Predicted Negative	Total
Actual Positive	0	1	1
Actual Negative	10	9	19
Total	10	10	20

Table 3.1: Confusion Matrices for The Infrared Sensor

Transparent Glass - Microwave Sensor

	Predicted Positive	Predicted Negative	Total
Actual Positive	7	5	12
Actual Negative	3	5	8
Total	10	10	20

Opaque Glass - Microwave Sensor

	Predicted Positive	Predicted Negative	Total
Actual Positive	8	1	9
Actual Negative	2	9	11
Total	10	10	20

Table 3.2: Confusion Matrices for The Microwave Sensor

The design of the case revolved around the Raspberry Pi and its two sensors. The main requirements were:

- Fit the Raspberry Pi while providing access to input and output port.
- Include fan mount to improve cooling of Raspberry Pi.
- An enclosure for the camera to be pointed at the windscreen.
- An movable mount for the microwave proximity sensor to reach the extremity of the window for better detection reliability.

- All of this requirements inside a portable package, therefore a lightweight small dimensions case.

The design and dimensions chosen can be seen in figures 3.2, 3.3 and 3.4.

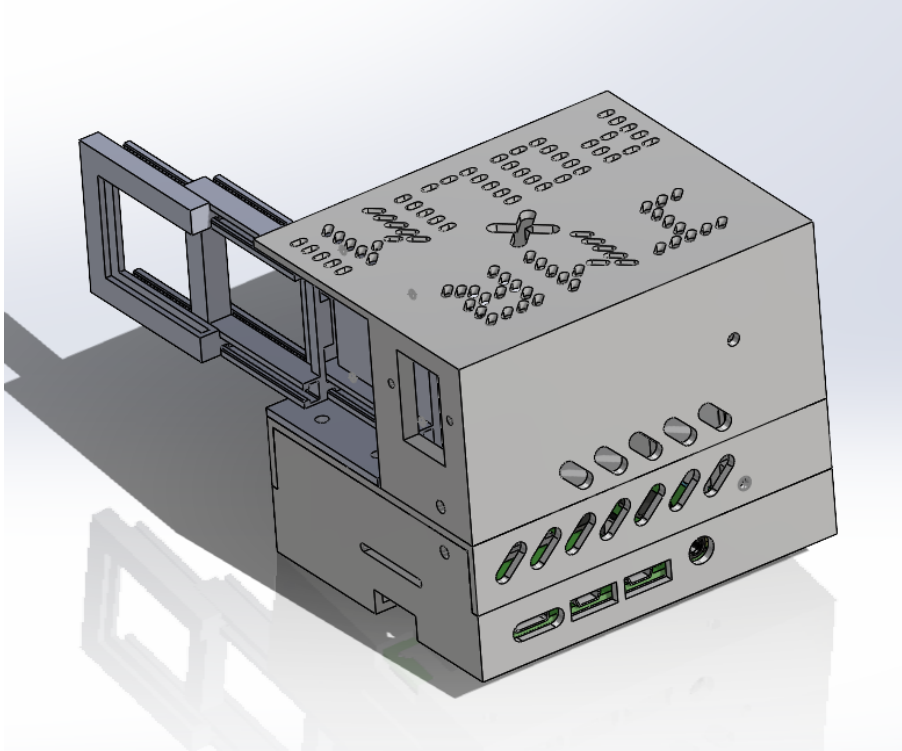


Figure 3.2: Case ISO View

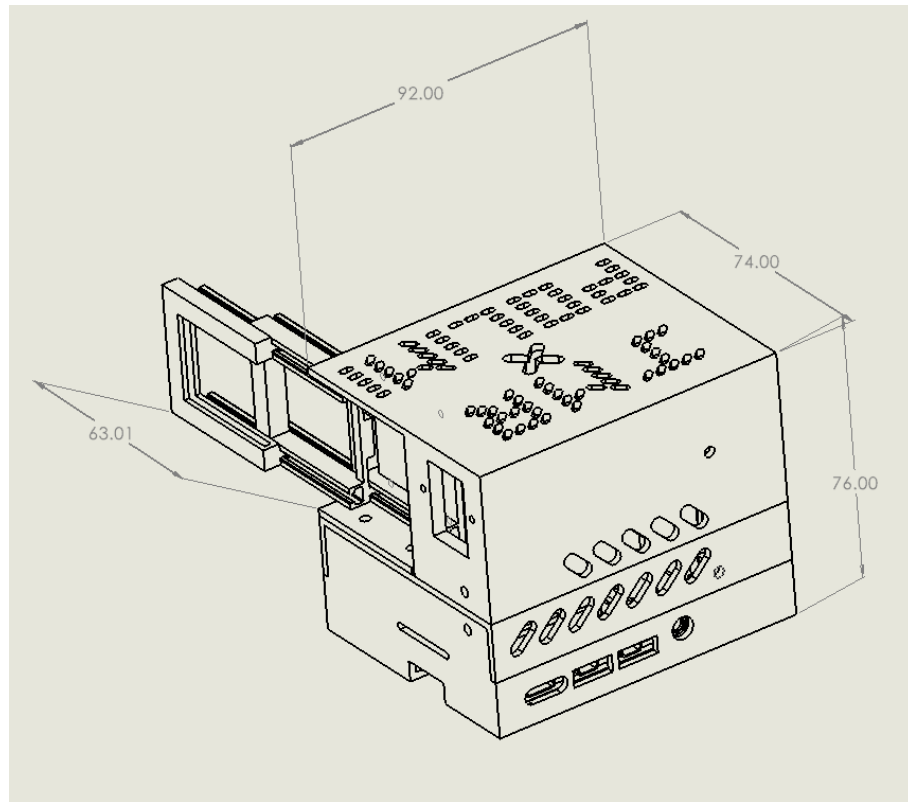


Figure 3.3: Case Dimensions in Millimeters

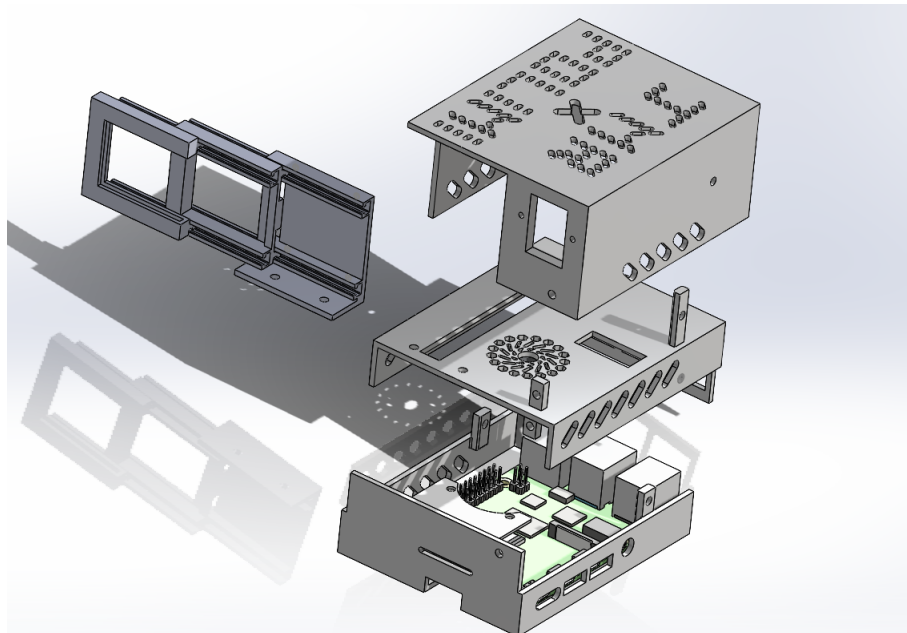


Figure 3.4: Exploded View of the Case

As can be seen, the Raspberry Pi seats at the bottom of the case which also features

opening for the various connections, the mount for the cooling fan and more openings for air circulation. The second layer of the case has openings for the fan and cables as well as mounting points for the third layer of the case. The third layer is where the camera and microwave sensor will be placed, facing the window. A slider mechanism has been developed for the microwave proximity sensor to allow it to reach the window extremity and avoid the heating coils of the window, which interfere with the proximity sensing.

As the case is made to be portable, there is a need for some type of support. This support was thought of as a simple 'box' with a large front surface for properly adhering to the window without need for additional mounting support. This support can be seen in figure 3.5

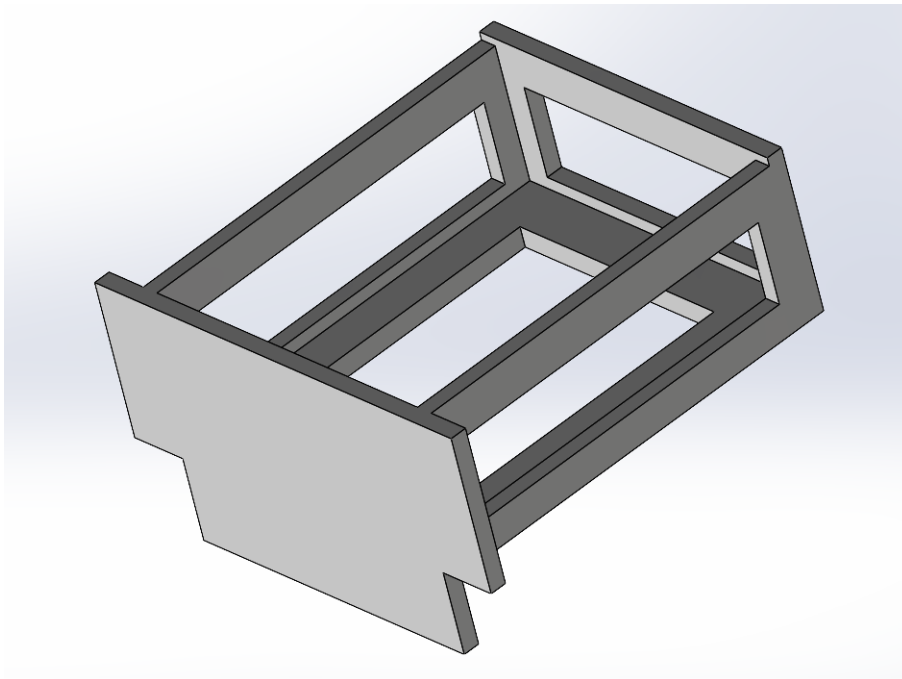


Figure 3.5: EZ10 Window Mount

The entire design of the case and the mount has been 3D printed (see figure 3.6) and is confirmed to be functional and up to dimensions and has been placed on the bus for testing as seen in figure 3.6 & 3.7.

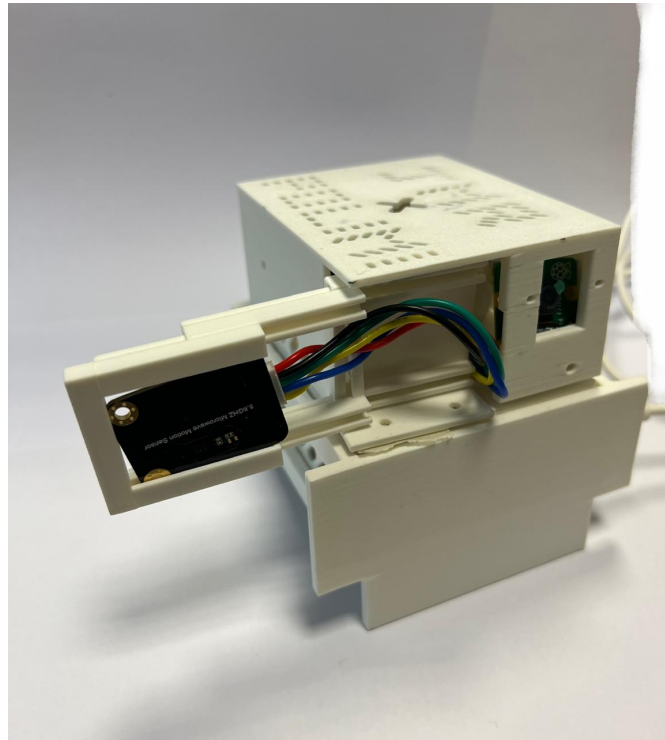


Figure 3.6: 3D Printed Case



Figure 3.7: Mounted 3D Printed Case

3.2. Software Design and Requirements

The V2U system created relies on the Raspberry Pi (RPi) micro controller. The RPi has been configured to run the Ubuntu operating software with a ROS integration. This decision is to facilitate the communication between the V2U system and the vehicle, as the later is also equipped with Ubuntu and ROS.

ROS (Robot Operating System) facilitates communication between components of a robotic system through a decentralized messaging model. In this model, each process running on the system is referred to as a node which communicate by sending messages over topics. This architecture separates the nodes that produce information (publishers) from those that use it (subscribers), making the system scalable and modular. Therefore, a publisher is a node that has data to share and publishes messages to a specific topic, while a subscriber is a node interested in a particular type of data subscribes to the corresponding topic. The ROS Master does not handle the actual data transmission but plays a crucial role in helping nodes find each other and establish peer-to-peer communication channels for efficient message exchange.

For instance, in the V2U system, the Raspberry Pi (RPi) is configured as a slave node that interacts with the vehicle (the master node) via ROS topics. As an example of communication in this system, when the camera captures image data (triggered by the proximity sensor), the RPi publishes this data to a designated topic (e.g., `/camera_feed`). The vehicle, as the master node, subscribes to the `/camera_feed` topic.

By leveraging the ROS publish/subscribe mechanism, the V2U system ensures that all components—from sensor activation to QR code decryption—operate seamlessly in real time. This robust communication framework is key to the efficient interaction between the Raspberry Pi and the vehicle, ensuring that tasks are executed as soon as sensor data becomes available.

The detection algorithm on the RPi has been configured to immediately run when the RPi is powered on, thus not needing a user input to make it work. At startup, the proximity sensor is enabled and waits for a detection which will trigger the camera to start its feed for 7 seconds. This duration is due to the window of the EZ10 being very difficult to penetrate even for the microwave sensor and thus it is not always able to pick up on little movement (which would be useful and thus as long as something is detected the camera would be on). The image data perceived by the camera is then published from the RPi to the vehicle via ROS. Once the image data is published, the vehicle automatically receives it, by subscribing to it, allowing further processing—such as employing the Z-Bar

library to recognize and decode QR codes. This library is able to decrypt the QR code present in the image as a string. This flow of information is illustrated in figure 3.8.

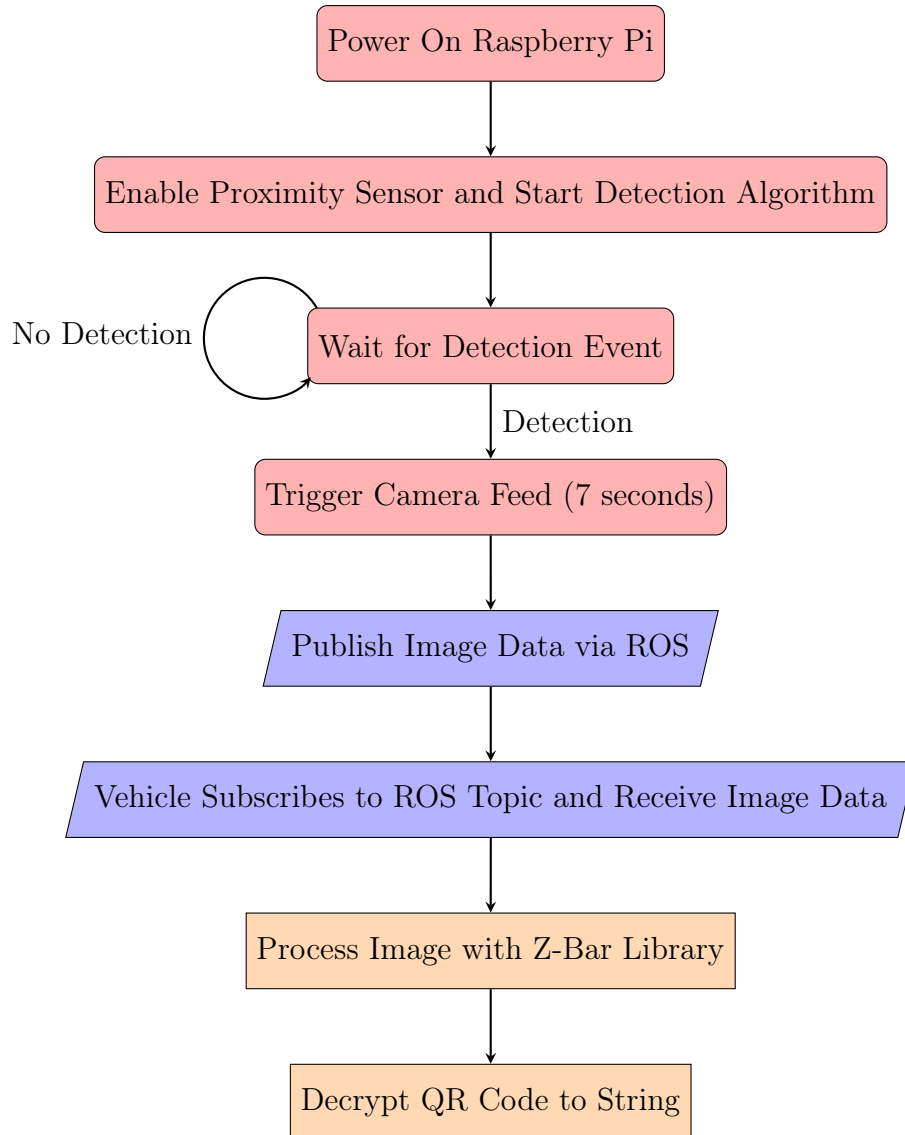


Figure 3.8: Flowchart of the V2U Communication

An algorithm has been created to use this string of various characters and assign each one to a specific value for the use case. This algorithm is the State Machine which will be discussed in chapter 4.

This part of the chapter covered how the V2U system handles the user input and transmits it for decryption. The user still needs to create the QR code. That is why a user interface needs to be created that allows the user to create a QR code that is readable by the decryption algorithm and provides it with the tasks the user requires

from the EZ10.

3.3. User Interface

Many HMIs have been discussed in section 1.3, with various way the user could provide input. For this V2U system it is needed to generate a QR code to provide instructions to allow the end-user to interact with the bus for delivery (needs more instruction than a simple reception). A user-friendly system can be simply made with a webpage. Of course that web page should present the user with the possibility to customize their instructions (see figure 3.9. In this work's case, the user is able to choose between reception or delivery, allowing a better and more comprehensive control of the ADV capabilities. In both cases they are able to request the door or the ramp of the EZ10 to open. And if the delivery action has been chosen they are able to select from the 4 locations that were presented in chapter 2.



Figure 3.9: User Interface

The webpage is linked to a python code that will use the inputted information from the user and creates a QR code based on that information. This qr code is created using the qr code python library. Once the button 'Generate Qr Code' is tapped, the python code is ran and the QR Code is displayed on the user's device to be shown to the camera in the

V2U system. To make the page available to everyone, it was uploaded on the internet via GitHub and OnRender and is available at <https://hmi-qr-generator.onrender.com/>.

The V2U system has been designed and its communication with the vehicle has been explained. As stated before, the user creates the QR code on the interface and shows it to the V2U system where it is sent to the vehicle. This is valid for every user of the V2U system, contributing to streamline parcel or package last-mile delivery. Once the vehicle gets the QR code and decrypts the data, it has to modify its behavior accordingly to what is present on the QR code. This is where behavioral planning is important. In fact, the data provided by the QR code determines the various state changes the vehicle must do as will be seen in the following chapter.

4 | Behavioral Planning

In the previous chapters, modules developed for an autonomous delivery vehicle during this thesis work have been presented. In this chapter, the role of behavioral planning as a way to manage the vehicle's diverse operating conditions will be explained. Behavioral planning not only serves as a bridge between perception and motion control but is also crucial at managing user input. It interprets sensor data, uses the user input, and determines high-level maneuvers, ensuring that the vehicle operates safely under varying conditions. By using a hierarchy of behaviors, the system can seamlessly transition between different modes such as planing, cruising and stopping.

This is why a robust state machine is fundamental for safe and efficient decision-making, and how multiple states enable the system to handle complex scenarios dynamically. A state machine with multiple states allows the system to adapt quickly to new information or changing circumstances and at the same time clearly defined states ensure that appropriate safety protocols are activated without delay.

4.1. The Finite State Machine

A Finite State Machine (FSM) represents the vehicle's operational modes as discrete states, with transitions defined based on sensor inputs, system status, and user inputs.

This enables the system to execute tasks in a modular and predictable manner. Moreover, isolating behaviors into individual states makes it easier to test and verify each behavior under controlled conditions. There needs to be well-defined transitions between states to prevent conflicts, ensuring that the vehicle can switch behaviors smoothly without abrupt changes that could compromise safety. The state machine is generally composed of 3 elements:

- States, that define the specific behaviors or modes (idle, cruise, etc.) and encapsulates unique control logic tailored to that behavior.
- Transitions, that dictate the conditions under which the system shifts from one state to another and are typically triggered by sensor inputs, events, or predefined rules.

- Actions, which are the tasks executed upon entering, during, or exiting a state such as adjusting speed, opening doors, or activating sensors.

For every FSM, an initial state needs to be defined, it represents the first state to be entered when the system is started and it is maintained until its transition conditions are satisfied. Furthermore, for each state, it is possible to set an initial action, executed only once when the state is entered, and a recursive action, which is repeated while the state is operative.

Figure 4.1 represent the simplest state machine possible with 2 states. The state machine will do the transition from state A (initial state) to state B when the conditions for the transition have been met. Then the actions that are pertinent to state B will be executed.

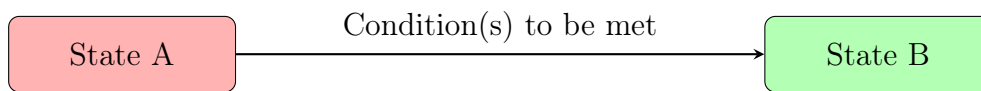


Figure 4.1: Basic State Machine Diagram

It is important to not that the FSM can run only one finite state at a time, meaning there is no possibility for parallel states to run during the same iteration. Moreover, the transition from one state to another does not need to be reversible, and the passage to another state can require to return to the initial one.

4.2. A ROS Finite State Machine

For effective operation, the state machine must be seamlessly integrated with other modules such as perception, localization, and control. Moreover, the events triggering state transitions must be clearly defined, ensuring that the vehicle reacts promptly to external changes.

For this work, the FSM was written in python with the use of the StateMachine library which offers integrated states and transitions to facilitates the FSM setup. This FSM communicates with various sensors of the system such as the proximity sensor using ROS nodes (publishers and subscribers).

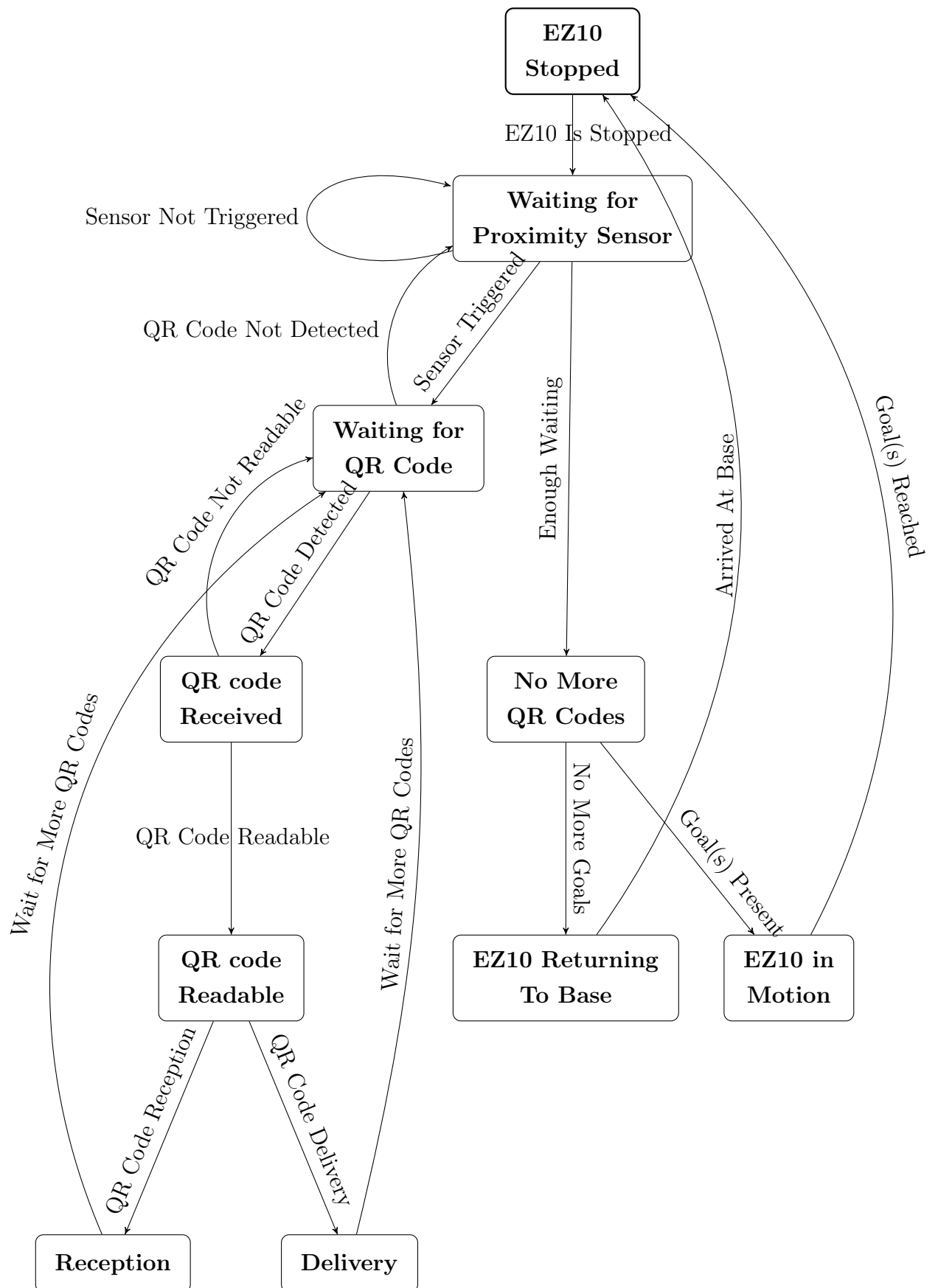


Figure 4.2: EZ10 Finite State Machine Diagram

Figure 4.2 represents the flow of the FSM created for the EZ10 to implement the navigation and V2U system. The FSM is composed of 10 states and the initial state is the *EZ10 Stopped* state where the bus is parked. This state is also the final state of the EZ10 as it needs to come to a stop for the state machine to end. The *EZ10 Stopped* state only verifies that the EZ10 is at zero velocity (via the ROS information from the EZ10's wheel encoders) and thus stopped. Once it is verified, the transition to *Waiting for Proximity Sensor* is done.

The *Waiting for Proximity Sensor* state is getting information published via ROS from the proximity sensor implemented into the V2U system. It has 3 possibles outcomes:

- The proximity sensor is not triggered and the waiting time is below the threshold -> Stay in *Waiting for Proximity Sensor*.
- The proximity sensor is not triggered, the doors are closed and the waiting time is above threshold. This would mean the packages have been loaded and the additional time given in case some person would also add to it has passed. -> Transition to *No More QR Codes*.
- The proximity sensor is triggered -> Transition to *Waiting for QR codes*.

The *Waiting for QR Code* state receives information from the `/qr_codes` ROSTopic generated by the `qr_detector` library. The qr code image received from the camera via the `/image` ROSTopic of the V2U system is decrypted to be sent as a script. Moreover the state of the doors is monitored via a ROSTopic transmitting the value of the CAN id representing the state of the doors. In *Waiting for QR Code* there are 2 possible scenarios:

- No QR code was detected, the doors are open and the waiting time has passed. This would mean there are still objects to be loaded in the EZ10. -> Transition back to *Waiting for QR Code*.
- A QR code was detected -> Transition to *QR Code Received*.

The state *QR Code Received* receives script information of the QR code. The information has been split in a way that in *QR Code Received* the first 5 characters of the QR code must match as a sort of recognition pattern, a small step at reducing the risk of processing a QR code that is not generated for the EZ10. In *QR Code Received* there are 2 possible scenarios:

- The first 5 characters of the QR code have been matched. The QR code does conform to the predefined design. -> Transition to *QR Code Readable*.
- The first 5 characters of the QR code have not been matched. The QR code does

not conform to the predefined design. -> Transition back to *Waiting for QR Code* to wait for a QR code that is readable.

The state *QR Code Readable* receives the next character of the string which informs if it is a delivery or a reception. In this case as the string has already been verified there is no need for a back transition. Therefore from *QR Code Readable* if the character is a 0 the transition will be to *Reception* whereas if the character is 1 the transition will be to *Delivery*.

In both states *Delivery* and *Reception* the next character of the string is evaluated to decide if to open the door or the ramp. The actuation of these two is guided by sending a message via ROS to the CAN of the EZ10 thus actuating the door or ramp. However this does not take into account if they were opened or closed it will just actuate them. To remedy this problem a 'fail safe' has been implemented where if the door or ramp is asked to be actuated, the state of the door or ramp will be evaluated and if it is already open the actuation will be neglected, as the QR code is meant to only open and not to close. In the case of *Reception* once the door or ramp has been opened the EZ10 transitions back to *Waiting for QR Code*.

In the *Delivery* state once the door or ramp has been opened, the next characters of the string, where the goals are present are evaluated and appended to the list of goals for delivery. Once this is done the EZ10 transitions back to *Waiting for QR Code*.

The state *No More QR Codes* verifies if there are any delivery goals in the goals list. There are 2 possible transitions:

- If there is no goal in the list, it is assumed the EZ10 has completed its job. Therefore the RRT*KD path from the last goal to the Start position is calculated and sent to the MPC. -> Transition to *EZ10 Returning To Base*.
- If there is at least one goal in the list, it needs to be processed. If there is only one goal then it is immediately processed by the high resolution RRT*KD. If there are multiple goals they have to be ordered by trajectory length first. The cost of the path from the EZ10's present location to the goals will be evaluated using a low resolution RRT that is fast and reliable. The goals will then be ordered by lower cost and the goal with the lower cost will be processed by the high resolution RRT*KD. When the trajectory has been found by the RRT*KD it can be sent to the MPC. -> Transition to *EZ10 in Motion*.

The state *EZ10 in Motion* gets as an input the trajectory from the RRT*KD. It then has to start the controller which is a MPC that will then through ROS and the

topic `/cmd_vel` give the proper velocity and steering inputs to make the EZ10 follow the obtained trajectory to the goal. Once the goal has been reached, the EZ10 will transition back to the initial *EZ10 Stopped* state.

The state *EZ10 Returning To Base* gets the EZ10 back to its base position. That is, the EZ10 has completed all of its deliveries and no goals have been added therefore to avoid having it stationed at one of the goal location where other vehicles might want to offload packages, the EZ10 returns to a location that is safe and away from disturbing the flow of traffic and other deliveries. The trajectory from the RRT*KD is given to the MPC present in the state (same as in the moving state). Once the vehicle has reached the base position and comes to a stop, the EZ10 transitions to the initial *EZ10 Stopped*.

5 | Results

All of the modules that were used in this work were presented in the previous chapter. The result presented in these chapters while positive, were individual module verifications and independent of other modules in the autonomous delivery system pipeline as the state machine had not yet been formulated. This chapter is to ensure that these results are replicable in the real world and to ensure the functioning of the whole system connected.

An important parenthesis, the MPC motion controller is based on the vehicle's model and calibrated to ensure that all commands given are able to be performed by the EZ10. The analysis of the results has demonstrated that the simulation environment is capable of accurately predict the absolute trajectory and the control history performed by the real vehicle. The MPC was even tested on its own in the EZ10 with GNSS-RTK and results were promising. However, an unrelated mechanical failure on the EZ10 prevented any experimental testing of the motion controller real-life capabilities. Therefore, the experiment will test the proper functioning of the V2U system and the state machine on the EZ10 with real life results, but the MPC will be run as simulation only.

This chapter is thus going to present the results obtained in different scenarios that the EZ10 could be subjected to in real-life use.

5.1. Simulation Settings

As mentioned the real vehicle could not be moved, therefore the experiment was conducted in the same location. The setup for this experiment is as follows:

- The EZ10, operating on Linux/Ubuntu, is made the ROS Master. From there, the CAN network was launched, enabling communication with the CAN network of the vehicle with the ROS environment.
- The V2U system was connected to the vehicle via Ethernet as a ROS Slave. This enabled the system to access and publish the necessary topics in the same ROS environment. The V2U was able to transmit the proximity sensor information and the image captured by the camera.

- A laptop was also connected as a slave to run the state machine and all other pertinent algorithms such as the QR code decoder, RRT*KD, MPC and door/ramp commands. This was a preferred method for debugging purposes and ease of use.

Figure 5.2 illustrate the connections between the various systems for this experiment.

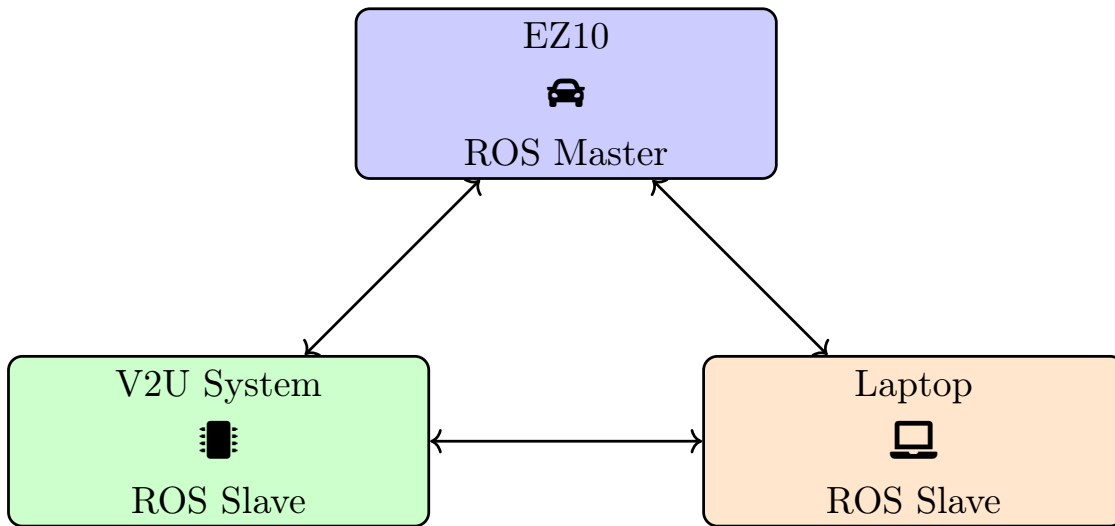


Figure 5.2: Overall System Communication

In the next sections of this chapter, the experimental data, which was recorded on bags (ROS bags capable of recording messages under all the topics for a certain period of time), will be presented. A useful tool to visualize the content of a bag is with `rqt_bag` which is an embedded visualizer of ROS bags. This simple tool allows to display the topics in a bag over a timeline and shows at what time each topic was published. It is able to display some plot of specific message types. However in this work some of the published messages were strings which is not supported in the plot function. Therefore, by simply accessing the content of the topic, it is possible to read the published message at that specific timestamp. Figure 5.1 shows the typical bag to be expected throughout this chapter.

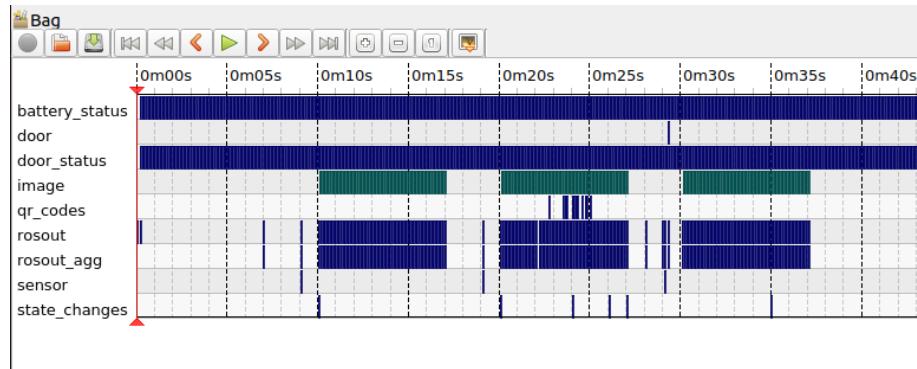


Figure 5.1: Bag Topics Visualization

The topic's name is present to the left of the window and then it is possible to see each time a topic has been published with the vertical lines along the timeline of the bag. The topics of interest for the results are:

- The *door* topic is published from the EZ10 to represent when the door command is sent to the CAN to actuate the door. It only reflects the actuation from the state machine. Therefore if a manual actuation of the door happens (with the dedicated button inside or outside of the bus), it will not appear in the door topic.
- The *door_status* topic is published from the EZ10 at 10Hz and constantly informing, with an integer value, if the door is opened (1), closed (2) or in movement (0).
- The *image* topic is sending images taken from the camera of the V2U system at 10Hz to be analyzed and decoded if they contain a QR code.
- The *qr_codes* topic is published from the laptop and corresponds to each time a QR code has been detected inside of the *image* topic and decoded. The message published is the decoded string of the data inside the QR code.
- The *sensor* topic is published from the V2U system and represents the triggering of the proximity sensor. This a delay of one second has been placed between when the sensor is triggered and the *image* topic starts publishing to allow for the camera system to properly start.
- The *state_changes* topic is published from the laptop and represents every time the vehicle is entering into a new state. The output of this topic is the name of the state with the timestamps at which it has entered the state. An important note is that the first *EZ10 Stopped* state (initial state) does not appear to be published as the time taken to initialize the node is higher than the time in which the state gets published.

The visualization of the data will only consist of these topics as there is no need for the battery status and the rosout topics are a repetition of some of the aforementioned topics. The state changes mentioned can be better visualized in figure 4.2. For additional clarity, plot of the various events happening during the timeline will be presented. The different states have been given a numerical value to better visualize the transitions. These values can be found in table 5.1

State	Assigned Value
EZ10 Stopped	2.5
Waiting for Proximity Sensor	3
Waiting for QR Code	3.5
QR Code Received	4
QR Code Readable	4.5
Delivery	5
Reception	5.5
No More QR Codes	6
EZ10 in Motion	6.5
EZ10 Returning To Base	7

Table 5.1: State-to-Value Mapping

Various scenarios were studied in the setup described above, adding more steps to be taken in each scenario, increasing the complexity of the task each time. The path that would have been taken by the EZ10 is simulated with the MPC controller taking into account the vehicle model of the EZ10 as presented in section 2.4.1.

5.2. Accidental Triggering of Proximity Sensor

This section will show how the state machine handles when the proximity sensor is activated but no QR code is scanned. This has been done as the proximity sensor could be activated by something or someone without the intention of scanning a QR code. Therefore it is necessary to check that the FSM has an adequate response.

Figure 5.3 shows a section of a recorded bag where a delivery was created to be dispatched to a goal. In this section the two first failed detection are visible. It is visible that the sensor was triggered twice which in turn started the publishing of the image topic and made the FSM transition. No QR codes were detected as can be seen by the absence of vertical lines as well as the door was not triggered by the FSM.

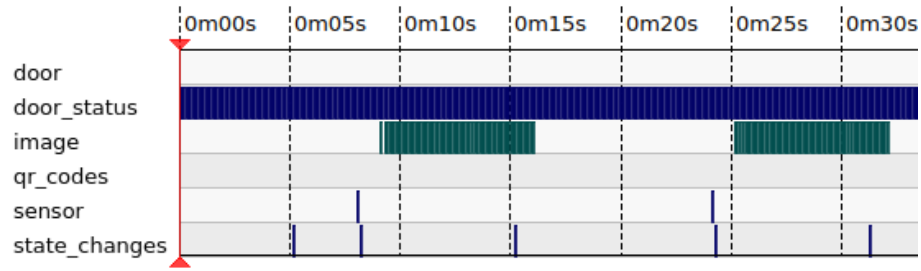


Figure 5.3: Failed Detection

To better focus on the necessary topics to see, figure 5.4 shows a more detailed plot of the bag data. As can be seen the door status is valued at 2 signifying that the door remained closed for this section of the bag. It is clearly visible that as soon as the sensor is triggered, the FSM transitions from the state value 3 to 3.5. Here the various state changes are represented by green squares and the state corresponding to the y-value 3 is *Waiting for Proximity Sensor* and the y-value 3.5 is *Waiting for QR Code*.

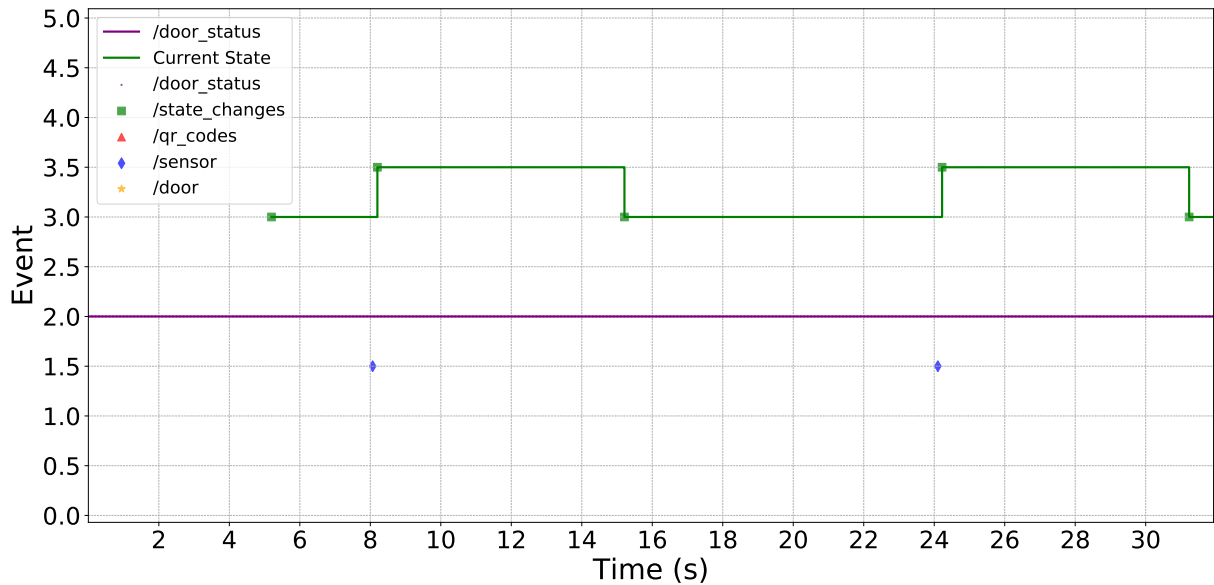


Figure 5.4: Failed Detection Event Plot

Therefore the FSM reacts as expected and when the time for QR detection is over, even if there are no QR codes detected, it goes back to waiting for the sensor to be triggered.

5.3. One Goal Delivery With Long Loading Time

As mentioned previously the FSM only opens the door of the bus and does not close them. This is to allow the user to take their time while loading. When the user finishes with the loading they are able to press on the door button present on the bus. This also avoids the EZ10 taking responsibility if the user was not finished and it started its trajectory.

Figure 5.5 shows the bag data of the simulation for a delivery to be dispatched to the goal *Garage*. The visualization tool gives a first glance at the data where it is possible to see that the door was actuated once, the proximity sensor 3 times and 3 QR codes were published. However in the first and last triggering of the sensor there was no QR code detection. Only in the second triggering was the QR code detected and more than once. The FSM has been created to handle only once therefore the additional QR code detection in this same triggering where not taken into account. This is possible as the QR code is only gathered in the *Waiting for QR Code* state and as soon as it is gathered the FSM transitions to the *QR Code Received* state and thus the additional QR codes are not processed.

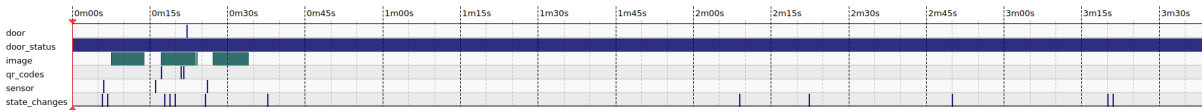


Figure 5.5: Door Opened Too Long

To have a better understanding of the events, figure 5.7 shows a more detailed plot of the bag data. The different door, sensor and qr codes observation can still be made. However this plot shows that the door was first closed and then when the door was actuated (gold star) the door status did indeed change and became open. Moreover, it can be seen the first trigger made the transition from *Waiting for Proximity Sensor* (3) *Waiting for QR Code* (3.5). The second time the sensor was triggered was still inside the *Waiting for QR Code* state. Only when the QR code was detected did the FSM transition to the *QR Code Received* (4) state. As the next two states were *QR Code Readable* (4.5) and *Delivery* (5), it can be deduced that the QR code was indeed readable with a delivery and door command.

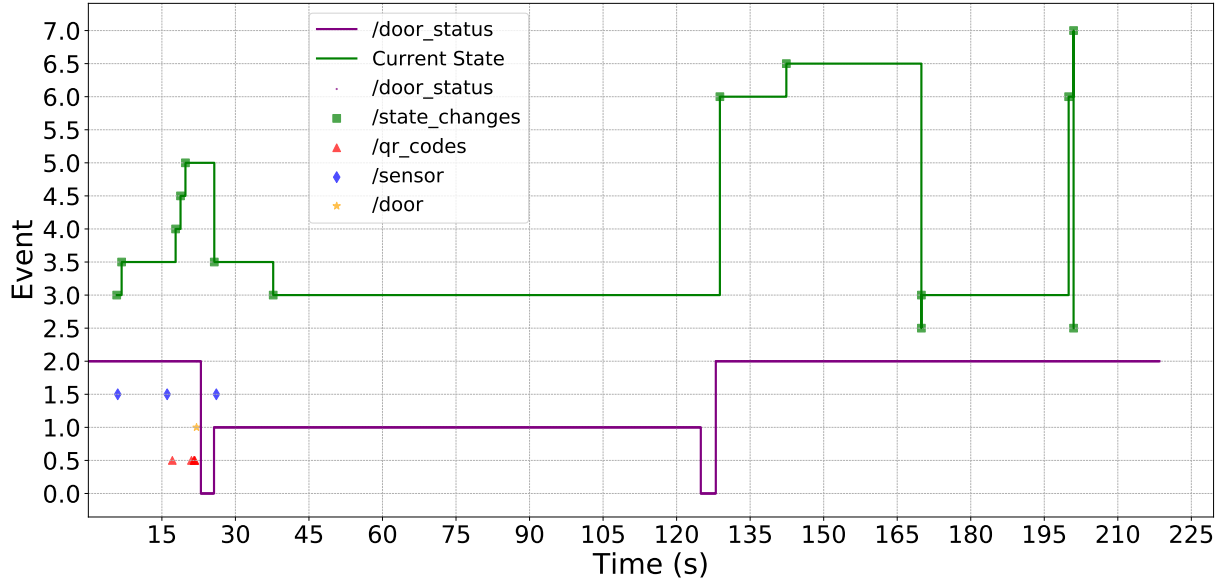


Figure 5.7: Door Opened Too Long Event Plot

As soon as the delivery goal has been processed the FSM transitioned back to *Waiting for QR Code* (3.5) and to *Waiting for Proximity Sensor* (3) where it stayed until the door was manually closed (no door command from the FSM). As soon as the door was closed ($\text{door_status} = 2$), the FSM transitions to *No More QR Codes* where as the goal list features one goal, the trajectory is calculated immediately on the RRT*^{KD} algorithm. The FSM then transitions to the *EZ10 in Motion* where the MPC uses the trajectory calculated to send commands to the EZ10. As mentioned previously the EZ10 is incapable of moving therefore a simulation was run on MATLAB/Simulink to give a realistic time of completion. Figure 5.6 shows the trajectory the EZ10 would have taken to reach the garage and the lateral error between the waypoints given to the MPC and the actual state of the vehicle. This error is represented by the gradient of color as seen that reflects the value of the error. Here the maximum lateral error encountered by the MPC is of 0.7 meters which is a relatively low value. Figure 5.8, shows the command the MPC is giving to the EZ10 to make it follow that trajectory. It is also possible to see that the velocity of the EZ10 was limited to below 1.0 m/s.

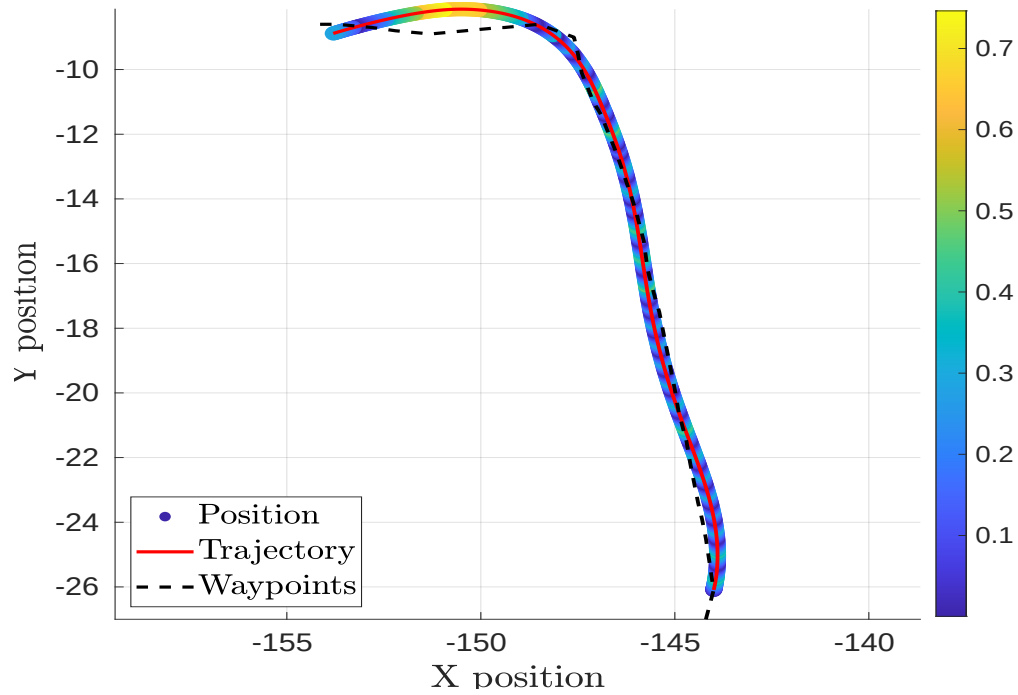


Figure 5.6: MPC Start-Garage Trajectory with Lateral Error

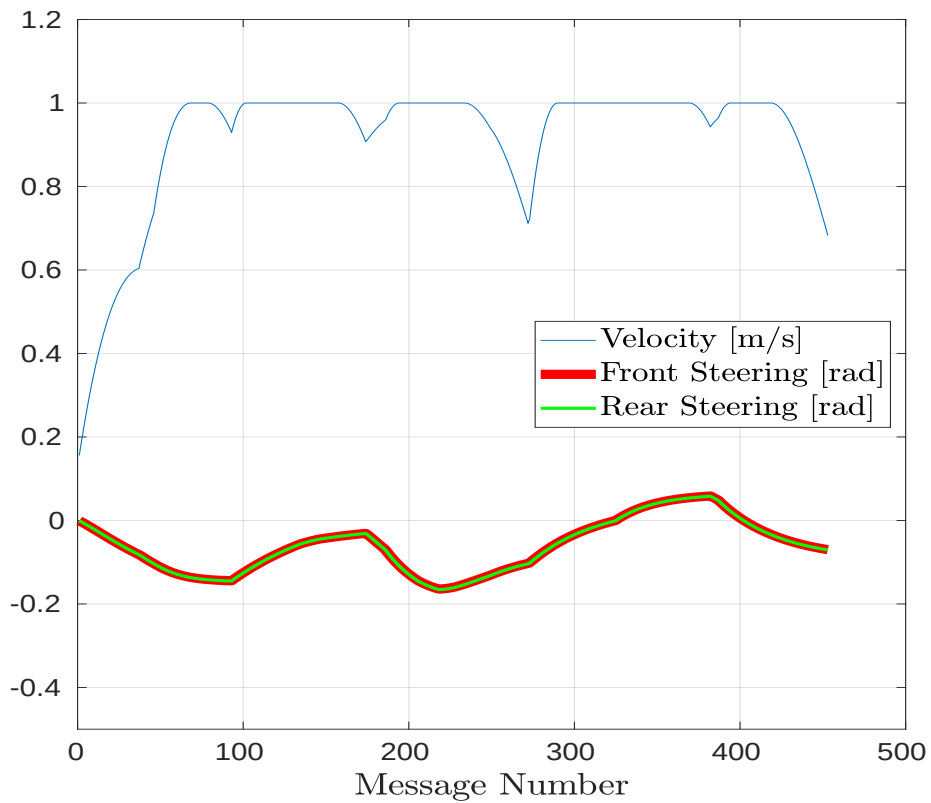


Figure 5.8: MPC Velocity and Steering Commands

Moreover, as the figure 5.8 contains the number of messages given by the MPC and it is publishing at 20Hz it is possible to evaluate that ideally, the EZ10 would take around 23 seconds to go from the Start position to the Garage position. Going back to figure 5.7 the time taken to actually compile the code is different than the real time the EZ10 would have taken. However the change of state that happens from *Bus in Motion* (6.5) to *EZ10 Stopped* (2.5) shows that the FSM handles well the stopping part of the bus. Immediately after reaching the stopped position, the FSM transitions to *Waiting for Proximity Sensor* (3) to wait for the sensor to be triggered. As nothing happens, the FSM transitions directly to *No More QR Codes*(6) where as there are no goals in the list, it goes into the *EZ10 Retuning to Base* (7) and finally stops with *EZ10 Stopped* (2.5) and shuts of the state machine as it is assumed it has terminated its job. The *EZ10 Retuning to Base* also uses

As the FSM has successfully handled delivering to one goal it is necessary to evaluate if it is able to handle two goals given by the same QR code.

5.4. Two Goals at Once

In this scenario, the FSM has to handle 2 goals coming from the same QR code. This experiment mimics a situation where the user would have multiple packages to deliver at multiple destinations. For this scenario, it was assumed that once arriving to the goal, the recipient was not be present to retrieve the package.

At first glance, the door was only actuated once which matches with our desired experiment, also a QR code was detected. There is also a trigger of the sensor without a QR detection that coincides with the opening of the doors. The triggering most likely is due to the vibrations cause by the door opening.

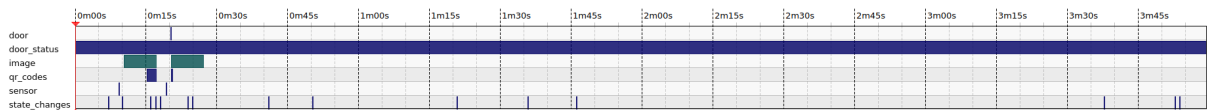


Figure 5.9: Two Goals Bag Data

With the better detail given by the plot, a similar situation to the previous scenario is perceived where all the transitions are done correctly. One difference between figure 5.7 and figure 5.10 is that the FSM is quicker to transition from *Waiting for QR Code* (3.5) to *Waiting for Proximity Sensor* (3) after having handled the delivery goals. This is due to the sensor not being re-triggered like in figure 5.7.

Because the QR codes contains now 2 different goals, when the FSM enter into the *No More QR Codes* state it needs to first order the goals and then select the closest to calculate its trajectory to give to the MPC. The FSM did its job as the first from the 2 goals (Garage and B22) Garage was indeed closer to the starting position. As the MPC result is similar to the one that was previously obtained, there is no need to include them in this section.

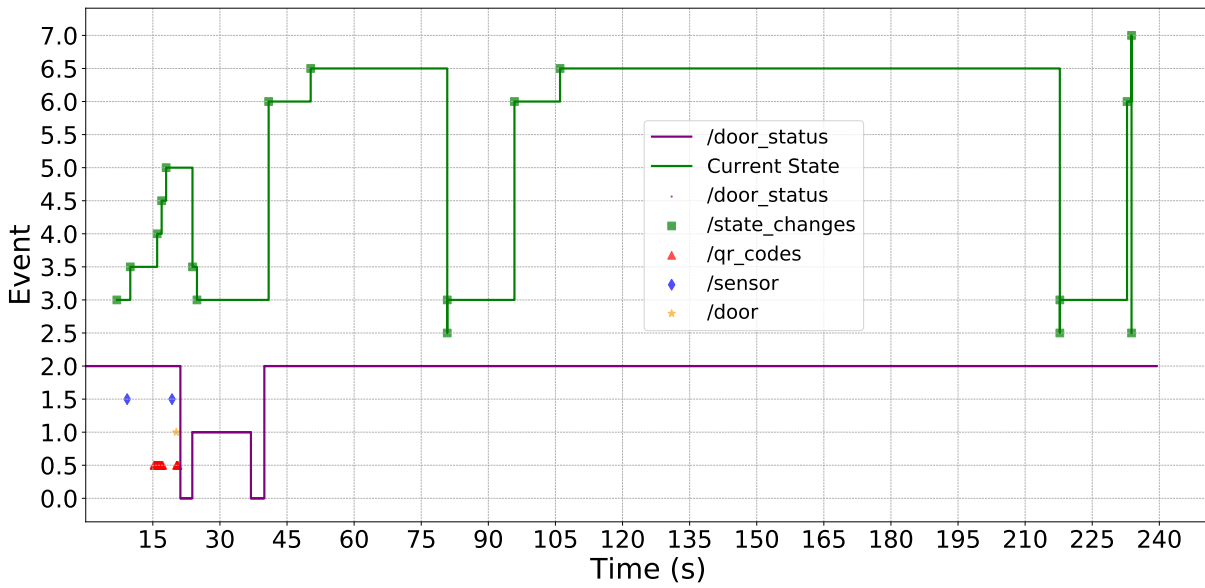


Figure 5.10: Two Goals Event Plot

To make sure that the second goal was also evaluated, it was expected of the FSM to transition back into *EZ10 in Motion* after reaching the first goal. Indeed, after reaching the first goal and coming to a stop, the FSM transitions directly to *Waiting for Proximity Sensor* and then to *No More QR Codes*. And as expected, the goal was still in the list, therefore the FSM transitioned to *EZ10 in Motion* to deal with the next goal and did not return to base.

Figure 5.11 represents the trajectory the EZ10 would have taken from Garage to B22. The same lateral error is represented here with a maximum of 1.1 meters. After a review this maximum error is located at the start of the trajectory, given that the vehicle is not in the optimal heading, it needs to turn to follow closely the trajectory. And as the vehicle is only able to turn up to 18 degrees, it will go a bit farther from the trajectory before coming back to it. Figure 5.12 shows the commands and it can be seen that the steering is limited to below 0.314 radians, this is because the EZ10 is limited to 18 degrees of

turning on its wheels. And it is possible to see the swerving motion done by the vehicle at the start to get back to properly following the trajectory, as the steering goes to 18 degrees comes back to almost zero and again to around 15 degrees.

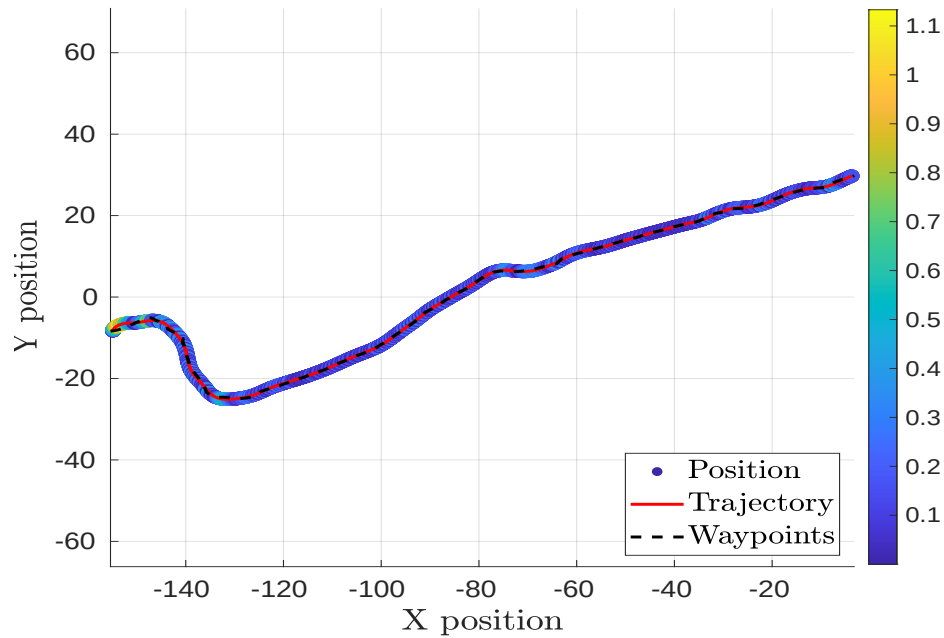


Figure 5.11: MPC Garage-B22 Trajectory with Lateral Error

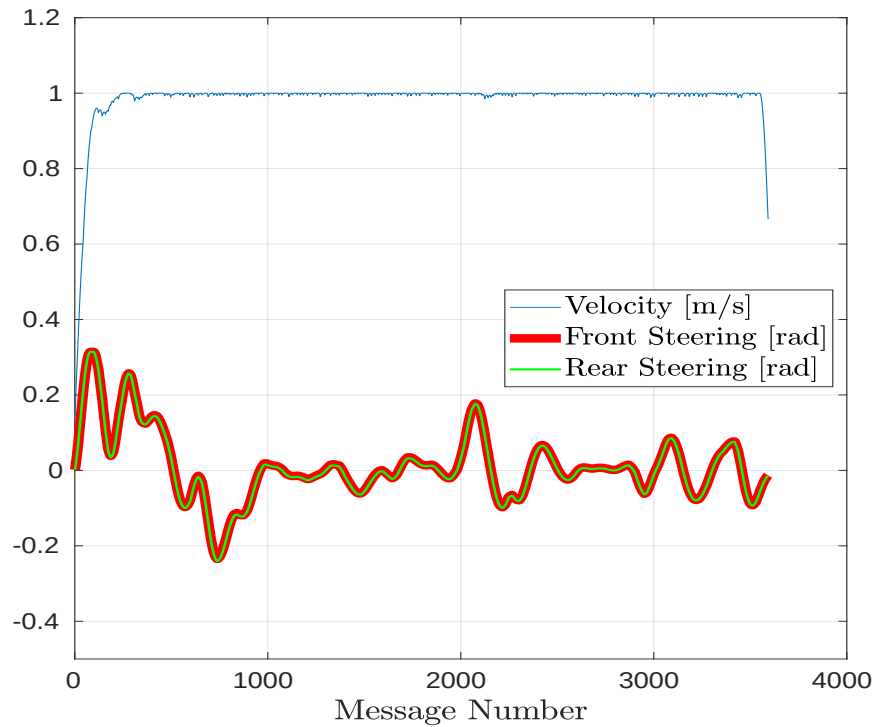


Figure 5.12: MPC Velocity and Steering Commands

From figure 5.12 it is also possible to deduce that the time taken to go from Garage to B22 would be around 3 minutes. As in the previous experiment after reaching the goal the EZ10 stops and waits for triggering but finally goes home due to a lack of goals and the FSM turns off after the EZ10 reaches a stop.

The results of this experiment show that the FSM is able to handle multiple goals at once. The next section will be a more complete experiment, gathering almost all the possibilities of the FSM.

5.5. Adding More Goals at a Delivery Point and Retrieving Packages

In this experiment two initial delivery goals were be given to be handled by the FSM. After completing the first stop, another QR code was scanned to add another 2 delivery goals. In the second stop, a reception QR code was also scanned to this time to access the packages inside of the bus.

This scenario is designed to perform almost all the transitions of the FSM in one experiment to simulate all the possible real-life scenarios. This is also to try to simulate the FSM working for an extended period of time to make sure it can handle repeated loops. The visualization of the bag in figure 5.14 gives us an good overview of the data. It can be seen that the door has been actuated 3 times, the sensor was triggered 6 times with only 3 of these triggers leading up to a QR code detection. The focus needs to be put on the plot of the data present in figure 5.15.

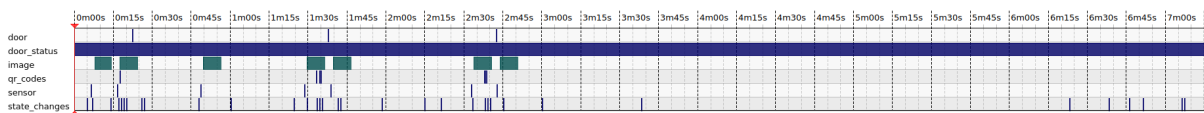


Figure 5.14: Four Goals Bag Data

In the plot many state transitions can be observed. As previously discussed, the first QR code given is of two goals, this is what can be seen from the plot where after handling the first goal the FSM transitions to *EZ10 in Motion*. After stopping for the first time the first difference with the previous scenarios is visible. The sensor is triggered, a QR code is detected and the door is actuated. This is when the QR code with 2 new delivery goals was scanned.

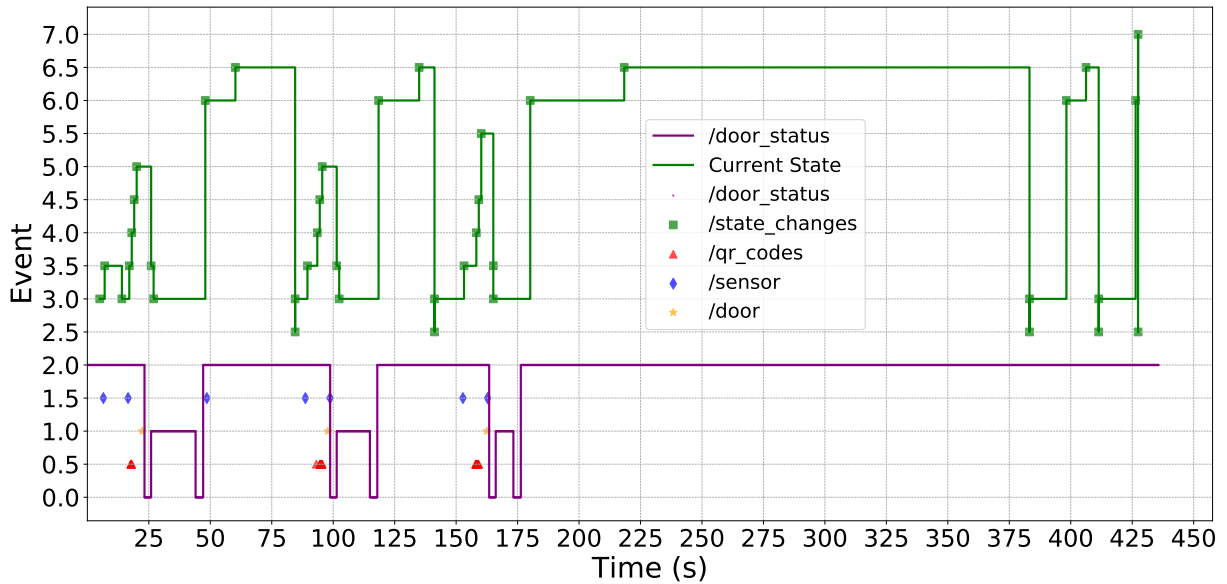


Figure 5.15: Four Goals Event Plot

After completing the path to the second goal (B22) which was selected among 3 goals as the closest to the first goal (Garage), a QR code was once again detected and the door was actuated. However, the state did not transition to the *Delivery* (5) state but rather to the *Reception* (5.5) state. This aligns with our intended experiment of wanting to get packages from the EZ10. After that the FSM managed to properly deal with the 2 other goals. And the resulting trajectories can be seen in figures 5.13, 5.16, 5.17 and 5.18. The same observation can be made for the figure regarding the lateral error. The maximum error is found at the beginning of the trajectory when the vehicle adjusts its heading to follow the trajectory. The rest of the trajectory has an error that is very low. A more detailed analysis of the MPC will be done in the next section.

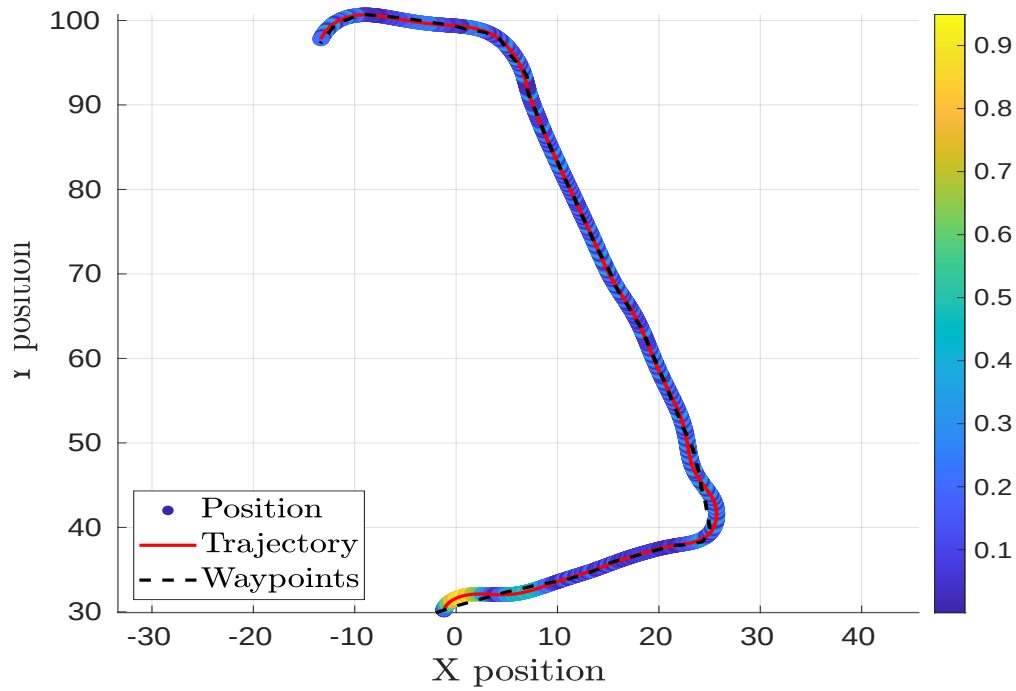


Figure 5.13: MPC Trajectory B22 Entrance with Lateral Error

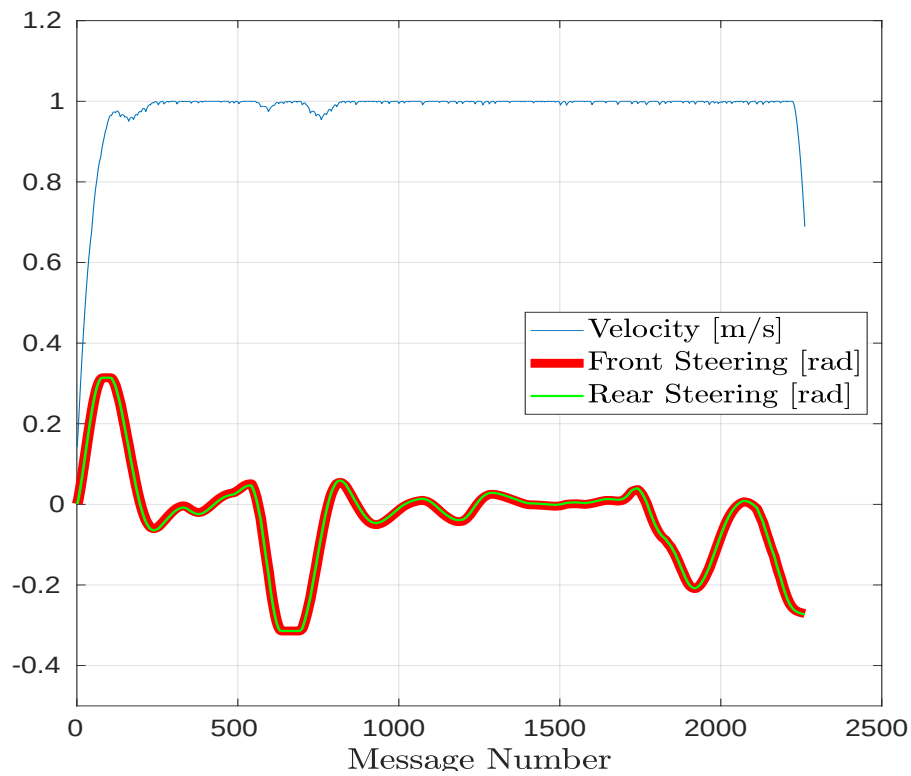


Figure 5.16: MPC Velocity and Steering Commands

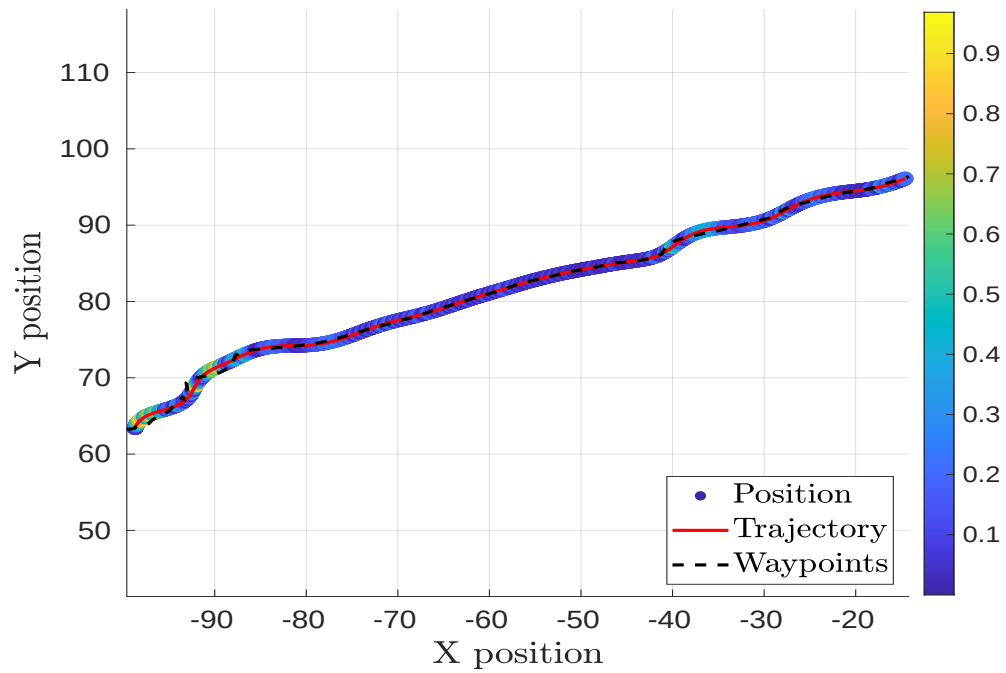


Figure 5.17: MPC Trajectory Entrance B15 with Lateral Error

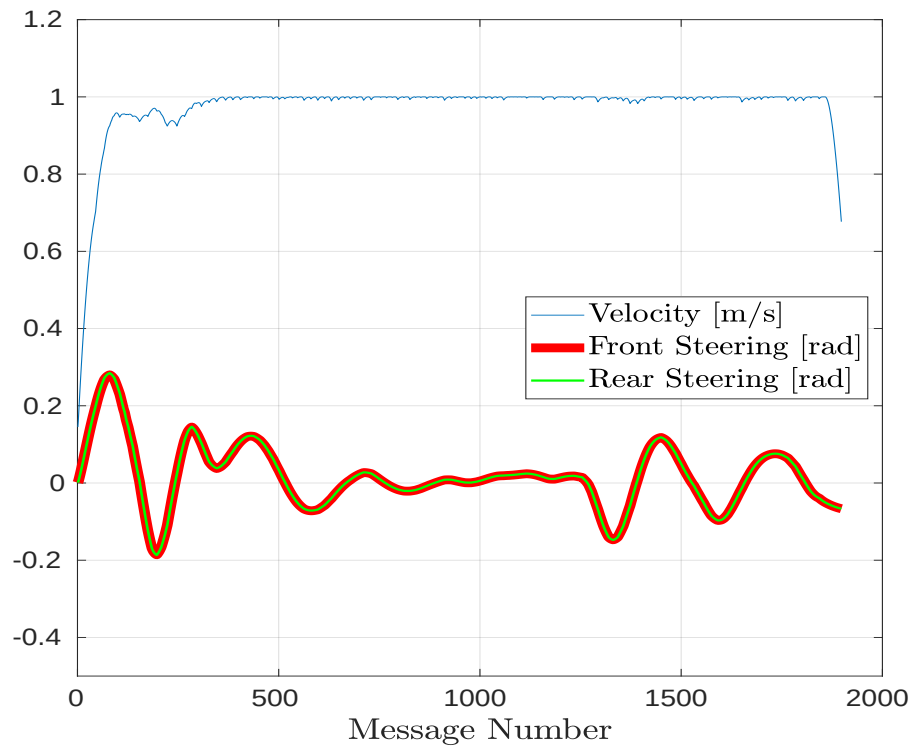


Figure 5.18: MPC Velocity and Steering Commands

This last experiment proves that the FSM and the system as a whole is able to communicate perform tasks without errors. Moreover, by adding goals during a stop of the EZ10, the first implementation of an ADV with the capacity to receive delivery orders from the end user has been validated. In fact, as mentioned before, all ADV systems work on the basis that there is one dispatcher that schedules the deliveries and receivers that will take the delivery. This work and experiment has proven that implementing a system where the end user can also dispatch delivery is possible and feasible.

5.6. Entire Loop Trajectory Comparison

In this section a comparison of the entire loop distance around the campus is simulated. The comparison is between the distances given by the RRT*KD, satellite image and the MPC controller. Moreover, this section is to assert the reliability and safety of the MPC.

From the path planner RRT*KD the whole trajectory loop length was estimated to be 535 meters (figure 5.19). This trajectory is rough and represented by small successive segments, therefore, it tends to increase the distance that will be covered by the ADV during its driving.

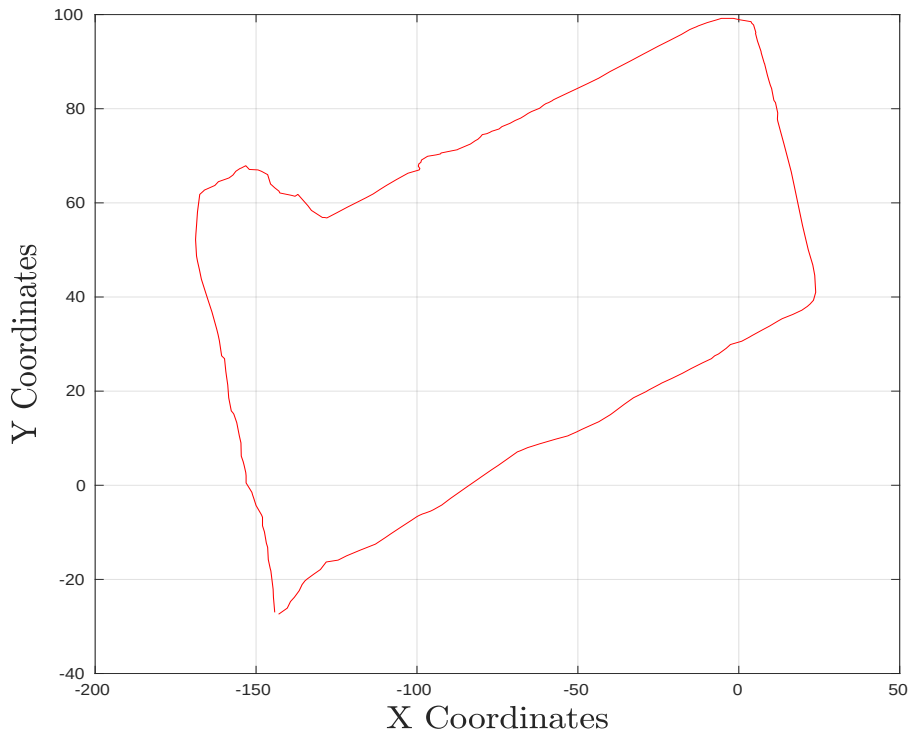


Figure 5.19: RRT*KD Trajectory of the Entire Loop

To have a better overview of the real distance, a simple measurement was made using the Google Maps feature of Measure Distance over the satellite image of the campus as seen in figure 5.20. This step shows that the approximated real distance (524 meters) is smaller than the one generated by the RRT*KD. This is due to using complete straight lines and not small segments between each node like in the RRT*KD. However, as the measurement is using straight lines, it does not reflect the proper distance due the corners taken as right angle. Nonetheless, it gives a good approximation and target for the final distance.

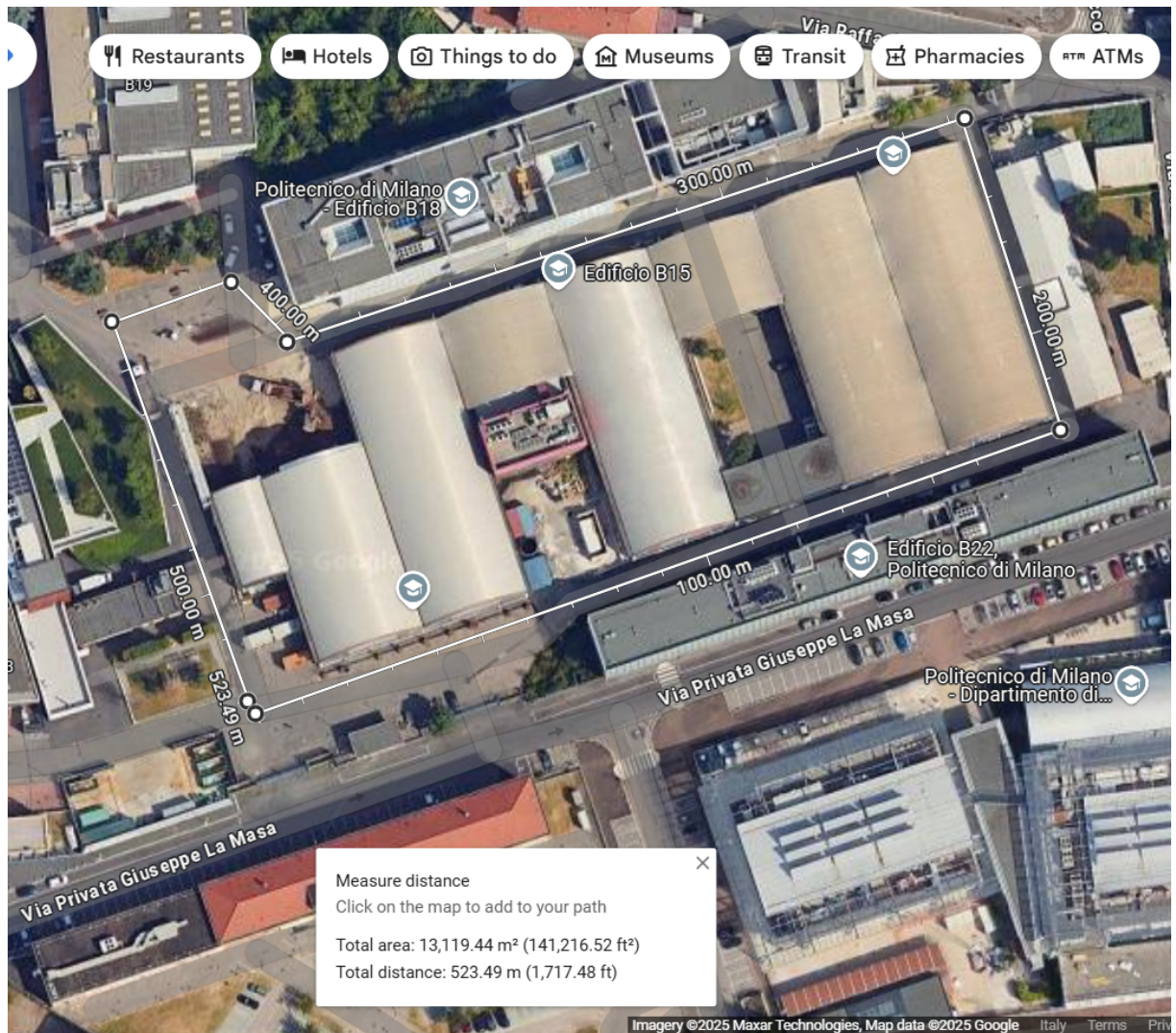


Figure 5.20: Real Distance of the Entire Loop [48]

Figure 5.21 gives the overall shape of the trajectory that is generated by the MPC. The distance obtained from the MPC was of 521 meters to complete the entire loop. The distance difference of the MPC with the approximated real distance can be attributed

to the corners being taking as a curve by the MPC unlike in the measurement. This small difference (3 meters) also suggests that the MPC is efficiently guiding the ADV to have an optimal trajectory, at least distance-wise.

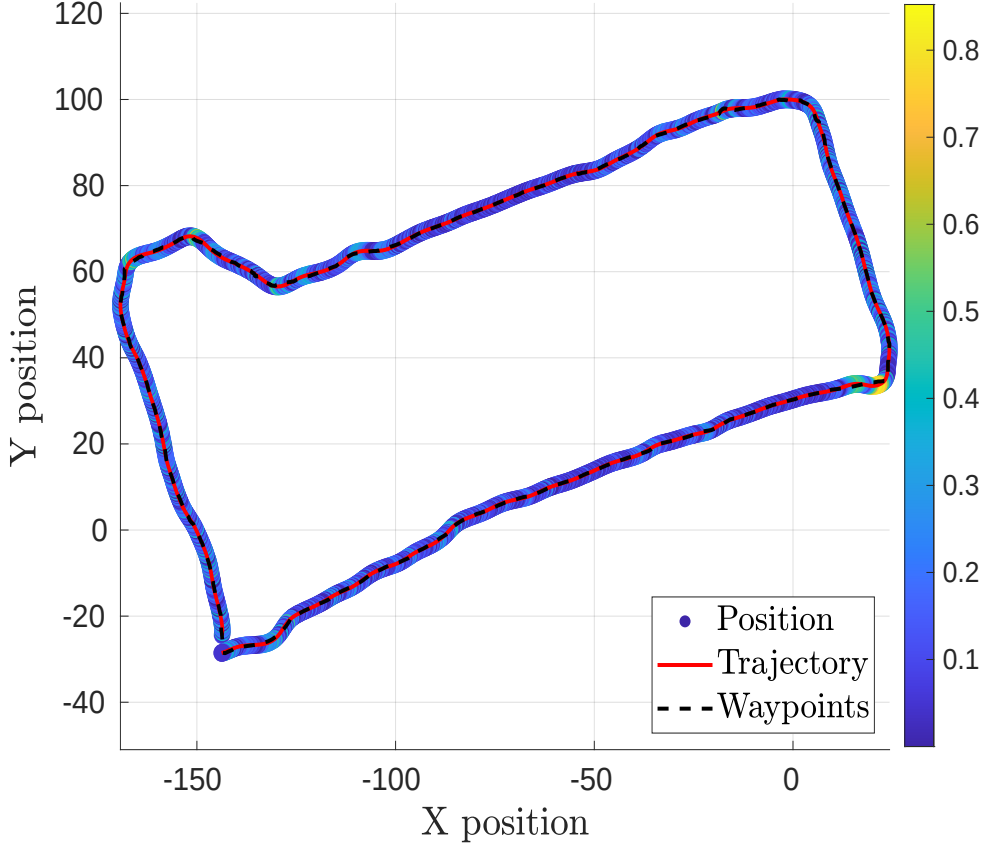


Figure 5.21: MPC Trajectory of the Entire Loop with Lateral Error

The MPC needs to ensure that it is within some safety margin on the whole trajectory that would make it acceptable to be used in real-world simulation.

In fact, as simulated, the MPC is not aware of the environment in which the EZ10 is driving. This means that the only safety to avoid contact with predetermined boundaries is by referring to the waypoints given by the RRT*KD. As mentioned in 2.3.4, the size of the vehicle is taken into account by the RRT*KD with a 25% safety factor. This safety factor allows the vehicle to clear any obstacle by at least 0.5 meter.

Figure 5.21 shows the lateral error (in meters) of the vehicle state given by the MPC with respect to the waypoints given by the RRT*KD. It clear that the vast majority of the trajectory is under the safety threshold of 0.5 meters, with some areas, mainly located around corners, exceeding the threshold. In fact, the area with the biggest lateral error

is located at the bottom right corner and is due to the vehicle limitations. As mentioned previously, the EZ10 has a steering limit of 18 degrees which can be seen reached in figure 5.22 at the same moment of that specific corner. This lateral error shows also shows that the smoothing done by the MPC is not exaggerated as the range of value is relatively low.

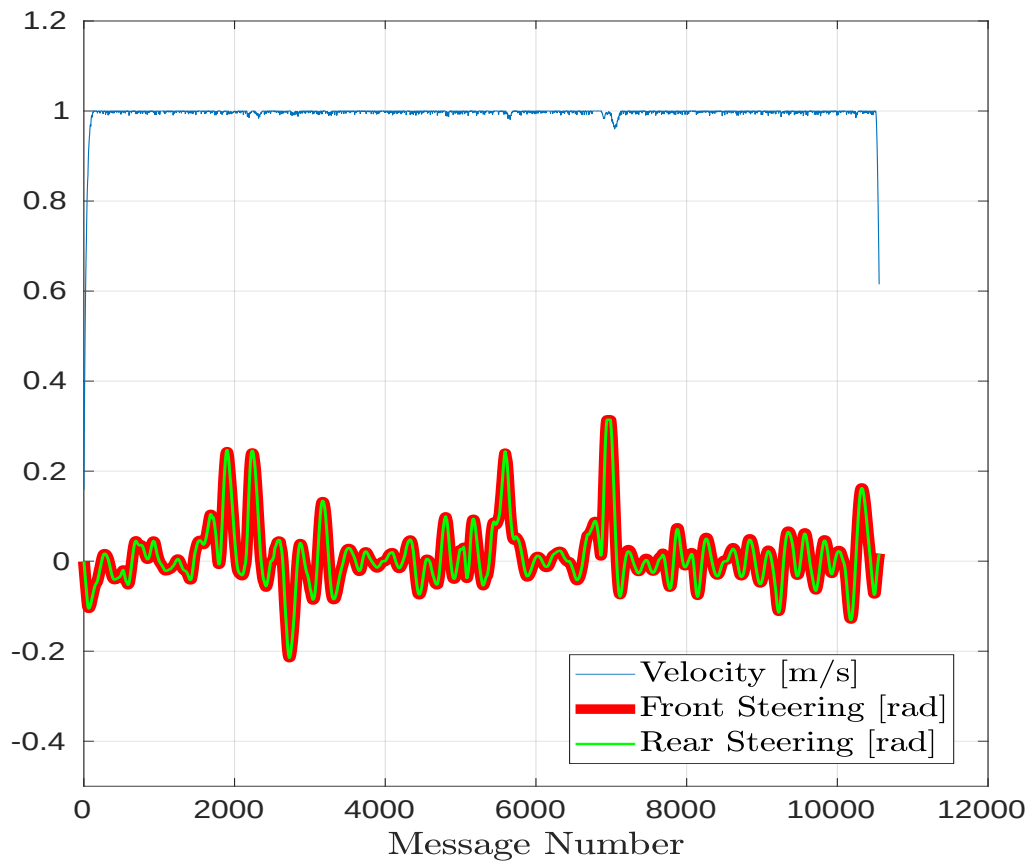


Figure 5.22: MPC Velocity and Steering Commands

Figure 5.22 also informs on the smoothness of the commands given to the EZ10. As can be seen there are not abrupt changes in velocity or steering command. This simulation reinforces that the MPC, given a better safety factor around corners, can be deemed safe and usable for real-life simulation.

Conclusion

The research presented in this thesis has successfully demonstrated the feasibility of implementing an autonomous delivery vehicle (ADV) system capable of efficiently managing and executing delivery tasks while interacting directly with users. By integrating the EasyMile EZ10 autonomous vehicle with a Vehicle-to-User (V2U) interaction system and a finite state machine (FSM) for behavioral planning, a structured and adaptive delivery mechanism has been developed, allowing for a seamless and autonomous package transportation process. This work has addressed key challenges in autonomous delivery, including navigation, user interaction, and behavioral decision-making, and has provided solutions that enhance efficiency, reliability, and safety in real-world applications.

One of the primary contributions of this research was the development of a navigation system that combines occupancy grid mapping, Rapidly-exploring Random Trees Star with KD-tree optimization (RRT*KD), and Model Predictive Control (MPC) for motion control. The navigation module demonstrated significant improvements in path optimization compared to standard RRT and RRT, reducing overall path length by approximately 10–20% while ensuring computational efficiency suitable for real-time execution. The implementation of the KD-tree algorithm for nearest neighbor searching further enhanced the path planning process by improving consistency and reducing computational overhead in longer routes. The MPC controller successfully generated smooth and dynamically feasible trajectories, maintaining a Root Mean Square Error (RMSE) below 2 meters in simulated environments, thus ensuring both stability and precision in motion execution.

In terms of user interaction, the Vehicle-to-User (V2U) system developed in this thesis introduced an innovative and cost-effective approach using QR code-based communication. The designed prototype, featuring a Raspberry Pi-based system with a microwave proximity sensor and a camera module, enabled secure and efficient QR code scanning while minimizing energy consumption and computational load. The integration of the V2U system with a ROS-based communication framework facilitated seamless data exchange between users and the ADV, allowing for user-initiated deliveries, package retrieval requests, and autonomous door control. The use of QR codes proved to be a highly practical and scalable solution, reducing the complexity and cost associated with traditional

Human-Machine Interfaces (HMIs) while maintaining high usability and accessibility.

The behavioral planning module, implemented through a finite state machine (FSM), played a crucial role in managing the different operational states of the ADV and ensuring smooth transitions between delivery tasks. The FSM was rigorously tested through multiple simulation scenarios, including single-goal and multi-goal deliveries, accidental triggering of the system, and dynamic user inputs. The results demonstrated that the FSM could efficiently handle complex delivery tasks, optimize route selection, and adapt to unexpected changes in user requests or environmental conditions. Additionally, the system successfully managed multi-goal deliveries by dynamically reordering the sequence of tasks based on proximity, thereby improving overall delivery efficiency.

Despite the successful development and validation of the proposed system, some limitations remain. Due to mechanical issues with the EasyMile EZ10, full-scale real-world testing of the autonomous motion control system could not be conducted. While simulations provided valuable insights into the system's performance and the technology for MPC ADV is already present, real-world deployment would introduce additional challenges, such as dynamic obstacles, sensor noise, and environmental uncertainties, that require further refinement. Future research should focus on enhancing the robustness of the localization system, integrating real-time obstacle avoidance mechanisms in the MPC (as mentioned previously), and improving computational efficiency for onboard processing. Moreover, incorporating machine learning techniques into the behavioral planning module could enable more adaptive and intelligent decision-making in complex urban environments.

In conclusion, this thesis has established a strong foundation for future developments in autonomous delivery vehicles by demonstrating the viability of a user-interactive, autonomous delivery system that integrates advanced navigation, motion control, and behavioral planning techniques. The combination of optimized path planning, a low-cost and efficient V2U interface, and a structured FSM-based behavioral planner provides a scalable and practical solution for last-mile delivery applications. With continued improvements and real-world testing, this system has the potential to significantly enhance the efficiency, reliability, and sustainability of autonomous delivery services, paving the way for widespread adoption in urban logistics and smart city applications.

List of Figures

1.1	Localization and Detection Sensors of the Easy Mile EZ10	8
1.2	Nuro Customized Storage [13]	9
1.3	Meituan AUtonomous Delivery Vehicle [17]	10
2.1	Front and rear Velodyne range of detection	20
2.2	Overview of GNSS with RTK correction	22
2.3	Default Gmapping parameters in pixel coordinates	23
2.4	Default Gmapping parameters in local coordinates	24
2.5	Tuned Gmapping parameters in local coordinates	25
2.6	Superimposed images	26
2.7	Map after sensor fusion in local coordinates	27
2.8	Basic RRT Construction Algorithm [24]	29
2.9	The Extend Part [24]	29
2.10	Basic RRT* Construction Algorithm [41]	30
2.11	KD-tree construction algorithm [42]	31
2.12	KD-tree query algorithm [42]	32
2.13	Footprint of the EZ10	34
2.14	Start and Delivery locations	35
2.15	Run Time Box Plot	36
2.16	Path Length Box Plot	37
2.17	MPC iteration	38
2.18	Vehicle Model Diagram	39
2.19	MPC results	42
2.20	Start to B15 MPC Trajectory Overlay	43
2.21	Start to Entrance MPC Trajectory Overlay	43
3.1	EZ10 Windscreen	46
3.2	Case ISO View	48
3.3	Case Dimensions in Millimeters	49
3.4	Exploded View of the Case	49

3.5	EZ10 Window Mount	50
3.6	3D Printed Case	51
3.7	Mounted 3D Printed Case	51
3.8	Flowchart of the V2U Communication	53
3.9	User Interface	54
4.1	Basic State Machine Diagram	58
4.2	EZ10 Finite State Machine Diagram	59
5.2	Overall System Communication	64
5.1	Bag Topics Visualization	65
5.3	Failed Detection	67
5.4	Failed Detection Event Plot	67
5.5	Door Opened Too Long	68
5.7	Door Opened Too Long Event Plot	69
5.6	MPC Start-Garage Trajectory with Lateral Error	70
5.8	MPC Velocity and Steering Commands	70
5.9	Two Goals Bag Data	71
5.10	Two Goals Event Plot	72
5.11	MPC Garage-B22 Trajectory with Lateral Error	73
5.12	MPC Velocity and Steering Commands	73
5.14	Four Goals Bag Data	74
5.15	Four Goals Event Plot	75
5.13	MPC Trajectory B22 Entrance with Lateral Error	76
5.16	MPC Velocity and Steering Commands	76
5.17	MPC Trajectory Entrance B15 with Lateral Error	77
5.18	MPC Velocity and Steering Commands	77
5.19	RRT*KD Trajectory of the Entire Loop	78
5.20	Real Distance of the Entire Loop [48]	79
5.21	MPC Trajectory of the Entire Loop with Lateral Error	80
5.22	MPC Velocity and Steering Commands	81
23	TF parameters for the EZ10	95
24	Sensor Fusion parameters for the EZ10	95

List of Tables

2.1	RRT Comparison Table	35
3.1	Confusion Matrices for The Infrared Sensor	47
3.2	Confusion Matrices for The Microwave Sensor	47
5.1	State-to-Value Mapping	66
2	Default Gmapping parameters of figure 2.4	97
3	Tuned Gmapping parameters of figures 2.5 and 2.7	99

Bibliography

- [1] THE EUROPEAN PARLIAMENT and THE COUNCIL OF THE EUROPEAN UNION. Regulation (eu) 2019/631 of the european parliament and of the council of 17 april 2019 setting co2 emission performance standards for new passenger cars and for new light commercial vehicles, and repealing regulations (ec) no 443/2009 and (eu) no 510/2011. *Official Journal of the European Union*, 2019.
- [2] Chuck Martin. Automatic delivery robots highlighted at ces, 2024.
- [3] Yinying He and Csaba Csiszár. Model for crowdsourced parcel delivery embedded into mobility as a service based on autonomous electric vehicles. *Energies*, 14(11), 2021.
- [4] Babak Shahian Jahromi, Theja Tulabandhula, and Sabri Cetin. Real-time hybrid multi-sensor fusion framework for perception in autonomous vehicles. *Sensors*, 19(20), 2019.
- [5] Benjamin Gillies Mark Fagan and Edward Guo. Autonomous delivery vehicles: Why you should care and what you should do. *Harvard*, pages 1–15, 2022.
- [6] LMAD. An overview of the existing autonomous delivery solutions in 2022, 2022.
- [7] Patrick Geragersian, Ivan Petrunin, Weisi Guo, and Raphael Grech. Multipath detection from gnss observables using gated recurrent unit. In *2022 IEEE/AIAA 41st Digital Avionics Systems Conference (DASC)*, pages 1–7, 2022.
- [8] Giuseppe Mollica, Marco Legittimo, Alberto Dionigi, Gabriele Costante, and Paolo Valigi. Integrating sparse learning-based feature detectors into simultaneous localization and mapping—a benchmark study. *Sensors*, 23(4), 2023.
- [9] Farooq Sijal Shaqwi, Lukman Audah, Mustafa Hamid Hassan, Mohammed Ahmed Jubair, Mohd Helmy Abd Wahab, and Salama A. Mostafa. A concise review of deep learning deployment in 3d computer vision systems. In *2021 4th International Symposium on Agents, Multi-Agent Systems and Robotics (ISAMSR)*, pages 157–160, 2021.

- [10] Hermann Rohling and Christof Moller. Radar waveform for automotive radar systems and applications. In *2008 IEEE Radar Conference*, pages 1–4, 2008.
- [11] Himansu Sekhar Behera and Anandakumar M Ramiya. Urban flood modelling simulation with 3d building models from airborne lidar point cloud. In *2022 IEEE Mediterranean and Middle-East Geoscience and Remote Sensing Symposium (M2GARSS)*, pages 145–148, 2022.
- [12] Antuan Goodwin. Nuro r3 ev is an adorable self-driving snack bar, 2024.
- [13] Jaclyn Nash for the Smithsonian Institution. Nuro’s autonomous delivery vehicle drives into the smithsonian’s collections this earth day, 2023.
- [14] Alan Ohnsman. Uber taps nuro’s street-legal robots for food deliveries, 2022.
- [15] Starship Technologies. Starship technologies advances adoption of autonomous delivery as demand quadruples in the last year, 2021.
- [16] Dimsumdaily Hong Kong. Meituan introduces self-driving delivery vehicles in mainland, 2024.
- [17] Jon Niccum. Autonomous vehicles could prove to be future model for delivery services, study finds, 2022.
- [18] Erik Einhorn, Christof Schröter, and Horst-Michael Gross. Finding the adequate resolution for grid mapping - cell sizes locally adapting on-the-fly. In *2011 IEEE International Conference on Robotics and Automation*, pages 1843–1848, 2011.
- [19] Dong-Il Kim, Heewon Chae, Jae-Bok Song, and JiHong Min. Point feature-based outdoor slam for rural environments with geometric analysis. In *2015 12th International Conference on Ubiquitous Robots and Ambient Intelligence (URAI)*, pages 218–223, 2015.
- [20] Leyao Huang. Review on lidar-based slam techniques. In *2021 International Conference on Signal Processing and Machine Learning (CONF-SPML)*, pages 163–168, 2021.
- [21] E. W. Dijkstra. *A Note on Two Problems in Connexion with Graphs*, page 287–290. Association for Computing Machinery, New York, NY, USA, 1 edition, 2022.
- [22] Rina Dechter and Judea Pearl. Generalized best-first search strategies and the optimality of a^* . *J. ACM*, 32(3):505–536, July 1985.
- [23] L.E. Kavradi, M.N. Kolountzakis, and J.-C. Latombe. Analysis of probabilistic

- roadmaps for path planning. *IEEE Transactions on Robotics and Automation*, 14(1):166–171, 1998.
- [24] Steven M LaValle and James J Kuffner. Rapidly-exploring random trees: Progress and prospects: Steven m. lavalle, iowa state university, a james j. kuffner, jr., university of tokyo, tokyo, japan. *Algorithmic and computational robotics*, 2001.
- [25] Fahad Islam, Jauwairia Nasir, Usman Malik, Yasar Ayaz, and Osman Hasan. Rrt-smart: Rapid convergence implementation of rrt towards optimal solution. In *2012 IEEE International Conference on Mechatronics and Automation*, pages 1651–1656, 2012.
- [26] Mark J Willis. Proportional-integral-derivative control. *Dept. of Chemical and Process Engineering University of Newcastle*, 6, 1999.
- [27] Max Schwenzer, Muzaffer Ay, Thomas Bergs, and Dirk Abel. Review on model predictive control: An engineering perspective. *The International Journal of Advanced Manufacturing Technology*, 117(5):1327–1349, 2021.
- [28] TEKLA. The evolution of human machine interfaces and their future potential for construction, 2018.
- [29] Zhengyou Zhang. Microsoft kinect sensor and its effect. *IEEE MultiMedia*, 19(2):4–10, 2012.
- [30] Sumit Tiwari. An introduction to qr code technology. In *2016 International Conference on Information Technology (ICIT)*, pages 39–44, 2016.
- [31] Joseph El Chartouni, Elio El Ghaoui, Gaby Abou Haidar, and Roy Abi Zeid Daou. Design and implementation of a qr-enabled autonomous bartender using a user-friendly interface. In *2024 28th International Conference on Methods and Models in Automation and Robotics (MMAR)*, pages 35–40, 2024.
- [32] Jingda Wu, Chao Huang, Hailong Huang, Chen Lv, Yuntong Wang, and Fei-Yue Wang. Recent advances in reinforcement learning-based autonomous driving behavior planning: A survey. *Transportation Research Part C: Emerging Technologies*, 164:104654, 2024.
- [33] Scott Drew Pendleton, Hans Andersen, Xinxin Du, Xiaotong Shen, Malika Meghjani, You Hong Eng, Daniela Rus, and Marcelo H. Ang. Perception, planning, control, and coordination for autonomous vehicles. *Machines*, 5(1), 2017.
- [34] Kirill Svyatov. Architecture of the computer-aided design toolchain for the

- highly-automated vehicles. In *2024 International Russian Automation Conference (RusAutoCon)*, pages 61–67, 2024.
- [35] Valentino Razza Massimo Canale. Automated driving control in highway scenarios through a two-level hierarchical architecture. *IEEE ACCESS*, 1(1), 2024.
- [36] Satyesh Shanker AWASTHI Pawas AWASTHI. Implementation of a control interface for an autonomous shuttle. Master’s thesis, Politecnico Di Milano, 2020.
- [37] Safanah M. Raafat Zuhair A. Ahmed. An extensive analysis and fine-tuning of gmap-ping’s initialization parameters. *International Journal of Intelligent Engineering and Systems*, 16(3), 2023.
- [38] QGIS. Spacial without compromise, 2024. Version 3.34.9.
- [39] Steven M. LaValle. Rapidly-exploring random trees: A new tool for path planning. *Iowa State University*, 1998.
- [40] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *International Journal of Robotic Research - IJRR*, 30:846–894, 06 2011.
- [41] Iram Noreen, Amna Khan, and Zulfiqar Habib. Optimal path planning using rrt* based approaches: a survey and future directions. *International Journal of Advanced Computer Science and Applications*, 7(11), 2016.
- [42] Anna Yershova and Steven M. LaValle. Improving motion-planning algorithms by efficient nearest-neighbor searching. *IEEE Transactions on Robotics*, 23(1):151–157, 2007.
- [43] S. Arrigoni, F. Braghin, and F. Cheli. Mpc trajectory planner for autonomous driving solved by genetic algorithm technique. *Vehicle System Dynamics*, 60(12):4118–4143, December 2021.
- [44] Mohammed Irshadh Ismaaeel Sathyamangalam Imran, Satyesh Shanker Awasthi, Michael Khayyat, Stefano Arrigoni, and Francesco Braghin. A rule-defined adaptive mpc based motion planner for autonomous driving applications. In Giampiero Mastinu, Francesco Braghin, Federico Cheli, Matteo Corno, and Sergio M. Savaresi, editors, *16th International Symposium on Advanced Vehicle Control*, pages 540–549, Cham, 2024. Springer Nature Switzerland.
- [45] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen,

and Moritz Diehl. acados – a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 2021.

- [46] GPDP. Video surveillance guidelines in the italian dpa, 2010.
- [47] kolosy (<https://electronics.stackexchange.com/users/9983/kolosy>). What proximity detection technologies can work through glass? Electrical Engineering Stack Exchange. URL:<https://electronics.stackexchange.com/q/171640> (version: 2015-05-21).
- [48] Google. Measure distance, 2025.

Appendix A

```
<!-- Static Transform Publisher (base_link to velodyne_front) -->

<node pkg="tf" type="static_transform_publisher" name="base_to_velodyne_front"
args="1.925 0.0 0.55 0.0 -0.0003 0.0 base_link velodyne_front 10" />

<node pkg="tf" type="static_transform_publisher" name="base_to_velodyne_rear"
args="-1.925 0.0 0.55 3.14 -0.0003 0.0 base_link velodyne_rear 10" />

<node pkg="tf" type="static_transform_publisher" name="odom_to_map"
args="0.0 0.0 0.0 0.0 0.0 0.0 map odom 10" />

<node pkg="tf" type="static_transform_publisher" name="odom_to_imu"
args="0.0 0.0 0.0 0.0 0.0 0.0 odom imu 10" />

<!--node pkg="tf" type="static_transform_publisher" name="imu_to_base"
args="0.0 0.0 0.0 0.0 0.0 0.0 imu base_link 10" /-->

<node pkg="rrt_node" type="odom_imu_tf" respawn="false" name="odom_imu_tf" output="screen"/>
```

Figure 23: TF parameters for the EZ10

```
<node pkg="ira_laser_tools" name="laserscan_multi_merger" type="laserscan_multi_merger" output="screen">
  <param name="destination_frame" value="base_link"/>
  <param name="cloud_destination_topic" value="/merged_cloud"/>
  <param name="scan_destination_topic" value="/scan_multi"/>
  <param name="laserscan_topics" value="/velodyne_front/scan /velodyne_rear/scan" />
  <param name="angle_min" value="-3.1415927"/>
  <param name="angle_max" value="3.1415927"/>
  <param name="angle_increment" value="0.007000000216066837"/>
  <param name="scan_time" value="0.0"/>
  <param name="range_min" value="0.0"/>
  <param name="range_max" value="200.0"/>
</node>
```

Figure 24: Sensor Fusion parameters for the EZ10

Parameters	Default Value	Description
Group (1): Coordinate system parameters		
base_frame	Base link frame	The coordinate system of the mobile robot platform
odom_frame	Odometry frame	
map_frame	Map frame	
transform_publish_period	-	The publishing rate of data among frames
Group (2): OGM parameters		

xmin	-100 m	The initial map dimensions represented as a rectangular region defined by two opposite corners
ymin	-100 m	
xmax	100 m	
ymax	100 m	
delta	0.05 m	The length of one side of the grid cell in meters
occ_thresh	0.25	The threshold used to determine whether a cell is an obstacle or not.
Group (3): Measurement Update Parameters		
linearUpdate	1.0 m	The translational and rotational threshold of measurement update
angularUpdate	0.5 rad	
temporalUpdate	-1.0	The time threshold of the measurement update
Group (4): Motion model parameters		
srr	0.1	Covariance of sampling motion noise
str	0.1	
str	0.2	
stt	0.2	
Group (5.1): Scan matching and LiDAR parameters		
maxRange	-	LiDAR maximum distance range
maxUrange	-	The usable LiDAR range
throttle_scans	1	The number of throttle scans
sigma	0.05	Standard deviation (σ) of the score and likelihood functions of a single beam of the LiDAR
lsigma	0.075	
kernelSize	1	The size of the kernel's search window for matching scans
lstep	0.05 m	Linear and Angular displacement step increment size
astep	0.05 m	
iterations	1	The number of refinement steps used in the scan matches
lskip	0	The number of beams skipped in each scan

Group (5.2): Likelihood sampling parameters		
llsamplerange	0.01	Translation sampling range of scan matching
llsamplestep	0.01	Translation sampling step of scan matching
lasamplerange	0.005	Rotational sampling range of scan matching
lasamplestep	0.005	Rotational sampling step of scan matching
Group (6): PF parameters		
particles	30	The number of particles
map_update_interval	5.0	The map update period
minimumScore	0	The lowest score that a scan-matching algorithm must achieve for the outcome of the scan to be considered valid
ogain	3.0	The gain of weight normalizer function for smoothing the resampling step
resampleThreshold	0.5	Resampling threshold

Table 2: Default Gmapping parameters of figure 2.4

Parameters	Default Value	Description
Group (1): Coordinate system parameters		
base_frame	Base link frame	The coordinate system of the mobile robot platform
odom_frame	Odometry frame	
map_frame	Map frame	
transform_publish_period	0.01	The publishing rate of data among frames
Group (2): OGM parameters		
xmin	-25 m	The initial map dimensions represented as a rectangular region defined by two opposite corners
ymin	-25 m	
xmax	25 m	
ymax	25 m	

delta	0.05 m	The length of one side of the grid cell in meters
occ_thresh	0.5	The threshold used to determine whether a cell is an obstacle or not.
Group (3): Measurement Update Parameters		
linearUpdate	0.3 m	The translational and rotational threshold of measurement update
angularUpdate	1.0 rad	
temporalUpdate	-1.0	The time threshold of the measurement update
Group (4): Motion model parameters		
srr	0.0005	Covariance of sampling motion noise
str	0.0005	
str	0.0005	
stt	0.0005	
Group (5.1): Scan matching and LiDAR parameters		
maxRange	80.0 m	LiDAR maximum distance range
maxUrange	30.0 m	The usable LiDAR range
throttle_scans	1	The number of throttle scans
sigma	0.05	Standard deviation (σ) of the score and likelihood functions of a single beam of the LiDAR
lsigma	0.075	
kernelSize	1	The size of the kernel's search window for matching scans
lstep	0.05 m	Linear and Angular displacement step increment size
astep	0.05 m	
iterations	1	The number of refinement steps used in the scan matches
lskip	0	The number of beams skipped in each scan
Group (5.2): Likelihood sampling parameters		
llsamplerange	0.01	Translation sampling range of scan matching

llsamplestep	0.01	Translation sampling step of scan matching
lasamplerange	0.005	Rotational sampling range of scan matching
lasamplestep	0.005	Rotational sampling step of scan matching
Group (6): PF parameters		
particles	24	The number of particles
map_update_interval	1.0	The map update period
minimumScore	800	The lowest score that a scan-matching algorithm must achieve for the outcome of the scan to be considered valid
ogain	3.0	The gain of weight normalizer function for smoothing the resampling step
resampleThreshold	0.2	Resampling threshold

Table 3: Tuned Gmapping parameters of figures 2.5 and 2.7

